

Riassunto e appunti completi per il corso di Modelli e algoritmi di ottimizzazione

Modelli e algoritmi di ottimizzazione (9 crediti) (Università degli Studi di Bergamo)

Modelli ed Algoritmi di Ottimizzazione

Bombarda Andrea

28 marzo 2017

Indice

I	La ricerca operativa	4
1	Ricerca Operativa	5
1.1	Cosa è la ricerca operativa	5
1.2	Cenni storici	5
1.3	Programmazione matematica	5
1.4	Aspetti principali di un problema di programmazione	5
1.4.1	Variabili decisionali	6
1.4.2	Tipico modello di programmazione lineare	6
2	Metodo risolutivo per i problemi di PL: il metodo del simplesso	8
2.1	Struttura dell'algoritmo del simplesso per problemi di minimo	8
2.1.1	Fase di ammissibilità	8
2.1.2	Fase di ottimalità	9
2.2	Metodo algebrico: algoritmo del simplesso	9
2.3	Variabili libere	10
2.4	Cambio di base ad ogni iterazione	10
2.5	Forma canonica	11
2.6	Problemi di PL con vincoli di uguaglianza o vincoli di $\geq$	11
2.6.1	Problemi con vincoli funzionali generali	11
2.6.1.1	Problema ausiliario in forma standard	12
2.6.1.2	Fase 1: Ricerca di una s.b.a.	12
2.6.1.3	Fase 2: Ricerca di una s.b.a. ottima	13
3	Analisi di sensitività o di post-ottimalità	14
3.1	Determinazione dell'insieme di valori di b_i per i quali la base ottima rimane ammissibile	14
3.2	Determinazione dell'insieme dei valori c_k per i quali la base ottima rimane ottima	14
3.3	Determinazione dell'insieme dei valori c_j per i quali la base ottima rimane ottima	14
3.4	Inserimento di nuove variabili	15
3.5	Inserimento di un nuovo vincolo	15
4	Formalizzazione del metodo del Simplexso	16
5	La teoria della dualità nella programmazione lineare	17
5.1	Primo esempio di problema duale	17
5.2	Generalizzazione del problema duale	18
5.3	Relazione tra il valore di ω in una soluzione del primale ed il valore di φ_D in una soluzione del duale	19
5.3.1	Variabili complementari	19
5.4	Relazione tra il valore di ω in una soluzione ammissibile del primale ed il valore di φ_D in una soluzione ammissibile del duale	20
5.4.1	Teorema di dualità debole	20
5.4.2	Conseguenze del teorema della dualità debole	20
5.4.2.1	Corollario 1	20
5.4.2.2	Corollario 2	20
5.5	Basi complementari	20
5.6	Relazioni tra soluzioni del problema primale e del problema duale	21
5.7	Tableau ridotti	21
5.8	Sottomatrici quadrate complementari	21
5.8.1	Teorema	21

5.8.2	Osservazione	22
5.9	Teorema degli scarti complementari	22
6	Programmazione lineare intera	23
6.1	Problemi di programmazione lineare intera	23
6.2	Metodo di Branch & Bound	24
6.3	Metodo dei piani di taglio o di Gomory	25
6.3.1	Costi fissi di attivazione	25
6.3.2	Lotti minimi di produzione	26
6.3.3	Vincoli disgiuntivi	27
6.3.4	Problema di selezione di sottoinsiemi	27
6.3.5	Localizzazione di impianti	27
6.3.6	Metodo dei piani di taglio di Gomory	28
7	Ottimizzazione su grafi	30
7.1	I grafi, le loro applicazioni, definizioni e rappresentazioni	30
7.1.1	Grafi non orientati	30
7.1.2	Grafi orientati	30
7.1.3	Definizioni comuni	31
7.1.4	Rappresentazione di grafi tramite matrici	31
7.2	Il problema della raggiungibilità e algoritmo di visita	32
7.3	Alberi di supporto di costo minimo	33
7.3.1	Metodo di Kruskal	34
7.3.2	Algoritmo di Prim	34
7.3.3	Lato di diminuzione	35
7.3.3.1	Procedura per la verifica di ottimalità	35
7.3.4	Applicazione del calcolo di alberi di supporto: diffusione ottimale di un messaggio	36
7.4	Cammino di costo minimo	36
7.4.1	Algoritmo di Dijkstra	36
7.4.2	Algoritmo delle etichette	38
7.4.3	Algoritmo di Floyd-Warshall	39
7.5	Esempio di calcolo dei cammini minimi: Ispezioni su una linea di produzione	41
8	Problema del flusso massimo	43
8.1	Cosa è il flusso	43
8.2	Formulazione PL del problema di massimo flusso	43
8.2.1	Esempio di modello per problema del massimo flusso	44
8.3	Definizioni	44
8.4	Algoritmo di Ford-Fulkerson	45
8.4.1	Teorema del massimo flusso-minimo taglio	47
8.4.2	Determinazione del cammino aumentante	47
8.5	Esempio pratico di massimo flusso: abbinamento di cardinalità massima con vincoli di compatibilità	47
8.6	Problema del flusso a costo minimo	48
8.6.1	Eliminazione dei cicli di costo negativo	49
8.6.2	Formalizzazione dell'algoritmo di eliminazione dei cicli di costo negativo	50
8.7	Problema di dimensionamento dei lotti (Lot Sizing)	51
8.8	Piano di carico di un aereo	53
9	Il problema dello zaino	54
9.1	Formulazioni alternative del problema	54
9.2	Condizioni sui dati	54
9.3	Formulazione come problema di ottimizzazione combinatoria	54
9.3.1	Formulazione come PLI	55
9.3.1.1	Dimostrazione di ottimalità della soluzione	55
10	Il problema del commesso viaggiatore (TSP)	57
10.1	Tipiche applicazioni di TSP	57
10.2	Problema visto come su un grafo	57
10.2.1	Formulazione PLI di TSP	58
10.2.2	Formulazione come ottimizzazione combinatoria di TSP	58
10.2.2.1	Algoritmo di Branch and Bound per TSP simmetrico	59

10.2.2.2	Algoritmo GREEDY per il problema del commesso viaggiatore	59
10.2.2.3	Euristica alternativa al Nearest Neighbour: Twice Around MST	60
11	Problemi basati su variabili intere binarie	62
11.1	Ordinamento di lavori su macchine con minimizzazione del numero di macchine (MCMS)	62
11.1.1	Formulazione PLI del problema MCMS	62
11.1.2	Algoritmi greedy per MCMS	63
11.2	Ordinamento di lavori su macchine con minimizzazione del tempo di completamento (MMMS) .	64
11.2.1	Formulazione PLI del problema MMMS	64
11.2.2	Algoritmi greedy di List Scheduling	65
11.2.2.1	Analisi dell'errore relativo di LS	65
11.2.2.2	Particolarità dell'ordinamento SPT	66
11.3	Problema di Graph Colouring	66
11.3.1	Formulazione PLI del problema	66
11.4	Problemi di selezione di sottoinsiemi	67
11.4.1	Rappresentazione della famiglia F di sottoinsiemi di E	67
11.4.2	Come individuare una sottofamiglia $D \subseteq F$	67
11.4.2.1	Problema di copertura	67
11.4.2.2	Problema di partizione	68
11.4.2.3	Problema di riempimento	69
11.4.2.4	Famiglia di algoritmi greedy per PC	69
12	Weighted vertex cover (WVC)	70
12.1	Algoritmo greedy per WVC	70
12.1.1	Algoritmo «Greedy-WVC»	71
12.1.1.1	Valutazione a priori dell'efficacia dell'algoritmo Greedy-WVC	72
13	Smart Grids	73
13.1	Differenti approcci	73
13.2	Progetto di ricerca	73
13.2.1	Modello non lineare	74
13.2.1.1	Flussi di corrente	75
13.2.1.2	Vincoli sulle batterie	76
13.2.1.3	Controllori locali	76
13.2.1.4	Funzione obiettivo	76
13.2.1.5	Modello non lineare	77
13.3	Conclusioni	78

Parte I

La ricerca operativa

Capitolo 1

Ricerca Operativa

1.1 Cosa è la ricerca operativa

La **ricerca operativa** si occupa dello studio e della messa a punto di **metodi matematici** per la soluzione di **problemi di decisione**.

Parliamo di **problemi di decisione** in merito a problemi per i quali:

- La soluzione deve soddisfare un insieme di predefinite condizioni.
- Esistono diverse soluzioni alternative che soddisfano le condizioni richieste (**soluzioni ammissibili**).
- E' definito un criterio di valutazione della singola soluzione ammissibile, che introduce un ordinamento di preferenza nell'insieme delle soluzioni ammissibili.
- Si determina una soluzione **ottima**.

1.2 Cenni storici

La ricerca operativa venne storicamente utilizzata, durante la seconda guerra mondiale, per *pianificare azioni militari, pianificare la rotazione delle colture, fare routing tra i vari porti e pianificare il flusso dei beni tra i settori dell'economia*. Nel 1947, **tutti questi aspetti vennero formalizzati nella programmazione lineare** che fornisce

- lo strumento della modellazione matematica, in modo da poter formulare i problemi con un linguaggio matematico.
- Un **metodo di calcolo**, cioè l'**algoritmo del simplesso**, per risolvere i problemi in modo efficiente.

Il maggior sviluppo della programmazione lineare si deve anche all'introduzione dei calcolatori elettronici digitali che consentono di risolvere in modo più semplice i problemi, che talvolta sono molto complicati per un'eventuale soluzione «a mano».

1.3 Programmazione matematica

Con il termine **programmazione matematica** intendiamo l'uso di modelli matematici per risolvere problemi relativi alla programmazione lineare. In particolare si utilizzano metodi *computazionali* per scegliere la migliore delle soluzioni quando il problema da risolvere presenta:

- Una struttura esprimibile (o approssimabile) con un **modello matematico**, che rappresenta l'insieme di tutte le possibili soluzioni del problema.
- Un obiettivo che può essere quantificato per ognuna delle possibili soluzioni del problema.

1.4 Aspetti principali di un problema di programmazione

Tipicamente, un problema di programmazione presenta i seguenti aspetti principali:

- Presenza di un certo set di **risorse disponibili**. Queste daranno origine ai **vincoli funzionali** del problema.

- Presenza di un certo set di **prodotti** sui quali devono essere **effettuate delle decisioni**.
- Presenza di un certo **criterio di valutazione**, il quale deve essere usato per **scegliere quale, tra tutte le soluzioni ammissibili, è la soluzione ottima**.

La determinazione della soluzione «migliore» avviene grazie alla rappresentazione del problema tramite un **modello di programmazione matematica** sul quale viene calcolata la soluzione ottima tramite un **algoritmo di ottimizzazione**.

- Presenza di una o più **SOLUZIONI AMMISSIBILI**, cioè soluzioni che **soddisfano tutti i vincoli** imposti dal problema.
- Le *decisioni* sono rappresentate da **incognite**, chiamate **variabili decisionali** x_i . Generalmente le variabili decisionali sono sottoposte a **vincoli di non negatività**, del tipo $x_i \geq 0$.

Le variabili decisionali servono anche in quanto in funzione ad esse vengono espressi in **vincoli** e la **funzione obbiettivo**.

VAR DECISIONALI $\rightarrow x_i \geq 0$

1.4.1 Variabili decisionali

Generalmente, nei vincoli e nelle funzioni obbiettivo, si utilizzano **funzioni lineari** delle variabili decisionali. Questo però è possibile solamente se valgono le ipotesi di **PROPORZIONALITÀ** e **ADDITIVITÀ**:

- **PROPORZIONALITÀ**: il contributo di ciascuna attività all'uso della risorsa i deve essere proporzionale al livello x_j dell'attività. Cioè vale $a_{i,j}x_j$. Inoltre il contributo di ciascuna attività al profitto deve essere proporzionale al livello x_j dell'attività, cioè vale p_jx_j .

Un esempio di violazione della proporzionalità è il caso in cui si ha una pendenza della funzione profitto che cresce al crescere del valore della variabile decisionale, oppure decresce.

- **ADDITIVITÀ**: l'uso della risorsa i deve essere la **somma degli utilizzi individuali delle rispettive attività**. Questa è la **condizione che ci permette di definire i vincoli**.

Inoltre, il profitto totale deve coincidere con la somma dei profitti individuali delle rispettive attività. Questa è la condizione che ci permette di definire la **funzione obbiettivo** $\omega = \dots$

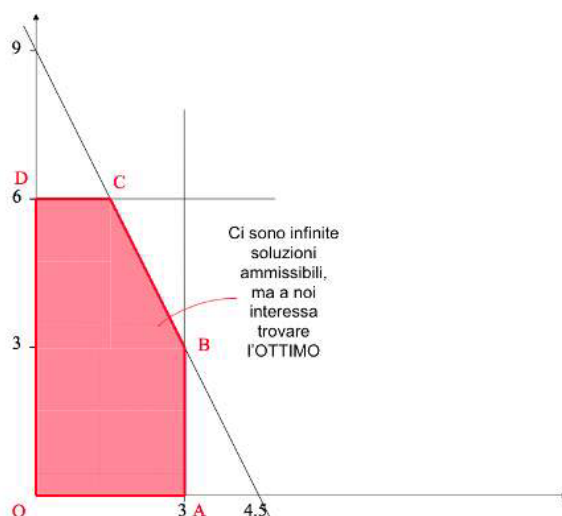
1.4.2 Tipico modello di programmazione lineare

Un tipico modello di programmazione lineare è:

$$\begin{cases} \max \omega = 4x_1 + 5x_2 & \text{funzione obbiettivo: definisce la variabile che deve essere massimizzata} \\ \text{s.a. } x_1 \leq 3 & \\ 2x_2 \leq 12 & \text{vincoli funzionali} \\ 4x_1 + 2x_2 \leq 18 & \\ x_1 \geq 0, x_2 \geq 0 & \text{vincolo di non negatività} \end{cases}$$

Es. PROFITTO.

Un generico modello, abbiamo visto che ha delle **soluzioni ammissibili**. In particolare l'insieme delle soluzioni ammissibili viene detto **regione ammissibile** e, nel caso di un problema in **due variabili decisionali**, questa regione è un piano:



- Se si analizza la forma grafica del problema, considerando la funzione obiettivo $\omega = \alpha x_1 + \beta x_2$, si può disegnare il fascio di rette $x_2 = -\frac{\alpha}{\beta}x_1 + \frac{\omega}{\beta}$ e, facendo variare il valore di ω si fa «salire» la retta fino all'ultimo punto della regione ammissibile. Questo punto sarà il *massimo* del problema (in questo caso sarebbe il punto C).
- Se si avessero più punti (*ad esempio il segmento CB*) che appartengono alla retta della funzione obiettivo, allora ci sono **soluzioni ottime alternative**.
- A volte la regione ammissibile potrebbe essere *illimitata*, quindi in un problema di massimo, il valore della funzione obiettivo potrebbe essere preso grande a piacere.
- Se la regione ammissibile è *vuota*, allora il problema non ha soluzione.

Definiamo **soluzioni di base** tutte le intersezioni di frontiere di vincoli. In particolare, le intersezioni di frontiere di vincoli che appartengono alla regione ammissibile (*cioè sono i vertici della regione ammissibile*) vengono dette **soluzioni di base ammissibili**.

DEF : SOLUZIONE DI BASE

Capitolo 2

Metodo risolutivo per i problemi di PL: il metodo del simplesso

Il **metodo del simplesso** è una intersezione di iperpiani che consiste in due parti:

1. Se la regione ammissibile non è vuota, essa contiene almeno una soluzione di base ammissibile.
In particolare, si determina che la regione ammissibile è vuota dimostrando che non esistono soluzioni di base ammissibili, cioè che tutte le soluzioni di base sono non ammissibili. Al contrario, si dimostra che la regione ammissibile non è vuota determinando una soluzione di base ammissibile.
2. Se la funzione obiettivo è limitata superiormente (*in un problema di massimo*) o inferiormente (*in un problema di minimo*), allora almeno una delle soluzioni ottime è una soluzione di base ammissibile.
In particolare, questo ci dice che per determinare se il problema abbia soluzione ottima finita o illimitata, è sufficiente prendere in considerazione le soluzioni di base ammissibili.

2.1 Struttura dell'algoritmo del simplesso per problemi di minimo

L'algoritmo del simplesso è definito per **problemi di minimo**. Nel caso in cui si abbiano problemi di massimo si fa il duale: vorremmo massimizzare una funzione, quindi andiamo semplicemente ad invertire i segni nella funzione obiettivo, considerandola poi come una funzione di minimo.

Il simplesso è composto da due fasi principali:

- FASE DI AMMISSIBILITÀ
- FASE DI OTTIMALITÀ

2.1.1 Fase di ammissibilità

Durante la fase di ammissibilità si definisce una **misura di inammissibilità** associata ad ogni soluzione di base, secondo la seguente regola:

- SE LA SOLUZIONE DI BASE È AMMISSIBILE, la sua misura di inammissibilità è **nulla**.
- SE LA SOLUZIONE DI BASE NON È AMMISSIBILE, la sua misura di inammissibilità è **positiva**.

A questo punto si esegue il seguente processo iterativo:

- **INIZIALIZZAZIONE:** Si denota con B_1 la soluzione di base individuata dall'intersezione degli iperpiani $x_j = 0$ con $j = 1, \dots, n$, cioè l'origine di $\mathbb{R}^n$ con tutte le variabili decisionali nulle.
- **PASSO ITERATIVO:** Per $k = 1, \dots, K_{max}$, cioè per tutte le possibili intersezioni delle rette, si controlla l'ammissibilità di B_k :
 - Se la misura di inammissibilità in B_k è nulla, allora si ferma l'iterazione, in quanto la soluzione di base B_k è ammissibile.
 - Altrimenti, si controlla se sia possibile determinare una soluzione di base B_{k+1} adiacente (cioè sullo stesso lato) a B_k con misura di inammissibilità minore di quella associata a B_k :
 - * Se questo è possibile, si calcola tale base B_{k+1} .
 - * In caso contrario, abbiamo dimostrato che non esistono soluzioni di base ammissibili e quindi che non esistono soluzioni ammissibili. Abbiamo, cioè, una *regione ammissibile vuota*.

2.1.2 Fase di ottimalità

Una volta arrivati a questa fase *si è certi che la regione ammissibile non sia vuota*. Anche in questa fase, il processo da seguire è di tipo *iterativo*:

- INIZIALIZZAZIONE: Si denota con V_1 la soluzione di base ammissibile determinata durante la fase di ammissibilità
- PASSO ITERATIVO: Per $k = 1, \dots, K_{max}$
 - Se V_k è una soluzione di base ammissibile ottima, si ferma l'algoritmo, in quanto si è arrivati all'ottimo.
 - Altrimenti, si controlla se sia possibile determinare una soluzione di base ammissibile V_{k+1} adiacente a V_k e con valore dell'obiettivo minore del valore di quello ottenuto con V_k .
 - * Se questo è possibile, si calcola tale soluzione di base ammissibile V_{k+1} adiacente a V_k .
 - * In caso contrario, la funzione obiettivo è inferiormente illimitata.

2.2 Metodo algebrico: algoritmo del simpleso

Per utilizzare il metodo del simpleso, *i vincoli funzionali in forma di disequazione vengono riformulati come equazioni*.

Ad esempio, il vincolo:

$x_1 \leq 3$

Viene portato in forma di equazione aggiungendo una **variabile scarto**

DA
DISEQ. A EQ.

$x_1 + s_1 = 3$

Alla quale viene aggiunto il **vincolo di non negatività**

$s_1 \geq 0$

Questa operazione viene **ripetuta per tutti i vincoli funzionali**, ottenendo un sistema di n equazioni in m incognite (dove n è il numero di disequazioni, ed m è la somma del numero di variabili decisionali e delle variabili scarto).

A questo punto è possibile definire una **soluzione ammissibile del sistema dei vincoli funzionali** come una soluzione del problema di PL in cui tutte le sue componenti (*variabili decisionali e di scarto*) sono non negative.

Quando si è portato il sistema in **forma di minimo** (se inizialmente era di massimo) e si sono **trasformate** tutte le **disequazioni in equazioni**, allora è possibile rappresentare il **tableau** della situazione iniziale:

x_1	x_2	s_1	s_2	s_3	φ	
1	0	1	0	0	0	3
0	2	0	1	0	0	12
4	2	0	0	1	0	18
4	5	0	0	0	1	0

Colonne delle variabili decisionali Colonne delle variabili scarto

Nell'esempio dell'immagine precedente, la soluzione di base corrispondente è

x_1	x_2	s_1	s_2	s_3	φ
0	0	3	12	18	0

(poniamo $x_j = 0$)

Introduciamo il concetto di **test di ottimalità**:

La soluzione di base ammissibile è ottima se non esistono soluzioni ammissibili con valore della funzione obbiettivo più piccolo.

In questo esempio, la soluzione trovata non è ottima poiché i coefficienti di costo relativo di x_1, x_2 sono maggiori di zero, quindi possiamo far scendere ancora di più il valore della funzione obbiettivo.

Quindi si deve ora determinare una soluzione di base ammissibile adiacente con valore della funzione obbiettivo più piccolo. Questo provocherà ovviamente una variazione delle variabili che precedentemente erano in base (quelle con coefficiente di costo relativo - ultima riga del tableau - nullo). Generalmente, ad ogni iterazione, si porta in base la variabile che ha il costo relativo maggiore fra tutte.

Si procede, quindi, in questo modo fino a quando si avranno tutti i coefficienti di costo relativo minori di zero, cioè si arriverà alla **soluzione ottima**.

2.3 Variabili libere

Non per forza si devono avere variabili decisionali con un vincolo di non negatività. Infatti, si parla di **VARIABILI LIBERE** in riferimento a **variabili non vincolate in segno**.

Non avere un vincolo di positività su una variabile (ad esempio $x_1 \geq 0$), significa che l'asse relativo a tale variabile (nell'esempio, $x_1 = 0$) non è frontiera di vincolo per il problema.

Ad esempio, potremmo avere un problema del tipo

$$\begin{cases} \min \varphi = 3x_1 - 2x_2 \\ \text{s.a.} & -2x_1 + x_2 \leq 13 \\ & x_1 + x_2 \leq 10 \\ & x_2 \geq 0 \end{cases}$$

In questo caso abbiamo x_1 variabile libera. E' possibile esprimere il problema in forma «standard» rappresentando la variabile libera come la differenza di variabili non negative ($x_1 = x_1^+ - x_1^-$), cioè

$$\begin{cases} \min \varphi = 3(x_1^+ - x_1^-) - 2x_2 \\ \text{s.a.} & -2(x_1^+ - x_1^-) + x_2 \leq 13 \\ & (x_1^+ - x_1^-) + x_2 \leq 10 \\ & x_2, x_1^+, x_1^- \geq 0 \end{cases}$$

↓ ↓
 $x_1^+ > 0$ $x_1^- > 0$

In questo modo possiamo trovarci in 4 casi:

- $x_1^+ = 0, x_1^- = 0 \rightarrow x_1 = 0$
- $x_1^+ = 0, x_1^- > 0 \rightarrow x_1 < 0$
- $x_1^+ > 0, x_1^- = 0 \rightarrow x_1 > 0$
- $x_1^+ > 0, x_1^- > 0$. In realtà questo caso non si può avere, in quanto significherebbe avere le variabili linearmente dipendenti.

A questo punto, dopo aver portato le disequazioni sotto forma di equazioni, si ottiene un sistema dei vincoli che può essere schematizzato come segue

$$\begin{cases} \sum_{j=1}^n a_{i,j} x_j + s_i = b_i \\ s_i \geq 0 \end{cases}$$

2.4 Cambio di base ad ogni iterazione

Ad ogni iterazione, quando si hanno dei coefficienti di costo relativo positivi, bisogna fare un cambio di base, cioè portare in base la variabile che ha il costo relativo maggiore tra tutti.

Per capire le operazioni da fare si seguono questi passi:

- Si calcolano i rapporti tra tutti gli elementi della colonna $\underline{b}$ e quelli della colonna della variabile che si vuole portare in base.

- L'elemento positivo, della colonna della variabile da portare in base, che genera il rapporto inferiore viene detto **elemento pivot**.

Il nuovo tableau viene quindi scritto in base alle seguenti regole:

$$\text{nuova riga del pivot} = \frac{\text{riga del pivot vecchia}}{PIVOT}$$

$$\text{altra riga nuova} = (\text{altra riga vecchia}) - (\text{elemento vecchio colonna del pivot}) \cdot (\text{nuova riga del pivot})$$

2.5 Forma canonica

E' sempre possibile esprimere le equazioni della forma standard del problema in modo da avere le variabili in base e la funzione obiettivo in funzione delle variabili non di base:

$$\begin{cases} \underline{s} = \underline{b} - A\underline{x} \\ \varphi = \gamma - \underline{c}^T \underline{x} \end{cases}$$

Il tableau, cioè la matrice dei coefficienti del sistema aumentato con colonne nell'ordinamento originario, associato alla forma canonica è:

$$\mathbf{T} = \begin{array}{c} \underline{w}_N \quad \underline{w}_B \quad \varphi \\ \left[\begin{array}{ccc|c} \mathbf{A} & \mathbf{I} & \underline{\mathbf{0}} & \underline{\mathbf{b}} \\ -\underline{\mathbf{c}}^T & \underline{\mathbf{0}}^T & 1 & \gamma \end{array} \right] \end{array}$$

2.6 Problemi di PL con vincoli di uguaglianza o vincoli di $\geq$

Nel caso di problemi di PL in cui si hanno vincoli di uguaglianza o vincoli di $\geq$ è necessaria una fase preliminare, detta **Fase 1** o **fase di ammissibilità**, per stabilire se la regione ammissibile sia vuota oppure abbia qualche soluzione ammissibile.

Il **teorema fondamentale della programmazione lineare** afferma che in una regione ammissibile non vuota è contenuta almeno una soluzione di base ammissibile. Di conseguenza, la cosiddetta **Fase 1** è in grado:

- O di trovare una soluzione di base ammissibile, quindi dimostrare che la regione ammissibile non è vuota.
- O di mostrare che non esiste alcuna soluzione di base ammissibile, cioè dimostrare che la regione ammissibile è vuota.

2.6.1 Problemi con vincoli funzionali generali

Supponiamo di avere un problema qualsiasi, con anche vincoli di uguaglianza o vincoli di $\geq$. A meno di avere, a sinistra della disequazione un valore nullo (dato che i termini noti devono essere sempre tutti non negativi), quasi sicuramente l'origine non soddisferà tali vincoli.

Bisogna quindi capire come fare a stabilire se tale problema ha soluzioni ammissibili.

Supponiamo di avere il problema

$$\begin{cases} \min \varphi = 4x_1 + 5x_2 \\ \text{s.a. } 3x_1 + x_2 \leq 27 \\ 3x_1 + 2x_2 \geq 30 \\ x_1 + x_2 = 12 \\ x_1, x_2 \geq 0 \end{cases}$$

Scriviamo i tre vincoli funzionali nella forma standard

Problema originario (vincoli come equazioni)	Problema ausiliario (con l'origine ammissibile)	Problema ausiliario in forma std
$3x_1 + x_2 + s_1 = 27, s_1 \geq 0$	$3x_1 + x_2 + s_1 = 27, s_1 \geq 0$	$3x_1 + x_2 + s_1 = 27, s_1 \geq 0$
$3x_1 + 2x_2 - \bar{s}_2 = 30, \bar{s}_2 \geq 0$	$3x_1 + 2x_2 + u_2 = 30$ In questo caso si tiene u_2 libera.	$3x_1 + 2x_2 + u_2^+ - u_2^- = 30, u_2^+ \geq 0, u_2^- \geq 0$ In questo caso, dato che u_2 era una variabile libera, la si esprime come differenza di due variabili vincolate. Per semplicità di notazione utilizziamo $t_2 = u_2^+$ e $\bar{s}_2 = u_2^-$
$x_1 + x_2 = 12$	$x_1 + x_2 + t_3 = 12, t_3 \geq 0$	$x_1 + x_2 + t_3 = 12, t_3 \geq 0$

Come si nota dalla tabella, passando al problema ausiliario il primo vincolo non richiede modifiche, in quanto contiene già l'origine.

Con la costruzione del problema ausiliario si ha una **regione ammissibile modificata** ma mantiene le stesse rette «di supporto», quindi ammette le stesse soluzioni di base anche se non tutte sono ammissibili per il problema di base (*vedi l'origine, nella maggior parte dei casi*).

Così come nel problema classico del simpleso, la funzione obbiettivo ausiliaria assocerà ad ogni vertice della regione ammissibile una *misura di inammissibilità* che sarà nulla per i punti ammissibili anche per il problema originale, e positiva per tutti gli altri punti.

L'obbiettivo della **Fase 1** è minimizzare questa misura di inammissibilità!

2.6.1.1 Problema ausiliario in forma standard

Come detto in precedenza, nella ricerca di una prima soluzione ammissibile, bisogna modificare la funzione obbiettivo considerata. In particolare, nell'esempio fatto in precedenza, avremmo:

$$\begin{cases} \min \psi = t_2 + t_3 & \text{somma delle due variabili che danno inammissibilità} \\ \text{s.a. } 3x_1 + x_2 + s_1 = 27 \\ x_1 + 2x_2 + t_2 - \bar{s}_2 = 30 & \text{la variabile temporanea } t_2 \text{ serve per includere l'origine} \\ x_1 + x_2 + t_3 = 12 & \text{la variabile temporanea } t_3 \text{ serve per includere l'origine} \\ x_1, x_2, s_1, \bar{s}_2, t_2, t_3 \geq 0 \end{cases}$$

In base al valore minimizzato di ψ si ha:

- Se $\psi > 0$ la soluzione di base ammissibile del problema ausiliario non è ammissibile per il problema originario. Avendo dimostrato che non esistono soluzioni di base ammissibili per il problema ausiliario, allora non ne esistono nemmeno per quello originario.
- Se $\psi = 0$ la soluzione di base ammissibile del problema ausiliario è ammissibile per il problema originario. La soluzione di base ammissibile che si è trovata in questo modo viene usata come s.b.a. iniziale nella fase di ricerca della soluzione di base ammissibile ottima del problema originario.

Il primo di questi due punti è confermato dal **teorema fondamentale della PL**:

Se esiste una soluzione ammissibile, allora esiste una soluzione di base ammissibile,
 Letto al contrario, questo teorema ci dice che se non esiste una soluzione di base ammissibile, allora non esistono soluzioni ammissibili, cioè *la regione ammissibile del problema originario è vuota.*

2.6.1.2 Fase 1: Ricerca di una s.b.a. (Soluzione base ammissibile)

Una volta costruito il problema ausiliario, bisogna risolverlo per trovare l'ammissibilità:

- Partiamo dalla soluzione di base corrispondente all'origine ($x_1 = x_2 = \dots = 0$).
- Esprimiamo l'ultima riga del tableau, quella della funzione obbiettivo, in funzione delle variabili non di base.
- Risolviamo fino a quando possibile, tentando di arrivare all'ottimo $\psi = 0$.

2.6.1.3 Fase 2: Ricerca di una s.b.a. ottima

Una volta trovata una s.b.a., se esiste, bisogna procedere a determinare la soluzione ottima. Il procedimento da seguire è:

- Si rimuovono dal tableau finale della fase precedete le colonne relative alle variabili temporanee.
- Si sostituisce alla funzione obiettivo ausiliaria, la funzione obiettivo originaria, esprimendola in funzione delle variabili non di base.

Es.

$$\begin{cases} \max z = 3x_1 + 2x_2 \\ x_1 + x_2 \leq 4 \\ 2x_1 + x_2 \leq 6 \\ x_1, x_2 \geq 0 \end{cases}$$

FORMA CANONICA :

$$\begin{aligned} x_1 + x_2 + s_1 &= 4 \\ 2x_1 + x_2 + s_2 &= 6 \end{aligned}$$

Capitolo 3

Analisi di sensitività o di post-ottimalità

L'**analisi di sensitività** è basata sulle relazioni che esprimono, in funzione dei dati iniziali (*o parametri del modello*),

- i valori delle variabili di base: $\underline{w}_B = B^{-1}\underline{b} - B^{-1}N\underline{w}_N$;
- i coefficienti di costo relativo delle variabili non di base: $\underline{r}_N^T = \underline{c}_B^T B^{-1}N - \underline{c}_N^T$;
- il valore della variabile obbiettivo: $\varphi = \underline{c}_B^T \underline{w}_B + \underline{c}_N^T \underline{w}_N + \gamma = \underline{c}_B^T (B^{-1}\underline{b} - B^{-1}N\underline{w}_N) + \underline{c}_N^T \underline{w}_N + \gamma = \underline{c}_B^T B^{-1}\underline{b} + \gamma - (\underline{c}_B^T B^{-1}N - \underline{c}_N^T)\underline{w}_N$

3.1 Determinazione dell'insieme di valori di b_i per i quali la base ottima rimane ammissibile

Date le espressioni appena presentate, supponiamo una variazione del valore $\tilde{b}_i = b_i + \delta$, che può essere anche rappresentata con $\tilde{\underline{b}} = \underline{b} + \underline{e}_i \delta$.

Queste variazioni impattano sul valore delle variabili di base:

$$\tilde{\underline{w}}_B = B^{-1}\tilde{\underline{b}} = B^{-1}(\underline{b} + \underline{e}_i \delta) = \underline{w}_B + B^{-1}\underline{e}_i \delta$$

Di conseguenza, quindi, si avrà una variazione anche del valore della variabile obbiettivo:

$$\tilde{\varphi} = \underline{c}_B^T B^{-1}\tilde{\underline{b}} + \gamma = \underline{c}_B^T B^{-1}(\underline{b} + \underline{e}_i \delta) + \gamma = \varphi + \underline{c}_B^T B^{-1}\underline{e}_i \delta$$

3.2 Determinazione dell'insieme dei valori c_k per i quali la base ottima rimane ottima

Indichiamo con c_k il coefficiente nella funzione obbiettivo della variabile x_k non di base nell'ottimo.

Allora, avremo una variazione $\tilde{c}_k = c_k + \delta$. Allora, avremo una variazione del vettore dei costi relativi delle variabili non di base:

$$\tilde{\underline{r}}_N^T = \underline{c}_B^T B^{-1}N - \tilde{\underline{c}}_N^T$$

Guardando la variazione del coefficiente di costo relativo della singola variabile si ha:

$$\tilde{r}_k = \underline{c}_B^T B^{-1}\underline{f}_k - (c_k + \delta) = r_k - \delta$$

Come si vede da queste relazioni, una variazione del coefficiente di costo di una variabile non di base, determina **solamente** una variazione del suo coefficiente di costo relativo.

3.3 Determinazione dell'insieme dei valori c_j per i quali la base ottima rimane ottima

Indichiamo con c_j il coefficiente nella funzione obbiettivo della variabile x_j di base nell'ottimo.

Allora, avremo una variazione $\tilde{\underline{c}}_B = \underline{c}_B + \underline{e}_j \delta$. Allora, avremo una variazione del vettore dei costi relativi delle variabili non di base:

$$\tilde{\underline{r}}_N^T = \tilde{\underline{c}}_B^T B^{-1}N - \underline{c}_N^T = (\underline{c}_B^T + \underline{e}_j^T \delta) B^{-1}N - \underline{c}_N^T = \underline{r}_N^T + \delta \underline{e}_j^T B^{-1}N$$

dove $\underline{c}_i^T B^{-1} N$ è il vettore riga costituito dagli elementi della i -esima riga del tableau corrispondenti alle variabili non di base.

Come si vede da queste relazioni, una variazione del coefficiente di costo di una variabile di base, determina una variazione dei coefficienti di costo relativo di tutte le variabili non di base.

3.4 Inserimento di nuove variabili

Talvolta si può voler analizzare una situazione in cui si *inserisce una nuova variabile*. Una volta noti i coefficienti di utilizzo $\tilde{a}_i$ della nuova variabile, si deve generalmente valutare in quali condizioni si ha la convenienza nell'introduzione di questo nuovo prodotto nella produzione.

Quello che deve essere fatto è calcolare il coefficiente di costo relativo del prodotto stesso, ponendolo maggiore di zero (*si vuole quindi perdere l'ottimalità*):

$$r_i = \underline{c}_B^T B^{-1} \tilde{a}_i - \underline{c}_i > 0$$

dove $\underline{c}_B^T B^{-1}$ è l'ultima riga del tableau ottimo, in corrispondenza delle variabili non di base.

3.5 Inserimento di un nuovo vincolo

Quando si ha *l'aggiunta di un nuovo vincolo*, la soluzione può solamente peggiorare o al più rimanere uguale. Per verificare se la soluzione ottima trovata senza il nuovo vincolo è compatibile con il vincolo che si vuole aggiungere, ***si sostituisce la soluzione ottima nel nuovo vincolo*** e si vede se esso rimane soddisfatto.

Altrimenti, è necessario ripetere il calcolo del tableau, con l'inserimento del nuovo vincolo.

Capitolo 4

Formalizzazione del metodo del Simplexso

L'algoritmo del **Simplexso** ha il compito di cercare delle soluzioni ottime all'interno del sottoinsieme delle soluzioni si base ammissibili.

Tale algoritmo consta di due fasi:

- **FASE 1:** Si ha lo studio dell'esistenza di soluzioni ammissibili.

Il caso affermativo è certificato dalla determinazione di una soluzione di base ammissibile, che verrà poi utilizzata nella *Fase 2* come punto iniziale da cui calcolare la soluzione ottima.

Questa fase non è necessaria quando in F esiste una sottomatrice identità cui è associata una soluzione di base ammissibile. Ciò accade sicuramente quando tutti i vincoli funzionali sono vincoli di $\leq$, con $\underline{b} > \underline{0}$, in quanto una soluzione ammissibile è sicuramente l'origine degli assi di $\mathbb{R}^n$.

- **FASE 2:** Si ha la costruzione, a partire dalla soluzione di base ammissibile iniziale, di una sequenza di soluzioni di base ammissibili, controllando l'ottimalità di ogni nuovo elemento aggiunto alla sequenza e interrompendo questo calcolo quando si determina
 - che la soluzione di base ammissibile trovata attualmente è ottima.
 - che la funzione obbiettivo è inferiormente illimitata.

Capitolo 5

La teoria della dualità nella programmazione lineare

La **teoria della dualità** nella programmazione lineare stabilisce un legame logico tra due problemi di PL, che costituiscono una *coppia primale-duale*. La teoria della dualità offre contributi in merito ad:

- Approfondimenti nell'analisi del problema di partenza (*ad esempio il problema duale potrebbe fornire un'interpretazione economica del primale*).
- Avanzamenti sia di tipo algoritmico che modellistico. In particolare sarà possibile analizzare problemi in più rispetto a quelli analizzabili solamente in modo primale (*ad esempio programmazione intera o booleana*).

5.1 Primo esempio di problema duale

Il primo esempio di problema duale è il cosiddetto *problema del produttore o di allocazione delle risorse*.

«Un produttore dispone di m risorse che vengono impiegate per la produzione di n prodotti.»

I dati di questo problema sono:

- b_i , $1 \leq i \leq m$ cioè la quantità di risorsa i disponibile.
- $a_{i,j}$, $1 \leq i \leq m$ e $1 \leq j \leq n$ cioè la quantità di risorsa i necessaria per produrre un'unità di prodotto j .
- p_j , $1 \leq j \leq n$ cioè il profitto derivante dalla vendita di una unità di prodotto j .

Il produttore vuole determinare i livelli $x_j \geq 0$ di produzione degli n prodotti, compatibili con le limitate risorse disponibili, cioè che rispettano i vincoli

$$\sum_{j=1}^n a_{i,j} x_j \leq b_i$$

e che massimizzano il profitto derivante dalla vendita dei prodotti

$$\max \omega = \sum_{j=1}^n p_j x_j$$

Il problema del produttore è quindi un problema di massimo, con vincoli di $\leq$.

Il problema duale a quello appena presentato è il cosiddetto *problema del compratore o di acquisto delle risorse*.

«Un potenziale compratore, interessato ad acquistare le risorse b_i , $1 \leq i \leq m$, deve determinare, per ogni risorsa i il prezzo di acquisto $y_i \geq 0$ da proporre al proprietario/produttore»

Per determinare la convenienza o meno dell'offerta di acquisto ricevuta, il produttore dovrà confrontare, per ogni prodotto j :

- Il ricavo dalla vendite delle risorse $a_{i,j}$, $1 \leq i \leq m$ necessarie a produrre un'unità di prodotto j : se si danno le risorse a qualcun'altro, il guadagno deve essere almeno tanto quanto quello che si ha utilizzando tali risorse nella produzione.
- Il profitto unitario p_j del prodotto j .

Il produttore troverà conveniente vendere le risorse al compratore, anziché impiegarle per la produzione, se i prezzi unitari y_i offerti per le risorse sono tali che per ogni prodotto j il ricavo $\sum_{i=1}^m a_{ij}y_i$ dalla vendita delle risorse necessarie a produrre un'unità di prodotto j è maggiore o uguale al profitto unitario p_j del prodotto j

$$\sum_{i=1}^m a_{ij}y_i \geq p_j$$

In questo modo si ricavano una serie di $y_i \geq 0$ che soddisfano questo vincolo e per i quali si deve minimizzare la funzione obbiettivo, cioè devo tentare (*come compratore*) di pagare il meno possibile:

$$\min \varphi_D = \sum_{i=1}^m b_i y_i$$

Il problema del compratore è quindi un problema di minimo, con vincoli di $\geq$.

I due problemi hanno:

- Lo stesso numero di variabili decisionali.
- La matrice dei coefficienti A dei vincoli funzionali del compratore è la **trasposta** della matrice dei coefficienti dei vincoli funzionali del produttore.
- Il vettore dei coefficienti $\underline{p}$ della funzione obbiettivo del compratore coincide con il vettore dei termini noti del produttore.
- Il vettore dei termini noti del compratore $\underline{b}$ coincide con il vettore dei coefficienti della funzione obbiettivo del produttore.

5.2 Generalizzazione del problema duale

In generale, dato il problema di PL, detto **primale**

$$\begin{cases} \max & \omega = \underline{p}^T \underline{x} + \gamma \\ \text{s.a.} & A\underline{x} \leq \underline{b} \\ & \underline{x} \geq \underline{0} \end{cases} \quad \text{con } \underline{x}, \underline{p} \in \mathbb{R}^n, \underline{b}, \underline{y} \in \mathbb{R}^m, A \in \mathbb{R}^{m,n} \text{ e } \gamma \in \mathbb{R}$$

il problema **duale** è

$$\begin{cases} \min & \varphi_D = \underline{b}^T \underline{y} + \gamma \\ \text{s.a.} & A^T \underline{y} \geq \underline{p} \\ & \underline{y} \geq \underline{0} \end{cases}$$

Vale il seguente **teorema**: «Il duale del problema duale è il problema primale (*in ambito lineare*). Quindi si ha una relazione biunivoca».

Infatti, portando il problema duale in un problema di massimo si ha:

$$\begin{cases} \max & \omega_D = -\underline{b}^T \underline{y} - \gamma \\ \text{s.a.} & -A^T \underline{y} \leq -\underline{p} \\ & \underline{y} \geq \underline{0} \end{cases} \rightarrow \text{calcolo il duale} \rightarrow \begin{cases} \min & \varphi_{DD} = -\underline{p}^T \underline{z} - \gamma \\ \text{s.a.} & -A\underline{z} \geq -\underline{b} \\ & \underline{z} \geq \underline{0} \end{cases}$$

Ritornando al problema in forma di massimo si ha:

$$\begin{cases} \max & \omega_{DD} = \underline{p}^T \underline{z} + \gamma \\ \text{s.a.} & A\underline{z} \leq \underline{b} \\ & \underline{z} \geq \underline{0} \end{cases}$$

Passando dal problema primale al duale si ha:

Segno del vincolo nel problema primale	Variabile associata nel problema duale
$\leq$	≥ 0
$\geq$	≤ 0
$=$	<i>libera</i>

5.3 Relazione tra il valore di ω in una soluzione del primale ed il valore di φ_D in una soluzione del duale

Data la coppia *primale-duale*

$$\left\{ \begin{array}{ll} \max & \omega = \underline{p}^T \underline{x} + \gamma \\ \text{s.a.} & A\underline{x} \leq \underline{b} \\ & \underline{x} \geq \underline{0} \end{array} \right. \quad \left\{ \begin{array}{ll} \min & \varphi_D = \underline{b}^T \underline{y} + \gamma \\ \text{s.a.} & A^T \underline{y} \geq \underline{p} \\ & \underline{y} \geq \underline{0} \end{array} \right.$$

sia il vettore $\begin{bmatrix} \underline{x} \\ \underline{s} \end{bmatrix}$ una soluzione del problema primale (soddisfa $A\underline{x} + \underline{s} = \underline{b}$) e sia il vettore $\begin{bmatrix} \underline{y} \\ \underline{z} \end{bmatrix}$ una soluzione del problema duale (soddisfa $A^T \underline{y} - \underline{z} = \underline{p}$).

Nel prodotto scalare $\underline{p}^T \underline{x}$ sostituiamo il valore di $\underline{p}$:

$$\underline{p}^T \underline{x} = (A^T \underline{y} - \underline{z})^T \underline{x} = \underline{y}^T A\underline{x} - \underline{z}^T \underline{x}$$

da cui segue

$$\underline{p}^T \underline{x} + \underline{z}^T \underline{x} = \underline{y}^T A\underline{x}$$

Analogamente, nel prodotto scalare $\underline{b}^T \underline{y}$ sostituiamo il valore di $\underline{b}$:

$$\underline{b}^T \underline{y} = (A\underline{x} + \underline{x})^T \underline{y} = \underline{x}^T A\underline{y} + \underline{s}^T \underline{y}$$

da cui segue

$$\underline{b}^T \underline{y} - \underline{s}^T \underline{y} = \underline{x}^T A\underline{y}$$

Poiché il prodotto $\underline{y}^T A\underline{x}$ è uno scalare, allora è possibile trasporlo (*dato che il trasporto di uno scalare è lo scalare stesso*):

$$\underline{y}^T A\underline{x} = (\underline{y}^T A\underline{x})^T = \underline{x}^T A^T \underline{y}$$

Allora vale che

$$\underline{p}^T \underline{x} + \underline{z}^T \underline{x} = \underline{y}^T A\underline{x} = \underline{x}^T A^T \underline{y} = \underline{b}^T \underline{y} - \underline{s}^T \underline{y}$$

Aggiungendo γ ad entrambi i membri di questa equazione si ha

$$(\underline{p}^T \underline{x} + \gamma) + \underline{x}^T \underline{z} + \underline{s}^T \underline{y} = (\underline{b}^T \underline{y} + \gamma)$$

cioè

$$\omega(\underline{x}) + \underline{x}^T \underline{z} + \underline{s}^T \underline{y} = \varphi_D(\underline{y}) \quad \rightarrow \quad \omega(\underline{x}) + \sum_{j=1}^n x_j z_j + \sum_{i=1}^m s_i y_i = \varphi_D(\underline{y})$$

5.3.1 Variabili complementari

Tra le variabili del problema primale e quelle del problema duale esistono le seguenti **relazioni di complementarità**:

- Ogni variabile decisionale x_j del problema primale è associata ad una variabile scarto z_j del problema duale.
- Ogni variabile scarto s_i del problema primale è associata ad una variabile decisionale y_i del problema duale.

Quindi, si dicono **colonne complementari** le colonne associate alle variabili complementari.

5.4 Relazione tra il valore di ω in una soluzione ammissibile del primale ed il valore di φ_D in una soluzione ammissibile del duale

5.4.1 Teorema di dualità debole

Se $\begin{bmatrix} \underline{x} \\ \underline{s} \end{bmatrix}$ è soluzione ammissibile per il problema di massimo e $\begin{bmatrix} \underline{y} \\ \underline{z} \end{bmatrix}$ una soluzione ammissibile per il problema di minimo, allora, tra i valori assunti in queste due soluzioni da ω e φ_D , vale la relazione $\omega \leq \varphi_D$.

Dimostrazione

Consideriamo la relazione $\omega(\underline{x}) + \underline{x}^T \underline{z} + \underline{s}^T \underline{y} = \varphi_D(\underline{y})$. Allora, dati i vincoli di non negatività, risulta evidente che $\omega \leq \varphi_D$.

5.4.2 Conseguenze del teorema della dualità debole

5.4.2.1 Corollario 1

Se $\begin{bmatrix} \underline{x}^* \\ \underline{s}^* \end{bmatrix}$ è soluzione ammissibile per il problema di massimo, $\begin{bmatrix} \underline{y}^* \\ \underline{z}^* \end{bmatrix}$ una soluzione ammissibile per il problema di minimo e $\omega(\underline{x}^*, \underline{s}^*) = \varphi_D(\underline{y}^*, \underline{z}^*)$, allora le due soluzioni ammissibili sono ottime per i rispettivi problemi.

Dimostrazione

Dato che $\begin{bmatrix} \underline{x}^* \\ \underline{s}^* \end{bmatrix}$ è una soluzione ammissibile per il primale, per il teorema di dualità debole si ha che $\omega(\underline{x}^*, \underline{s}^*) \leq \varphi_D(\underline{y}, \underline{z})$ (per ogni $\begin{bmatrix} \underline{y} \\ \underline{z} \end{bmatrix}$ ammissibile per il duale), cioè ω è un minorante dell'insieme dei valori assunti da φ_D . Per ipotesi, però, sappiamo che $\omega(\underline{x}^*, \underline{s}^*) = \varphi_D(\underline{y}^*, \underline{z}^*)$, quindi possiamo scrivere

$$\varphi_D(\underline{y}^*, \underline{z}^*) \leq \varphi_D(\underline{y}, \underline{z})$$

Di conseguenza abbiamo che $\varphi_D(\underline{y}^*, \underline{z}^*)$ è il minimo assumibile da φ_D . In modo analogo si dimostra che $\omega(\underline{x}^*, \underline{s}^*)$ è il massimo assumibile da ω .

5.4.2.2 Corollario 2

In una coppia di problemi *primale-duale*, se un problema è illimitato allora l'altro è inammissibile.

Dimostrazione

Sia ad esempio il problema *primale* di massimo superiormente illimitato e supponiamo, per assurdo, che il problema *duale* abbia una soluzione ammissibile $\begin{bmatrix} \underline{y} \\ \underline{z} \end{bmatrix}$. Allora, per il teorema della dualità debole, la funzione obbiettivo del *primale* sarebbe limitata superiormente, contraddicendo l'ipotesi.

Attenzione!!

Il viceversa del teorema non vale, ossia, se il primale o il duale è inammissibile, non è detto che il duale sia illimitato perché potrebbe essere anch'esso inammissibile.

Se in una coppia di problemi duali, uno dei problemi non è ammissibile, l'altro problema

- o è illimitato.
- o non è ammissibile.

5.5 Basi complementari

Due basi vengono dette **complementari** quando le variabili che sono nella base di un problema sono complementari a quelle in base dell'altro problema.

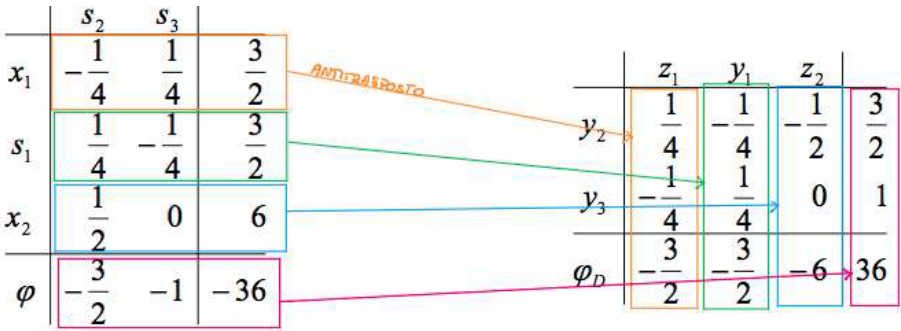
5.6 Relazioni tra soluzioni del problema primale e del problema duale

Problema primale	Problema duale
Ammissibilità	Ottimalità
Ottimalità	Ammissibilità
Ammissibile - Non Ottima	Non Ammissibile - Ottima
Non Ammissibilità	Non Ottimalità
Non Ammissibile - Non Ottima	Non Ammissibile - Non Ottima

Se nel simplesso si dispone di una soluzione ottima ma non ammissibile, si sa che questa corrisponde ad una soluzione duale ammissibile ma non ottima. Quindi, il ragionamento che si fa, è quello di mantenere il primale come riferimento ma aggiungere dei passi che lavorano come se stessimo lavorando con il duale. In questo caso si parla di **simplesso duale**. Questo ci permette di lavorare fuori dalla regione di ammissibilità con una garanzia di convergenza.

5.7 Tableau ridotti

Esiste una regola pratica per passare dal tableau ridotto (contiene solo le colonne delle variabili non di base e dei termini noti) del primale a quello del duale. In particolare i due tableau sono uno l'antitrasposto dell'altro:



In particolare, i due tableau sono complementari tra di loro.

5.8 Sottomatrici quadrate complementari

Denotiamo con

- P_m una sottomatrice quadrata di ordine m di P (cioè un sottoinsieme di m colonne del tableau del primale)
- D_n una sottomatrice quadrata di ordine n di D (cioè un sottoinsieme di n colonne del tableau del duale)

Le due sottomatrici si dicono **complementari** se le colonne complementari alle colonne di P_m non appartengono a D_n .

Inoltre, le due matrici P_m e D_n non singolari e complementari, formano una coppia di **basi complementari**.

5.8.1 Teorema

Siano

- $P = \begin{bmatrix} A & I_m \end{bmatrix}$ e sia P_m una sua sottomatrice quadrata di ordine m
- $D = \begin{bmatrix} I_n & -A^T \end{bmatrix}$ e sia D_n una sua sottomatrice quadrata di ordine n

Allora P_m è una base di P se e solo se D_n è una base di D . Questo significa che data una coppia di sottomatrici quadrate complementari, esse sono o entrambe nonsingolari e entrambe singolari.

Dimostrazione

Per prima cosa mostriamo che se P_m è singolare, allora D_n è singolare:

1. Si può dimostrare che ogni riga di P è ortogonale ad ogni riga di D . Infatti, il prodotto scalare tra le due matrici è:

$$PD^T = \begin{bmatrix} A & I_m \end{bmatrix} \begin{bmatrix} I_n \\ -A \end{bmatrix} = 0$$

2. Se P_m è singolare, esiste una combinazione lineare delle colonne di P_m a coefficienti non tutti nulli, che risulta nel vettore nullo, cioè

$$\exists \underline{v} \in \mathbb{R}^n, \underline{v} \neq \underline{0}, \text{ tale che } \underline{v}^T P_m = \underline{0}^T$$

3. Riordiniamo le colonne di P in modo che le colonne di P_m siano le prime m di $P \rightarrow PQ = \begin{bmatrix} P_m & P_n \end{bmatrix}$.
4. Effettuiamo il corrispondente riordinamento sulle colonne di D , ottenendo che le colonne di D_n sono le ultime n di $D \rightarrow DQ = \begin{bmatrix} D_m & D_n \end{bmatrix}$.
5. Dai due punti precedenti segue che $P_m D_m^T + P_n D_n^T = P Q Q^T D^T = P D^T = 0$.
6. Poiché $P_m D_m^T + P_n D_n^T = 0$, allora il risultato della combinazione lineare delle righe della matrice $P_m D_m^T + P_n D_n^T$ è il vettore nullo, qualunque sia il vettore dei coefficienti di combinazione.
7. Per quanto visto al punto 2, $\underline{v}^T P_m = \underline{0}^T$. Ma allora $\underline{v}^T P_m D_m^T = \underline{0}^T$.
8. Di conseguenza, dati i punti 6 e 7, vale che $\underline{v}^T P_n D_n^T = \underline{0}^T$.
9. La matrice P ha rango m , quindi la combinazione lineare delle righe di PQ con vettore dei coefficienti di combinazione $\underline{v} \in \mathbb{R}^m - \{\underline{0}\}$ risulta in un vettore non nullo: $\underline{v}^T PQ \neq \underline{0}^T$.
10. Dalle relazioni dei punti 1 e 8 segue che $\underline{v}^T P_n \neq \underline{0}^T$.
11. Dalle relazioni ottenute ai punti 8 e 9 segue che D_n è singolare.

5.8.2 Osservazione

In una coppia di *problemi duali* (primale in forma di massimo e duale in forma di minimo), le ***soluzioni di base complementari hanno lo stesso valore della funzione obiettivo***.

5.9 Teorema degli scarti complementari

La soluzione $\begin{bmatrix} \underline{x} \\ \underline{s} \end{bmatrix}$ ammissibile per il problema primale e la soluzione $\begin{bmatrix} \underline{z} \\ \underline{y} \end{bmatrix}$ ammissibile per il problema duale sono ottime per i rispettivi problemi *se e solo se*

$$x_j \cdot z_j = 0 \quad \forall j \quad e \quad s_i \cdot y_i = 0 \quad \forall i$$

Queste due condizioni vengono dette **condizioni di complementarità** e possono essere anche espresse nella forma

$$x_j \cdot \left(\sum_{i=1}^m a_{i,j} y_i - p_j \right) = 0 \quad \forall j \quad e \quad \left(\sum_{i=1}^m a_{i,j} x_j - b_i \right) \cdot y_i = 0 \quad \forall i$$

$$\mathbf{x}(\mathbf{A}\mathbf{y}-\mathbf{p})=0$$

Capitolo 6

Programmazione lineare intera

La classe dei problemi di programmazione matematica con *variabili intere e funzione obiettivo e funzioni vincolo lineari* è indicata con il termine **ottimizzazione lineare discreta**. Se, inoltre, tutte le variabili sono intere si utilizza anche il termine **programmazione lineare intera (PLI)**.

Se, invece, solo un sottoinsieme di variabili è composto da variabili intere si parla di **programmazione lineare intera mista (PLIM)**.

Quando si deve risolvere un problema di **programmazione lineare intera** il metodo del simplesso, generalmente, non può aiutare in quanto trova solamente delle *soluzioni nei vertici della regione ammissibile*, che potrebbero essere anche soluzioni non intere. Inoltre, nei problemi di PLI, la regione ammissibile è composta da un insieme discreto di punti, quindi non è più considerabile come un insieme convesso.

Si può quindi utilizzare il **rilassamento continuo** cioè lavorare con il simplesso sul problema **continuo** associato a quello interno (*cioè viene eliminato il vincolo di integrità*). La soluzione che si trova per il rilassamento potrebbe soddisfare già i vincoli di integrità, oppure ci fornisce una indicazione sulla regione di piano nel cui intorno sappiamo troveremo l'ottimo.

I ragionamenti che vengono fatti nella ricerca dell'ottimo sono:

- METODI DI BRANCH & BOUND
- METODI DEI PIANI DI TAGLIO

6.1 Problemi di programmazione lineare intera

I problemi di programmazione lineare intera, con variabili decisionali intere, sono più difficili da analizzare rispetto a quelli continui. Infatti, in PLI, la regione ammissibile è costituita da punti isolati (*quelli con valore intero*) e la soluzione ottima, se esiste, non necessariamente si trova sull'intersezione di n frontiere dei vincoli funzionali che definiscono la regione ammissibile.

Questo significa che non possiamo restringere la ricerca delle *soluzioni di base ammissibili ottime* al solo sottoinsieme delle soluzioni di base (*cioè ai vertici*).

I problemi di PLI possono essere classificati come segue:

- PROBLEMI LA CUI STRUTTURA GARANTISCE L'INTEREZZA DELLA SOLUZIONE: se sono soddisfatte certe condizioni di interezza sui dati, allora tutte le soluzioni di base sono a componenti interi.
Questo ci permette di utilizzare l'*algoritmo del simplesso* oppure *metodi ad Hoc* costruiti per avere un costo inferiore. Sono problemi di questo tipo i problemi di trasporto, i problemi di assegnamento, ecc...
- PROBLEMI LA CUI INTEREZZA DELLA SOLUZIONE È GARANTITA DALL'USO DI OPPORTUNI METODI DI RICERCA DELLA SOLUZIONE (Branch & Bound e metodo dei piani di taglio).

Nonostante la somiglianza tra la risoluzione di problemi di PL e problemi di PLI, i problemi di ottimizzazione discreta sono in generale più difficili da risolvere, poiché per essi non esistono algoritmi di risoluzione con complessità polinomiale. Quindi, nell'affrontare problemi di PLI:

- O ci si accontenta di una soluzione sub-ottima ottenuta mediante algoritmo euristico. Questo metodo risolutivo è in genere utilizzato quando si hanno problemi da risolvere in tempo reale o comunque in breve tempo.
- O, se si vuole determinare la soluzione ottima, si devono utilizzare i cosiddetti **metodi esatti** che nel caso peggiore possono arrivare a richiedere un tempo di calcolo esponenziale rispetto alla dimensione del problema. Per problemi che non richiedono una soluzione istantanea e che presentano una dimensione non eccessiva, questo è il metodo generalmente utilizzato.

6.2 Metodo di Branch & Bound

Il metodo di *branch & bound* è un metodo basato, implicitamente, sul sempliceo duale. Supponiamo di avere a che fare con il seguente problema di PLI:

$$P_0 = \begin{cases} \max & \omega = 4x_1 + 9x_2 \\ \text{s.a.} & 2x_1 + 5x_2 \leq 8 \\ & 3x_1 + 2x_2 \leq 9 \\ & x_1, x_2 \geq 0 \text{ e intere} \end{cases} \rightarrow \text{rilassamento continuo} \rightarrow P_{0\text{-rilassato}} = \begin{cases} \max & \omega = 4x_1 + 9x_2 \\ \text{s.a.} & 2x_1 + 5x_2 \leq 8 \\ & 3x_1 + 2x_2 \leq 9 \\ & x_1, x_2 \geq 0 \end{cases}$$

Graficamente si determina che la soluzione ottima del rilassamento continuo di P_0 è l'intersezione dei due vincoli. In particolare, le coordinate dell'ottimo sarebbero

$$\begin{cases} x_1 = \frac{29}{11} \\ x_2 = \frac{6}{11} \end{cases}$$

Queste non sono intere, quindi non sono accettabili per il problema P_0 .

La parte intera del valore ottimo della funzione obiettivo del rilassamento continuo di P_0 è un upper bound per il valore ottimo della funzione obiettivo di P_0 (perciò è un maggiorante del valore ottimo a cui posso arrivare):

$$UB_o = \lfloor \omega \rfloor = \left\lfloor 4 \cdot \frac{29}{11} + 9 \cdot \frac{6}{11} \right\rfloor = \left\lfloor \frac{170}{11} \right\rfloor = 15$$

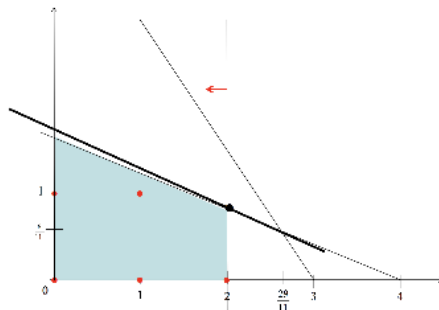
In questo caso si ha una soluzione non intera sia per la variabile x_1 che per la variabile x_2 , quindi si può scegliere in modo indifferente su quale variabile fare il **branching**. Se, ad esempio, scegliamo di fare un *branching* su x_1 si generano due problemi figli P_1 e P_2 :

$$P_1 = \begin{cases} \max & \omega = 4x_1 + 9x_2 \\ \text{s.a.} & 2x_1 + 5x_2 \leq 8 \\ & 3x_1 + 2x_2 \leq 9 \\ & x_1 \leq 2 \\ & x_1, x_2 \geq 0 \text{ e intere} \end{cases} \quad P_2 = \begin{cases} \max & \omega = 4x_1 + 9x_2 \\ \text{s.a.} & 2x_1 + 5x_2 \leq 8 \\ & 3x_1 + 2x_2 \leq 9 \\ & x_1 \geq 3 \\ & x_1, x_2 \geq 0 \text{ e intere} \end{cases}$$

Proseguendo in questo modo, la regione ammissibile di ogni sottoproblema generato con il *branch* è inferiore rispetto a quella del problema da cui il sottoproblema deriva.

L'idea base è quella di calcolare l'upper bound ad ogni passaggio, effettuando il rilassamento continuo di ogni sottoproblema, fino a quando non si arriva ad un valore intero.

La determinazione del punto di ottimo, a ciascuna iterazione, avviene tramite l'utilizzo del *metodo grafico*:

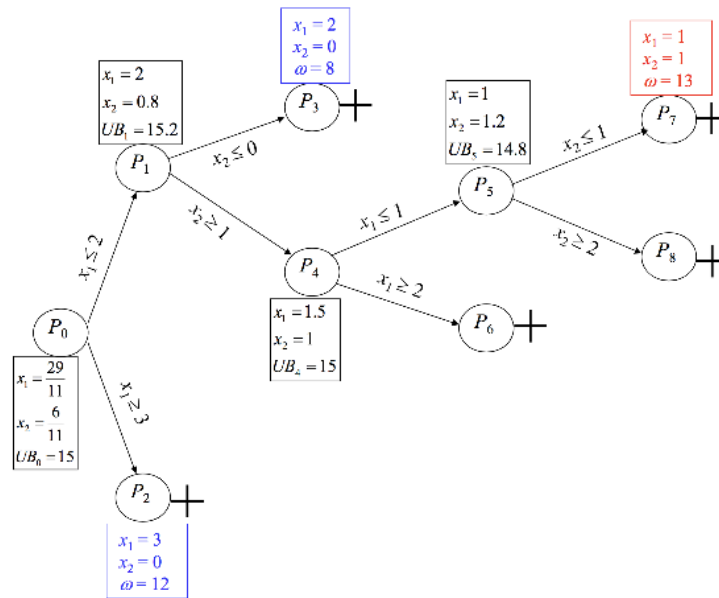


Con questa procedura si generano tanti sottoproblemi, di ciascuno dei quali possiamo trovare l'upper bound. In particolare ci fermeremo e non faremo più branching su di un ramo quando:

- L'UPPER BOUND PER QUEL RAMO È INTERO: questo perché anche ammesso che facendo dei branch si trovino nuove regioni ammissibili, di certo queste non saranno migliori dell'upper bound trovato. Questa soluzione sarà ammissibile, ma non possiamo (*al momento*) dire nulla sull'ottimalità.
- L'UPPER BOUND PER QUEL RAMO È INFERIORE AD UNA SOLUZIONE AMMISSIBILE GIÀ TROVATA: questo perché se già abbiamo un upper bound inferiore ad una soluzione ammissibile già trovata, anche continuando ed eventualmente arrivando ad un'altra soluzione ammissibile, essa non sarà sicuramente ottima.

- LA REGIONE AMMISSIBILE PER IL SOTTOPROBLEMA CONSIDERATO È VUOTA.

Per tenere traccia di tutti i sottoproblemi analizzati e dei valori trovati, si tiene un **albero dei branch**:



Il simbolo + in fondo ai rami indica la chiusura di quel ramo. In particolare, guardando l'albero, si nota che ci si fermerà e non si faranno più branch quando tutti i rami sono chiusi.

6.3 Metodo dei piani di taglio o di Gomory

6.3.1 Costi fissi di attivazione

Supponiamo di voler analizzare il seguente problema:

Un'azienda dispone di H ore di manodopera che possono essere utilizzate per la produzione di n articoli diversi. Per ogni unità dell'articolo j consideriamo:

- h_j ore di manodopera;
- g_j guadagno derivante dalla vendita del prodotto.

L'attivazione della linea di produzione dell'articolo j ha un costo fisso di F_j .

Determinare i livelli di produzione degli n articoli in modo da massimizzare il guadagno totale della fabbrica.

Le **variabili decisionali** del problema sono:

- x_j cioè il livello di produzione dell'articolo j . In base ai valori di x_j sono possibili due casi:
 - x_j è una variabile reale non negativa, quando l'articolo j è composto da unità divisibili.
 - x_j è una variabile intera, se l'articolo j è composto da unità indivisibili.
- y_j , *variabile binaria*, che assumerà valore 1 se la linea del prodotto j viene attivata, oppure 0 se non viene attivata.

Questa variabile introduce la necessità di avere dei vincoli:

- Se $y_j = 0$, cioè non si è attivata la linea per l'articolo, allora è necessario $x_j = 0$.
- Se $x_j > 0$, allora è necessario $y_j = 1$.

Per garantire che questi due vincoli siano rispettati, può essere utile introdurre il vincolo

$$\text{se } y_j = 1 \quad x_j \leq \frac{H}{h_j}$$

Infatti, questo vincolo:

- Esclude le soluzioni in cui $x_j > 0$ e $y_j = 0$, cioè si deve avere una produzione nulla se l'impianto non è stato attivato.
- Ammette le soluzioni in cui $x_j > 0$ e $y_j = 1$, purché quando la linea è attivata non venga superato il rapporto tra il numero di ore di manodopera disponibili e la necessità di manodopera per il singolo articolo.
- Ammette le soluzioni in cui $x_j = 0$ e $y_j = 0$.
- Ammette le soluzioni in cui $x_j = 0$ e $y_j = 1$, cioè il caso scomodo in cui l'impianto è attivo ma non sta producendo nulla.

La tecnica di modellazione così utilizzata è detta **Big-M**: si utilizza la disuguaglianza

$$x_j \leq M_j y_j \quad 1 \leq j \leq n$$

dove M_j indica una quantità sufficientemente grande da rendere il vincolo $x_j \leq M_j y_j$ *ridondante* (**cioè sempre verificato**) quando $y_j = 1$. Questa relazione ci dice che M_j deve essere un valore maggiorante del valore che può essere assunto dagli x_j , quindi si può ulteriormente semplificare la trattazione supponendo di utilizzare un unico valore per tutti i prodotti

$$M = \max_{1 \leq j \leq n} \frac{H}{h_j}$$

Quindi non si hanno vincoli su quanto grande debba essere M_j ma si dimostra che le formulazioni con valori di M più piccoli (*ma comunque che verificano la disequazione*) producono risultati migliori dal punto di vista numerico.

Introduciamo inoltre il **vincolo sulla disponibilità totale di ore**

$$\sum_{j=1}^n h_j x_j \leq H$$

La **funzione obiettivo** sarà invece

$$\max \sum_{j=1}^n g_j x_j - \sum_{j=1}^n F_j y_j$$

Quello che si nota è che, a causa della forma della funzione obiettivo, ogni soluzione con $x_j = 0$ e $y_j = 1$ (ammessa dal vincolo utilizzato) ha un valore della funzione obiettivo strettamente maggiore dell'obiettivo nella soluzione che da essa differisce solamente per $y_j = 0$.

La soluzione ottima non può quindi avere $x_j = 0$ e $y_j = 1$.

6.3.2 Lotti minimi di produzione

Consideriamo il problema uguale al precedente con, però, due modifiche:

- Non consideriamo i costi fissi di produzione.
- Introduciamo un valore q_j cioè il valore minimo che deve essere prodotto dell'articolo j , qualora si scelga di produrlo.

Queste due modifiche portano una semplificazione della funzione obiettivo, che diventa

$$\max \sum_{j=1}^n g_j x_j$$

con il vincolo aggiuntivo

$$x_j \geq q_j y_j \quad 1 \leq j \leq n$$

6.3.3 Vincoli disgiuntivi

Supponiamo di voler analizzare il seguente problema:

Si devono sequenzializzare n lavorazioni indipendenti su di una macchina che può eseguire solamente una lavorazione per volta e che non può essere interrotta durante una lavorazione. Si denota con p_j il tempo necessario per l'esecuzione della j -esima lavorazione. Si vuole determinare la sequenza con cui eseguire le lavorazioni sulla macchina in modo da minimizzare la media degli istanti di completamento delle varie lavorazioni:

$$\min \frac{1}{n} \sum_{j=1}^n (t_j + p_j)$$

dove $t_j \geq 0$ sono variabili continue che rappresentano l'istante di inizio di esecuzione della lavorazione j .

Il modello, date due lavorazioni j e k deve imporre che solo uno dei due seguenti casi si possa verificare:

- Se j precede k , allora $t_j + p_j \leq t_k$
- Se j segue k , allora $t_k + p_k \leq t_j$

I due casi sono **mutuamente esclusivi**, quindi i vincoli associati devono valere in modo alternativo. Si parla di *vincoli disgiuntivi*.

Per modellare vincoli disgiuntivi si introduce una variabile binaria $y_{j,k}$ per ogni coppia di vincoli associata alle lavorazioni j e k , in modo che valga 1 se il modello impone il **primo vincolo** e 0 se il modello impone il **secondo**:

$$\begin{cases} t_j + p_j \leq t_k + M(1 - y_{j,k}) \\ t_k + p_k \leq t_j + My_{j,k} \end{cases}$$

dove M rappresenta una costante sufficientemente grande da non introdurre altre restrizioni nella regione ammissibile, oltre all'imposizione del secondo vincolo di precedenza o del primo. Ad esempio è sufficiente porre $M = \sum_{j=1}^n p_j$.

6.3.4 Problema di selezione di sottoinsiemi

In generale, in un problema di **set-covering** sono dati:

- Un insieme $M = \{1, \dots, m\}$
- Una famiglia di suoi sottoinsiemi $S_j \subseteq M$, a ciascuno dei quali è associato un costo c_j per $j = 1, \dots, n$.

Si devono selezionare alcuni sottoinsiemi S_j (ossia determinare $T \subseteq \{1, \dots, n\}$) in modo tale che **la loro unione copra** M ($\bigcup_{j \in T} S_j = M$) e che il loro **costo totale sia minimo** ($\min \sum_{j=1}^n c_j$).

6.3.5 Localizzazione di impianti

Il problema della **localizzazione di impianti** fa riferimento al concetto di *grafo bipartito*: l'insieme dei nodi è suddiviso in due sottoinsiemi, cioè i **nodì di partenza** ed i **nodì di arrivo**, senza collegamenti tra elementi di uno stesso insieme.

Ad esempio consideriamo un problema di *trasporto* con m possibili origini (*generalmente impianti produttivi*) ed n destinazioni (*generalmente i clienti*).

Il problema è stabilire quali tra le m possibili origini devono essere operanti, dato che rendere operante un'origine i comporta un costo fisso di avviamento F_i , per avere un costo totale minimo. Inoltre si conoscono:

- $c_{i,j}$ cioè i costi per trasportare un unità di prodotto dall'origine i alla destinazione j .
- d_j cioè la domanda del cliente j .

Definiamo:

- Le variabili $x_{i,j} \geq 0$ per rappresentare la quantità trasportata dall'origine i alla destinazione j .
- Le variabili binarie $y_i = \begin{cases} 1 & \text{se l'impianto } i \text{ è operante} \\ 0 & \text{altrimenti} \end{cases}$

Con queste definizioni il problema può essere modellato nel seguente modo:

$$\left\{ \begin{array}{ll} \min & \sum_{i=1}^m \sum_{j=1}^n c_{i,j} x_{i,j} + \sum_{i=1}^m F_i y_i \\ \text{s.a.} & \sum_{i=1}^m x_{i,j} = d_j \quad 1 \leq j \leq n \quad (\text{vincoli di domanda}) \\ & \sum_{j=1}^n x_{i,j} \leq D y_i \quad 1 \leq i \leq m \quad (1) \\ & x_{i,j} \geq 0 \quad 1 \leq i \leq m \quad 1 \leq j \leq n \\ & y_i \in \{0, 1\} \quad 1 \leq i \leq m \end{array} \right.$$

Il vincolo indicato con (1) può essere modificato, con la tecnica del **Big-M**, mettendo $D = \sum_{j=1}^n d_j$ cioè imponendo che nulla possa essere trasportato da un impianto non operante.

6.3.6 Metodo dei piani di taglio di Gomory

Consideriamo il problema di PLI seguente:

$$P_0 = \left\{ \begin{array}{ll} \max & \omega = 5x_1 + 2x_2 \\ \text{s.a.} & 2x_1 + 2x_2 \leq 9 \\ & 3x_1 + x_2 \leq 11 \\ & x_1, x_2 \geq 0 \text{ e intere} \end{array} \right. \rightarrow \text{rilassamento continuo} \rightarrow P_{0-\text{rilassato}} = \left\{ \begin{array}{ll} \max & \omega = 5x_1 + 2x_2 \\ \text{s.a.} & 2x_1 + 2x_2 + s_1 = 9 \\ & 3x_1 + x_2 + s_2 = 11 \\ & x_1, x_2, s_1, s_2 \geq 0 \end{array} \right.$$

Il tableau ottimo associato alla soluzione ottima del rilassamento continuo è:

$$\begin{array}{ccccc|c} x_1 & x_2 & s_1 & s_2 & \varphi & \\ \hline 0 & 1 & \frac{3}{4} & -\frac{1}{2} & 0 & \frac{5}{4} \\ 1 & 0 & -\frac{1}{4} & \frac{1}{2} & 0 & \frac{13}{4} \\ \hline 0 & 0 & -\frac{1}{4} & -\frac{3}{2} & 1 & -\frac{75}{4} \end{array}$$

In questa soluzione ottima, sia le variabili di base che la variabile obbiettivo hanno un valore frazionario, quindi non sono soluzione ammissibile per il problema di PLI originario.

Esprimiamo le righe del tableau, mettendo le frazioni come somma di parte intera e frazionaria (*massimo intero che precede il numero + parte frazionaria*) composta da termini sempre non negativi. In particolare, per la situazione attuale abbiamo:

$$x_2 + \frac{3}{4}s_1 - s_2 + \frac{1}{2}s_2 = 1 + \frac{1}{4}$$

$$x_1 - s_1 + \frac{3}{4}s_1 + \frac{1}{2}s_2 = 3 + \frac{1}{4}$$

$$\omega + \frac{1}{4}s_1 + s_2 + \frac{1}{2}s_2 = 18 + \frac{3}{4}$$

Tutte queste equazioni possono essere generalizzate nella forma

$$A + B = C + D$$

dove:

- A è il polinomio contenente tutte le parti intere a sinistra dell'uguale;
- B è il polinomio contenente tutte le parti frazionarie a sinistra dell'uguale;
- C è la parte intera del termine noto;
- D è la parte frazionaria del termine noto.

A questo punto, tra tutte le righe si sceglie quella il cui termine noto assume valore maggiore e si impone la disuguaglianza, detta del **taglio**:

$$B \geq D$$

Questa disuguaglianza viene poi aggiunta come vincolo del problema, si aggiungono le variabili scarto o surplus, e poi si aggiunge la riga al tableau del simplesso:

x_1	x_2	s_1	s_2	s_3	φ	
0	1	$\frac{3}{4}$	$-\frac{1}{2}$	0	0	$\frac{5}{4}$
1	0	$-\frac{1}{4}$	$\frac{1}{2}$	0	0	$\frac{13}{4}$
0	0	$-\frac{1}{4}$	$-\frac{1}{2}$	1	0	$-\frac{3}{4}$
0	0	$-\frac{1}{4}$	$-\frac{3}{2}$	0	1	$-\frac{75}{4}$

Su questo tableau si applica ora il **simplesso duale**, fino ad arrivare ad una nuova soluzione ammissibile per il rilassamento continuo. Eventualmente questa nuova soluzione potrebbe generare un nuovo **taglio** oppure essere già ottima ed intera.

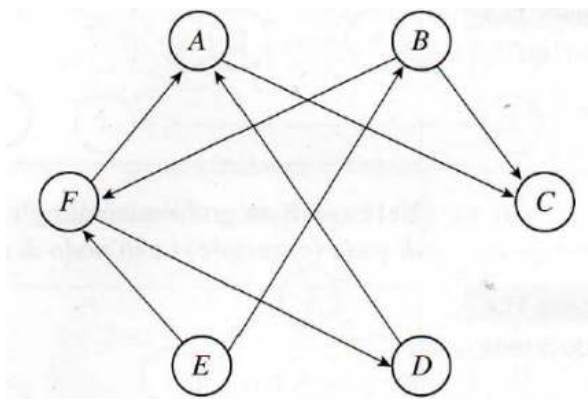
Capitolo 7

Ottimizzazione su grafi

7.1 I grafi, le loro applicazioni, definizioni e rappresentazioni

I **grafi** sono strumenti per la rappresentazione e la soluzione di problemi decisionali in cui le possibili alternative decisionali sono in numeri finito.

Un esempio di grafo è:



I punti A, B, C, D, E, F sono detti **nodi** del grafo, mentre i segmenti che congiungono i nodi sono detti **archi** del grafo.

7.1.1 Grafi non orientati

Un **grafo non orientato** (o *simmetrico*) $G = (N, E)$ è un insieme di nodi e di archi, in questo caso detti **lati**, che li collegano. L'insieme dei nodi si indica con $N = \{1, 2, \dots, n\}$ mentre l'insieme dei lati si indica con $E = \{\dots, \{i, j\}, \dots\}$ dove (i, j) indica che il lato collega il nodo i con il nodo j . In questo caso i lati non sono indicati con una freccia ma con un semplice segmento.

Si definiscono **estremi di un lato** $\{i, j\}$ i nodi i e j collegati dal lato. A sua volta, il lato $\{i, j\}$ si dice **indicante nei nodi i e j** .

Due nodi collegati da un lato si dicono **adiacenti**.

In un grafo simmetrico, l'insieme di tutti i lati incidenti in un nodo i è detto **stella** di archi incidenti in i e viene indicato con $S(i)$. La cardinalità dell'insieme $S(i)$ è detta **grado** del nodo i .

Un arco che collega un nodo con se stesso viene detto **autoanello**.

In un grafo non orientato si parla di **lati adiacenti** quando essi hanno un nodo in comune.

Una sequenza di lati consecutivi $\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{k-1}, v_k\}$ è detta **cammino** che collega i nodi v_1 e v_k .

7.1.2 Grafi orientati

Un **grafo orientato** $G = (N, A)$ è un insieme di nodi e di archi che li collegano. L'insieme dei nodi si indica con $N = \{1, 2, \dots, n\}$ mentre l'insieme degli archi si indica con $A = \{\dots, (i, j), \dots\}$ dove (i, j) indica che l'arco collega il nodo i con il nodo j .

Nel caso di un grafo orientato, gli estremi di un arco (i, j) si distinguono in **nodo di coda** e **nodo di testa**.

In un grafo orientato $G = (N, A)$ si definiscono:

- **Stella entrante** nel nodo i , indicata con $BS(i) = \{(j, i) \in A\}$ l'insieme degli archi che hanno per testa il nodo i . La cardinalità di questo insieme viene detta **grado entrante** nel nodo.
- **Stella uscente** dal nodo i , indicata con $FS(i) = \{(i, j) \in A\}$ l'insieme degli archi che hanno per coda il nodo i . La cardinalità di questo insieme viene detta **grado uscente** nel nodo.

Si possono inoltre individuare due altri insiemi:

- **Nodi predecessori** di un nodo i l'insieme dei nodi $P(i) = \{j \in N : (j, i) \in A\}$, cioè i nodi raggiungibili da altri nodi, tramite archi con testa i .
- **Nodi successori** di un nodo i l'insieme dei nodi $S(i) = \{j \in N : (i, j) \in A\}$, cioè i nodi raggiungibili da altri nodi, tramite archi con coda i .

Nel caso si stia analizzando un grafo orientato, allora una sequenza di archi costituisce un **cammino** se il nodo di testa di ogni arco coincide con il nodo di coda dell'arco successivo.

7.1.3 Definizioni comuni

Un cammino che non contiene archi ripetuti si dice **semplice** mentre un cammino che non contiene nodi ripetuti si dice **elementare**.

Si definisce **ciclo** un cammino in cui il nodo iniziale del primo arco ed il nodo finale dell'ultimo coincidono. Un grafo **aciclico** è un grafo che non contiene cicli.

Un **grafo connesso** è un grado i cui esiste almeno un cammino tra ogni coppia di nodi.

Un grafo si definisce **albero** se è *non connesso* ed è *senza cicli*.

Un grafo si dice **semplice** se non ha autoanelli e archi paralleli, ossia archi con stesso nodo di coda e stesso nodo di testa.

Un **grafo completo** è un grafo semplice tale che ogni nodo è collegato ad ogni altro. Un grafo completo è quindi un grafo semplice, con il massimo numero di archi/lati possibile.

Si definisce **grafo parziale** o **sottografo** un grafo ottenuto come parte di un grafo più grande.

Un **grafo bipartito** è un grafo in cui l'insieme dei nodi può essere partizionato in due sottoinsiemi N_1 ed N_2 tali che esistono archi solamente tra nodi di N_1 e nodi di N_2 .

7.1.4 Rappresentazione di grafi tramite matrici

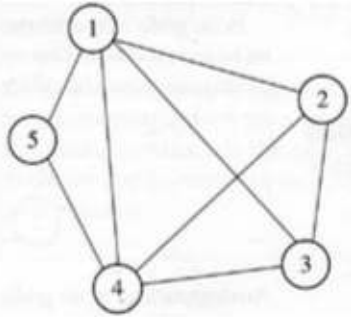
I grafi possono essere rappresentati algebricamente tramite matrici, utili per le fasi di modellazione.

Una matrice importante è la **matrice di incidenza nodi-lati** Γ di un grafo $G = (N, E)$ cioè la matrice $|N| \times |E|$ (cioè di dimensioni pari alle cardinalità dei due insiemi) con elementi

$$\gamma_{i,j} = \begin{cases} 1 & \text{se il lato } j \text{ è incidente nel nodo } i \\ 0 & \text{altrimenti} \end{cases}$$

Un esempio di matrice di incidenza è:

$$\Gamma = \begin{matrix} & \{1, 2\} & \{1, 3\} & \{1, 4\} & \{1, 5\} & \{2, 3\} & \{2, 4\} & \{3, 4\} & \{4, 5\} \\ \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \end{matrix}$$



La **matrice di incidenza nodi-archi** Γ di un grafo $G = (N, A)$ è la matrice $|N| \times |A|$ con elementi

$$\gamma_{i,j} = \begin{cases} -1 & \text{se il nodo } i \text{ è coda dell'arco } j \\ 1 & \text{se il nodo } i \text{ è testa dell'arco } j \\ 0 & \text{altrimenti} \end{cases}$$

La matrice di incidenza di un grafo orientato ha in ogni colonna esattamente un elemento $+1$ ed un elemento -1 . Si parla di **matrice totalmente Uni-Modulare** (qualsiasi sottomatrice estraibile avrà determinante $0, -1, 1$). Inoltre, almeno una delle righe della matrice si può ottenere come combinazione lineare delle altre, infatti si può dimostrare che il rango di Γ è uguale a $|N| - k$ dove k è il numero di componenti connesse al grafo non orientato ottenuto da G eliminando l'orientamento dei suoi archi.

Questa proprietà è molto comoda perché, in un problema di PLI, con matrice A totalmente Uni-Modulare, se ogni b_i è intero, allora anche i valori delle soluzioni $\underline{x}$ saranno interi, anche usando il simplesso per variabili continue.

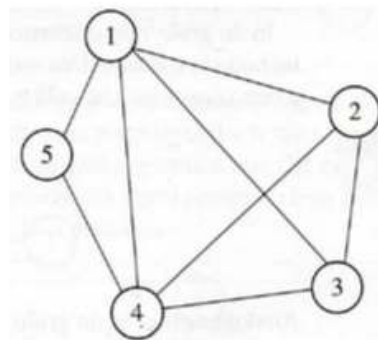
Una seconda matrice importante è la **matrice di adiacenza** Q di un grafo orientato $G = (N, A)$ cioè la matrice $|N| \times |N|$ con elementi

$$q_{i,j} = \begin{cases} 1 & \text{se l'arco } (i, j) \in A \\ 0 & \text{altrimenti} \end{cases}$$

Questa matrice può essere definita anche nel caso di grafi non orientati, considerando in $q_{i,j}$ il lato $\{i, j\}$ al posto dell'arco (i, j) : avrà quindi una struttura sempre simmetrica.

Un esempio di matrice di adiacenza è:

$$Q = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$



Se si utilizza un grafo **pesato**, in cui ai vari archi sono associati dei costi, è sufficiente sostituire nella matrice di adiacenza 1 con il *peso* dell'arco e 0 con un valore convenzionale (ad esempio ∞).

Queste matrici non possono tuttavia essere usate se si lavora con grafi **sparsi**. In questo caso si ricorre alla rappresentazione delle stelle entranti e delle stelle uscenti di ciascun nodo del grafo tramite la struttura dati di liste.

7.2 Il problema della raggiungibilità e algoritmo di visita

Siano $G = (N, A)$ un grafo orientato ed $s \in N$ un nodo del grafo.

Si vogliono determinare tutti i nodi raggiungibili dal nodo s , ossia il sottoinsieme $M \subseteq N$ tale che per ogni $t \in M$ esista un cammino da s a t .

Questo problema si presenta:

- Nell'individuazione delle componenti connesse di un grafo.
- Nell'algoritmo di Ford-Fulkerson per la determinazione del massimo flusso.

Per la risoluzione di questo problema si utilizza l'**algoritmo di visita**. Questo riceve in INPUT la descrizione del grafo G mediante la lista dei successori di ogni nodo.

L'algoritmo lavora su una struttura dati a *coda* (FIFO) che contiene i nodi raggiungibili da s e non ancora processati.

PSEUDO-CODICE DELL'ALGORITMO

```

Algoritmo  Visita_Grafo(N,S,s,M)
begin
    Q={s}, M=Q;
    while Q != InsiemeVuoto do
        begin
            scegliere il primo nodo h appartenente a Q;
            porre Q=Q\{h} e aggiungere h ad M;
            for each j in S(h) do
                begin
                    if j non appartiene a M then
                        Q=Q+{j};
                    end if
                end
            end
        end
    return M;
end

```

INIZIALIZZAZIONE

Si aggiunge il nodo s alla coda Q ($Q = \{s\}$). Allo stesso tempo si inizializza l'insieme M dei nodi raggiungibili a $M = \emptyset$.

PROCEDIMENTO

- Dalla coda Q viene estratto il primo elemento h , che viene inserito in M .
- In Q si inserisce ogni nodo successore di h che non appartiene ancora ad M .
- Si ripetono tali operazioni fino a quando la coda Q diventa vuota.
- Alla fine dell'algoritmo, M contiene tutti i nodi raggiungibili da s .

OSSERVAZIONE

Non sono mai presenti archi che vanno da M a $M \setminus N$, dato che altrimenti la testa di un tale arco farebbe parte dell'insieme M .

A tal proposito si definiscono:

- **Taglio entrante** introdotto da un insieme I l'insieme degli archi $\delta^-(I) = \{(i, j) \in A : i \in N \setminus I, j \in I\}$
- **Taglio uscente** introdotto da un insieme I l'insieme degli archi $\delta^+(I) = \{(i, j) \in A : i \in I, j \in N \setminus I\}$

COMPLESSITÀ COMPUTAZIONALE DELL'ALGORITMO

Ad ogni iterazione del ciclo *while* si sceglie un nodo $h \in Q$ non ancora elaborato sul quale vengono fatte due operazioni elementari. Inoltre si inseriscono in Q tutti i nodi j successori di h .

Poiché ogni nodo h non viene inserito in Q più di una volta, ogni arco (h, j) viene considerato al massimo una volta, perciò il numero di controlli *if* è al massimo $m = |A|$.

Questo ci porta a dire che la complessità dell'algoritmo di visita è $O(m)$.

7.3 Alberi di supporto di costo minimo

Un **albero di supporto** (o di *copertura*) di un grafo non orientato $G = (V, E)$ è un sottografo ad albero di G contenente tutti i vertici di G .

Il problema della determinazione di un albero di supporto a costo minimo si pone su di un grafo nel quale ad ogni lato $e \in E$ è associato il costo c_e . tale problema consiste nel trovare un albero T di supporto di G per cui è minima la somma dei costi dei suoi lati

$$\min c(T) = \sum_{e \in T} c_e$$

Un esempio in cui si necessita di trovare un albero di questo tipo è quando si vuole determinare quali linee ferroviarie costruire per collegare un certo numero di città, con il vincolo di non poterne lasciare isolata nemmeno una. L'albero che si viene a determinare, composto dall'insieme delle linee ferroviarie, dovrà essere di costo minimo e non deve presentare cicli.

7.3.1 Metodo di Kruskal

Il **metodo di Kruskal** per la determinazione dell'albero di supporto a costo minimo, è un algoritmo *greedy* cioè iterativo che tiene conto solamente di informazioni locali. Con algoritmi *greedy* in genere non si ottiene la soluzione ottima. Tuttavia si dimostra che l'algoritmo di Kruskal (*così come l'algoritmo di Prim*) arriva sicuramente ad una soluzione ottima.

I passaggi di cui il metodo si compone sono:

- **Ordinare** i lati in *ordine di costo non decrescente*. Se due lati hanno costo uguale, allora il loro costo è indifferente.
- Tra i lati non ancora considerati si **sceglie il lato di costo minimo** e lo si inserisce nell'albero di supporto se il suo inserimento non produce un ciclo. Altrimenti il lato viene eliminato.
- Si ripete il passo precedente, fino all'inserimento di $n - 1$ lati.

PSEUDOCODICE DELL'ALGORITMO

Algoritmo Kruskal(V,E,T*)

```
begin
    ordinare l'insieme di lati E secondo costi non decrescenti
    porre T*={insieme vuoto}
    while |T*|<n-1 do
        begin
            scegliere un lato "e" da E di costo minimo;
            Porre E=E-{e};
            if T*+{e} non ha cicli then
                T*=T*+{e}
            end if
        end
    end
    return T*;
end
```

OSSERVAZIONE

Durante l'esecuzione dell'algoritmo di Kruskal, la connessione tra i nodi non è garantita nei passi centrali ma solamente alla fine.

PSEUDOCODICE DELL'ALGORITMO PER LA DETERMINAZIONE DELL'ALBERO DI SUPPORTO DI COSTO MASSIMO

Algoritmo Kruskal_Max(V,E,T*)

```
begin
    ordinare l'insieme di lati E secondo costi non crescenti
    porre T*={insieme vuoto}
    while |T*|<n-1 do
        begin
            scegliere un lato "e" da E di costo massimo;
            Porre E=E-{e};
            if T*+{e} non ha cicli then
                T*=T*+{e}
            end if
        end
    end
    return T*;
end
```

7.3.2 Algoritmo di Prim

L'**algoritmo di Prim**, per la determinazione dell'albero di supporto di costo massimo, garantisce che la struttura ad ogni passo sia connessa (*a differenza di Kruskal che lo garantisce solo alla fine*).

PSEUDOCODICE DELL'ALGORITMO

Algoritmo Prim(V, E, T^*)

begin

 porre $T^* = \{\text{insieme vuoto}\}$

 porre $S = \{1\}$

 while $|T^*| < n-1$ do

 begin

 individuare un lato (v, h) in $\delta(S)$ di costo minimo

 Porre $T^* = T^* + \{(v, h)\}$

 porre $S = S + \{h\}$

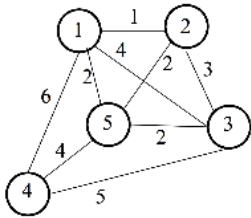
 end

 return T^* ;

end

SPIEGAZIONE DELL'ALGORITMO

Supponiamo di lavorare con il grafo seguente:



- Inizializziamo $T^* = \emptyset$ e $S = \{1\}$.
- I lati del taglio $\delta(S)$ sono $\{\{1, 2\}, \{1, 3\}, \{1, 4\}, \{1, 5\}\}$: il lato di costo minimo è $\{1, 2\}$ avendo costo unitario.
Allora si pone $T^* = \{\{1, 2\}\}$ e $S = \{1, 2\}$.
La procedura continua, essendo $|T^*| = 1 < n - 1 = 4$.
- Il taglio diventa $\delta(s) = \{\{1, 3\}, \{1, 4\}, \{1, 5\}, \{2, 3\}, \{2, 5\}\}$: il lato di costo minimo è $\{1, 5\}$ con costo 2.
Allora si pone $T^* = \{\{1, 2\}, \{1, 5\}\}$ ed $S = \{1, 2, 5\}$.
- Il taglio diventa $\delta(s) = \{\{1, 3\}, \{1, 4\}, \{2, 3\}, \{5, 3\}, \{5, 4\}\}$: il lato di costo minimo è $\{5, 3\}$ con costo 2.
Allora si pone $T^* = \{\{1, 2\}, \{1, 5\}, \{5, 3\}\}$ ed $S = \{1, 2, 3, 5\}$.
- Il taglio diventa $\delta(s) = \{\{1, 4\}, \{5, 4\}, \{3, 4\}\}$: il lato di costo minimo è $\{5, 4\}$ con costo 4.
Poiché $|T^*| = 4$, l'algoritmo si ferma ed in T^* abbiamo l'albero di supporto di costo minimo, con costo 9.
Interessante è notare che questa non è l'unica soluzione, infatti esistono altri alberi di supporto a costo minimo con lo stesso costo.

7.3.3 Lato di diminuzione

Sia $G_T = (V, T)$ un albero di supporto di $G = (V, E)$. Un lato $e \notin T$ è un **lato di diminuzione** di G_T se nel ciclo C che si viene a formare aggiungendolo a T esiste un lato $f \in C \setminus \{e\}$ con costo maggiore, cioè tale che $c_f > c_e$.

Data questa definizione, possiamo dare la seguente **condizione di ottimalità degli alberi**: «Un albero di supporto $G_T = (V, T)$ di $G = (V, E)$ è di costo minimo se e solo se non ha lati di diminuzione».

7.3.3.1 Procedura per la verifica di ottimalità

- Si scrivono tutti gli archi.
- Si cancellano dall'elenco gli archi dell'albero di supporto ottenuto.
- Si inserisce un arco, di quelli rimasti, nel grafo e si valuta il costo di ogni ciclo. Se il costo è più basso scegliendo l'arco inserito, allora l'albero di supporto che si era ottenuto non era ottimo.

Il numero di controlli che devono essere fatti per arrivare ad individuare un eventuale lato di diminuzione:

- Nel caso peggiore è $m - (n - 1)$, dove n è il numero di nodi ed m è il numero di lati del grafo.
- Dipende dall'ordine con cui i lati che non fanno parte dell'albero vengono considerati: una regola empirica consiste nel considerare i lati in ordine crescente di costo.

7.3.4 Applicazione del calcolo di alberi di supporto: diffusione ottimale di un messaggio

E' data una rete di comunicazione $G = (N, E)$ i cui lati $[i, j] \in E$ indicano quali coppie di «nodi» possono comunicare direttamente. Si conosce la probabilità $p_{i,j}$ (compresa tra 0 e 1) di intercettazione di un messaggio lungo ogni lato $[i, j] \in E$.

Si vuole **determinare come trasmettere a tutti i nodi un messaggio riservato** minimizzando la probabilità che sia intercettato (o massimizzando la probabilità di non-intercettazione):

$$\max \prod_{[i,j] \in T} (1 - p_{i,j})$$

Consideriamo l'albero di supporto T . Esso è *connesso*, in quanto il messaggio deve andare verso tutti i nodi, e deve essere *aciclico* per evitare ridondanza e probabilità di intercettazione più elevata.

Possiamo applicare una funzione monotona crescente alla funzione obbiettivo, senza modificarne le soluzioni ottime (*adattando opportunamente l'algoritmo di Prim*):

$$\max \log\left(\prod_{[i,j] \in T} (1 - p_{i,j})\right) = \max \sum_{[i,j] \in T} \log(1 - p_{i,j})$$

7.4 Cammino di costo minimo

Siano dati

- Un grafo $G = (N, A)$
- Una funzione di costo che associa il costo $c_{i,j}$ ad ogni arco $(i, j) \in A$

Si vuole trovare il cammino di costo minimo da un nodo origine s ad un nodo destinazione t , ossia il cammino da s a t per il quale è minima la somma degli archi che lo compongono.

Questo problema può essere risolto da tre algoritmi, ciascuno dei quali con requisiti diversi e complessità computazionali differenti:

1. *Algoritmo di Dijkstra*
2. *Metodo delle etichette*
3. *Algoritmo di Floyd-Warshall*

7.4.1 Algoritmo di Dijkstra

L'algoritmo di Dijkstra:

- Richiede un costo $c_{i,j} \geq 0$ per ogni arco $(i, j) \in A$.
- Esplora i nodi in ordine crescente del costo di cammino minimo da s a ciascuno di essi. Lavora quindi in modo *greedy*.

Esso si basa sull'associazione, ad ogni nodo, di etichette $L[j]$ (cioè il costo di un cammino minimo da s a j) e $Pred[j]$ (cioè il predecessore del nodo j nel cammino di costo minimo da s a j).

PSEUDOCODICE DELL'ALGORITMO

Algoritmo Dijkstra(N, A, c, s)

begin

```

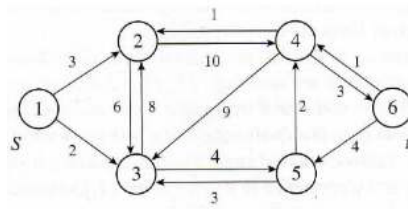
    porre  $S = \{s\}$ ,  $L[s] = 0$ ,  $Pred[s] = s$ ;
    while  $|S| \neq n$  do
        begin
            individuare  $(v, h)$  nel taglio uscente di  $S$ ;
            tale che  $L[v] + c(v, h) = \min \{L[i] + c(i, j) \mid (i, j) \text{ appartiene al taglio uscente di } S\}$ ;
            porre  $L[h] = L[v] + c(v, h)$ ;
            porre  $Pred[h] = v$ ;
            porre  $s = s \cup \{h\}$ ;
        end
    end

```

end

ESEMPIO APPLICATIVO

Supponiamo di lavorare con il grafo seguente:



Nell'inizializzazione, al nodo $s = 1$ sono assegnate le etichette $L[1] = 0$ e $Pred[1] = 1$ essendo nullo il costo del cammino minimo da s a se stesso, ed essendo il nodo 1 il predecessore di se stesso. Infine, l'insieme S dei nodi con etichette permanenti è $S = \{s\}$.

L'algoritmo di Dijkstra sceglie ad ogni iterazione l'arco $(v, h) \in \delta^+(S)$ tale che

$$L[v] + c_{v,h} = \min[L[i] + c_{v,h} : (i, j) \in \delta^+(S)]$$

e pone

$$L[h] = L[v] + c_{v,h}$$

$$Pred[h] = v$$

$$S = S \cup \{h\}$$

Ci si fermerà quando $S = N$, cioè quando tutti i nodi risultano etichettati.

COMPLESSITÀ COMPUTAZIONALE

La complessità totale dell'algoritmo è $O(nm)$ se l'arco (v, h) viene determinato mediante scansione esplicita degli m archi del grafo e scartando quelli che non appartengono a $\delta^+(S)$, essendo questa operazione effettuata $n = |N|$ volte.

Nel caso in cui si abbiano grafi **densi** la complessità dell'algoritmo è $O(n^3)$, anche se sfruttando strutture dati ad hoc è possibile scendere ad una complessità di $O(n^2)$.

CORRETTEZZA DELL'ALGORITMO DI DIJKSTRA

Dato un grafo orientato $G = (N, A)$ con una funzione di costo $c_{i,j}$ per ogni arco $(i, j) \in A$ ed $s \in N$, l'algoritmo di Dijkstra fornisce l'insieme dei cammini di costo minimo da s a tutti gli altri nodi.

Dimostrazione

La dimostrazione viene fatta per induzione sul numero di iterazioni dell'algoritmo: *vogliamo dimostrare che alla p -esima iterazione $L[j]$ rappresenta il costo di un cammino minimo da s a j per tutti i nodi $j \in S = \{i_0, \dots, i_p\}$ (essendo $i_0 = s$ ed i_p il nodo aggiunto ad S alla p -esima iterazione).*

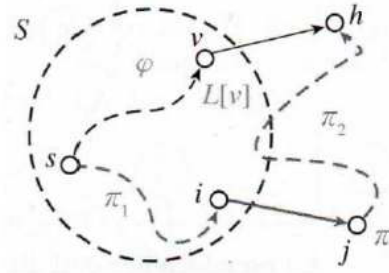
- Per prima cosa dimostriamo il **passo base dell'induzione**:

Se $p = 0$, allora $S = \{s\}$ e $L[s] = 0$ è effettivamente il costo minimo da s verso se stesso, dato che per ipotesi abbiamo i costi di tutti gli archi non negativi.

- Ora dimostriamo che se la tesi è vera alla p -esima iterazione, allora è vera anche per l'iterazione $p + 1$:

Sia $h \in S$ il nodo che viene inserito in S all'iterazione $p + 1$ e sia φ il cammino da s ad h tale che $L[v] + c_{v,h} = \min\{L[i] + c_{i,j} : (i, j) \in \delta^+(S)\}$. Occorre ora verificare che qualunque cammino π da s ad h ha costo non inferiore a quello del cammino φ .

Infatti esistono nodi $i \in S$ e nodi $j \notin S$ tali per cui $\pi = \pi_1 \cup (i, j) \cup \pi_2$ dove (i, j) è il primo arco di π a far part del taglio $\delta^+(S)$.



Risulta quindi $c(\pi) = c(\pi_1) + c_{i,j} + c(\pi_2)$. Per ipotesi d'induzione, $c(\pi_1) \geq L[i]$ essendo la tesi vera per $i \in S$. Inoltre, $c(\pi_2) \geq 0$ dato che, da ipotesi, tutti i costi devono essere positivi.

Allora vale che $c(\pi) = c(\pi_1) + c_{i,j} + c(\pi_2) \geq L[i] + c_{i,j} \geq L[v] + c_{v,h} = c(\varphi)$ cioè è dimostrato che φ è un cammino di costo minimo da s ad h .

7.4.2 Algoritmo delle etichette

L'**algoritmo delle etichette**, noto anche come algoritmo di **programmazione dinamica**, può essere applicato solo a **GRAFI ORIENTATI SENZA CICLI** (*non era richiesto da Dijkstra*). In questo caso, allora, i nodi possono essere ordinati in **ordine topologico** cioè in modo che

$$\forall (i, j) \in A \quad \text{risulta} \quad i < j$$

Questa prima fase viene eseguita con l'*algoritmo di ordinamento topologico*.

PSEUDOCODICE DELL'ALGORITMO DI ORDINAMENTO TOPOLOGICO

Algoritmo Ordinamento Topologico(N,A)

begin

 porre $k=1$;

 while N diverso da insieme vuoto do

 begin

 scegliere un nodo v in N con $BS(v)$ diverso da insieme vuoto;
 (se un tale nodo non esiste, siamo in presenza di un
 ciclo, quindi l'algoritmo si ferma)

 assegnare il numero k al nodo v ;

 porre $N=N \setminus \{v\}$;

 porre $A=A \setminus BS(v)$;

 porre $k=k+1$;

 end

end

In pratica, ad ogni iterazione, si assegna al nodo che ha solo archi uscenti (senza considerare gli archi provenienti da nodi già etichettati) l'etichetta k .

DETERMINAZIONE DI UN ALBERO DI COSTO MINIMO

In un grafo senza cicli, con nodi ordinati in modo topologico, supponiamo di voler determinare il cammino minimo dal nodo $s = 1$ al nodo t . Sia π_t un cammino di costo minimo dal nodo 1 a t .

Sia, ora, i il penultimo nodo di tale cammino.

A questo punto, il sottocammino dal nodo 1 al nodo i dovrà essere necessariamente di costo minimo, altrimenti π_i non sarebbe un cammino minimo.

Di conseguenza, $\pi_t = \pi_i \cup (i, t)$ dove π_i è il cammino di costo minimo dal nodo 1 al nodo i .

Quindi, definendo con $f(i)$ il costo di un cammino minimo dal nodo 1 al nodo i , $f(t)$ si ottiene considerando tutti i possibili predecessori del nodo t , ossia:

$$f(t) = \min_{i < t} \{f(i) + c_{i,t}\}$$

In modo analogo

$$f(t-1) = \min_{i < t-1} \{f(i) + c_{i,t-1}\}$$

...

$$f(1) = 0$$

Tali relazioni possono essere risolte in ordine inverso (da $f(1)$ a $f(t)$), ottenendo le *regole di aggiornamento* nell'algoritmo delle etichette.

Infatti, il **precedessore** di ogni nodo nel cammino di costo minimo viene determinato memorizzando, di volta in volta, il nodo che permette di realizzare l'etichetta minima

$$\text{pred}[t] = \arg \min_{i < t} \{f(i) + c_{i,t}\}$$

Nei grafi senza cicli si possono anche calcolare i **cammini di costo massimo**. E' sufficiente sostituire nella formula $f(t) = \min_{i < t} \{f(i) + c_{i,t}\}$ il minimo con il massimo, ottenendo

$$f(t) = \max_{i < t} \{f(i) + c_{i,t}\}$$

COMPLESSITÀ DELL'ALGORITMO DELLE ETICHETTE

La prima fase, obbligatoria, è quella dell'*ordinamento topologico* che richiede $O(m)$ operazioni, con $m = |A|$, se il grafo è memorizzato con la lista dei predecessori e dei successori di ogni nodo. Infatti, una volta numerato un nuovo v con $|BS(v)| = 0$, occorre decrementare di un'unità $|BS(h)|$ per ogni nodo h successore di v .

Dato che la stella di archi uscente dal nodo v viene cancellata, il numero totale di tali operazioni di decremento è $O(m)$, dato che ogni arco contribuisce ad un solo decremento.

7.4.3 Algoritmo di Floyd-Warshall

L'algoritmo di **Floyd-Warshall** permette di calcolare i cammini di costo minimo tra ogni coppia di nodi in un grafo qualunque, quindi anche in tutti i casi in cui i due algoritmi precedenti non funzionano. In particolare funziona anche se sono presenti:

- Archi di costo negativo
- Cicli

Nonostante questi vantaggi, questo algoritmo è molto più complicato dei precedenti, quindi viene usato solo nei casi in cui Dijkstra e l'algoritmo delle etichette non funzionano.

Consideriamo, quindi, grafi con cicli ed archi di costo negativo: in questo caso il problema della determinazione del percorso di costo minimo potrebbe non essere ben definito perché *se il grafo ha un ciclo di costo negativo, si avrebbe un problema inferiormente illimitato, cioè sarebbe conveniente percorrere all'infinito questo ciclo.*

INPUT DELL'ALGORITMO

Gli input ricevuti dall'algoritmo sono:

- Un grafo orientato $G = (N, A)$.
- Una matrice C di dimensione $n \times n$ contenente i costi.

OUTPUT DELL'ALGORITMO

L'algoritmo fornisce in output:

- Una matrice D di dimensione $n \times n$ in cui l'elemento (i, j) rappresenta il costo di un cammino dal nodo i al nodo j .
- Una matrice P di dimensione $n \times n$ in cui l'elemento (i, j) rappresenta il predecessore di j nel cammino da i a j .

PSEUDOCODICE DELL'ALGORITMO

Algoritmo Floyd-Warshall (N, A, c)

begin

for $i=1$ to n do

for $j=1$ to n do

begin

porre $d(i, j) = c(i, j)$;

porre $p(i, j) = i$;

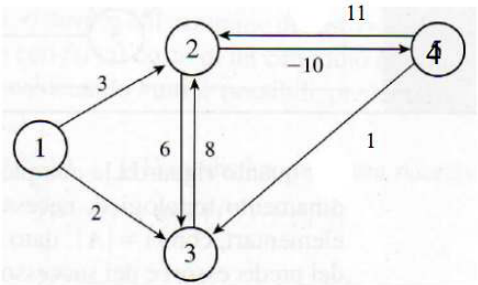
```

    end
    for h=1 to n do
    begin
        for i=1 to n do
            for j=1 to n do
                if d(i,j)+d(h,j)<d(i,j) then
                begin
                    porre d(i,j)=d(i,h)+d(h,j);
                    porre p(i,j)=p(h,i);
                end
            end
        end
        for i=1 to n do
            if d(i,i)<0 then STOP; //Ciclo negativo
        end
    end
end

```

ESEMPIO DI APPLICAZIONE

Vogliamo determinare il cammino di costo minimo dal nodo 1 al nodo 4 nel grafo G (con cicli di costo negativo, quindi Dijkstra ed il metodo delle etichette non possono essere usati):



Per prima cosa costruiamo la matrice dei costi C associata al grafo:

$$C = \begin{bmatrix} 0 & 3 & 2 & \infty \\ \infty & 0 & 6 & -10 \\ \infty & 8 & 0 & \infty \\ \infty & 11 & 1 & 0 \end{bmatrix}$$

All'inizio della prima iterazione, la matrice D coincide con C ed ogni elemento della matrice P è $p_{i,j} = 1, \forall i \in N$:

$$P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix}$$

All'iterazione h -esima viene eseguita la seguente **operazione triangolare** relativa al nodo h : per ogni coppia di nodi i, j si controlla se l'unione del migliore cammino trovato sino a quel momento dal nodo i al nodo h con quello dal nodo h al nodo j fornisce un cammino di costo complessivo inferiore a quello attuale, ossia si verifica se $d_{i,h} + d_{h,j} < d_{i,j}$.

Qualora tale condizione sia verificata si pone $d_{i,j} = d_{i,h} + d_{h,j}$ e $p_{i,j} = p_{h,j}$ perché ora il predecessore del nodo j nel migliore cammino sinora trovato da i a j coincide con il predecessore di j nel migliore cammino sinora trovato da h a j .

Se non si trova nessun elemento che soddisfa tale condizione, si lasciano inalterate le matrici D e P , e si incrementa h .

Dopo l'esecuzione di tutte le h iterazioni, se sulla diagonale della matrice D si riscontrassero elementi negativi, l'algoritmo terminerebbe immediatamente dato che avremmo individuato un ciclo di costo negativo, e quindi il problema risulterebbe inferiormente illimitato.

CORRETTEZZA DELL'ALGORITMO

La correttezza dell'algoritmo può essere dimostrata per induzione matematica sul numero di iterazioni.

COSTO COMPUTAZIONALE

Il costo computazionale dell'algoritmo è $O(n^3)$ perché nel caso peggiore l'operazione *triangolare* viene eseguita su tutti gli n nodi h , comportando la scansione di ogni coppia di nodi i e j (n^2 coppie).

7.5 Esempio di calcolo dei cammini minimi: Ispezioni su una linea di produzione

Supponiamo di avere una linea di produzione con n celle di lavorazione. Attualmente la produzione è fatta attraverso la seguente modalità:

- Si introduce nella linea di produzione un lotto costituito da B pezzi «senza difetti».
- In ognuna delle n celle, il lotto subisce una lavorazione con costo (*in ogni cella j*) pari al prodotto del prezzo unitario p_j di lavorazione nella cella per il numero B di pezzi.

Il costo complessivo di lavorazione del lotto è quindi $B \cdot \sum_{j=1}^n p_j$.

- Può accadere che nel corso delle lavorazioni, alcuni pezzi divengano difettosi. Essi vengono lasciato comunque nel lotto fino al termine della lavorazione nella cella n e subiscono comunque le lavorazioni nelle celle successive a quella in cui si è prodotto il difetto.

Quindi un pezzo divenuto difettoso nel corso della lavorazione $j < n$ viene sottoposto alle lavorazioni da $j + 1$ ad n .

- Alla fine di tutte le lavorazioni viene effettuata una ispezione per scartare i pezzi che sono diventati difettosi durante le n lavorazioni.

A questo punto, una variante potrebbe essere quella che prevede di controllare i pezzi alla fine della lavorazione $j < n$ in modo da non procedere nella lavorazione dei prodotti che sono in realtà danneggiati e che quindi non possono essere consegnati al cliente. Il numero di elementi che verrebbe passato dalla lavorazione j alla lavorazione $j + 1$ viene indicato con B_j .

Per introdurre questa lavorazione si devono sostenere:

- Un costo fisso, indipendente dal numero di elementi ispezionati.
- Un costo variabile, proporzionale al numero di elementi ispezionati, la cui entità dipende da quando è stata effettuata l'ispezione immediatamente precedente.

Valutiamo ora il **costo delle ispezioni intermedie**. Il costo di una ispezione fatta alla lavorazione j quando l'ultima ispezione è stata fatta alla lavorazione $i < j$ dipende da:

- $f_{i,j}$ cioè il costo fisso.
- $h_{i,j}$ cioè il costo per l'elemento ispezionato.

Se invece nella cella j viene effettuata la prima ispezione intermedia i costi saranno:

- $f_{0,j}$ cioè il costo fisso.
- $h_{0,j}$ cioè il costo per l'elemento ispezionato.

Denotiamo con B_i il numero di elementi non difettosi, che restano in lavorazione all'uscita dalla lavorazione i , $0 \leq i \leq n$. A questo punto, se la successiva ispezione viene effettuata nell'attività j , dall'attività i all'attività j sono stati lavorati B_i pezzi, per un costo totale di produzione pari a:

$$B_i \sum_{k=i+1}^j p_k$$

Il costo dell'ispezione fatta all'uscita dalla cella j sarà invece pari a

$$f_{ij} + B_i h_{ij}$$

Di conseguenza, il costo totale di attività produttiva ed ispezione, tra i e j è dato da:

$$c_{ij} = f_{ij} + B_i(h_{ij} + \sum_{k=i+1}^j p_k)$$

Con la variante, per la prima ispezione, che sarà

$$c_{0j} = f_{0j} + B_0(h_{0j} + \sum_{k=0+1}^j p_k)$$

Quindi il costo totale del segmento temporale $(i + 1) - j$ dipende dal numero di elementi non difettosi alla fine della lavorazione i .

Per determinare i valori B_i :

- Si raccolgono osservazioni per un certo periodo, in base alle quali si stima la probabilità α_i che un pezzo non difettoso, diventi difettoso durante la lavorazione i .
- Note queste probabilità ed il numero di pezzi inizialmente presenti nel lotto, il **numero atteso di pezzi non difettosi** alla fine della lavorazione i è

$$B_i = B \prod_{k=1}^i (1 - \alpha_k)$$

Ricordiamo inoltre che le ispezioni fatte prima dell'inserimento del lotto e alla fine della lavorazione n sono sempre effettuate, quindi:

- Il costo dell'ispezione prima dell'inserimento del lotto dipende solo da B , quindi fissato tale valore il costo è costante.
- Il costo dell'ispezione alla fine della lavorazione n dipende da quando è stata effettuata la lavorazione precedente.

CASO SPECIFICO CON $n = 4$

I possibili piani delle ispezioni possono essere 8 (2^{n-1} , quindi crescono esponenzialmente):

1. Nessuna ispezione;
2. Ispezione dopo l'attività 1;
3. Ispezione dopo l'attività 2;
4. Ispezione dopo l'attività 3;
5. Ispezione dopo le attività 1 e 2;
6. Ispezione dopo le attività 1 e 3;
7. Ispezione dopo le attività 2 e 3;
8. Ispezione dopo le attività 1, 2 e 3;

A questo punto calcoliamo i vari costi nei vari scenari, tralasciando il costo della prima ispezione (*che come abbiamo visto è costante*).

Ci troviamo quindi di fronte ad un piano di ispezione che minimizza i costi di produzione e ispezione del tipo

$$\min \text{costo} = c_{0,1}x_{0,1} + c_{0,2}x_{0,2} + \dots$$

dove $x_{i,j}$ è una variabile binaria che indica la necessità o meno di eseguire una ispezione tra l'attività i e j .

I vincoli a cui siamo sottoposti sono:

$$\begin{cases} x_{0,1} + x_{0,2} + x_{0,3} + x_{0,4} = 1 \\ x_{0,1} = x_{1,2} + x_{1,3} + x_{1,4} \\ x_{0,2} + x_{1,2} = x_{2,3} + x_{2,4} \\ x_{0,3} + x_{1,3} + x_{2,3} = x_{3,4} \\ x_{0,4} + x_{1,4} + x_{2,4} + x_{3,4} = 1 \end{cases}$$

Il problema di determinare il piano di ispezione ottimo che minimizza il costo totale, è equivalente al problema di determinare un cammino di costo minimo dal nodo 0 al nodo n su un grafo $G = (N, A)$ in cui $N = \{0, 1, 2, \dots, n\}$ ed $A = \{(i, j) : 0 \leq i \leq n-1, i+1 \leq j \leq n\}$.

In questo grafo avremo associato ad ogni arco (i, j) il costo $c_{i,j}$!

Capitolo 8

Problema del flusso massimo

Si dice **rete di flusso** un grafo orientato $G = (N, A)$ con

- capacità $u_{i,j}$ associata ad ogni arco $(i, j) \in A$.
- un nodo $s \in N$ detto **nodo sorgente**.
- un nodo $t \in N$ detto **pozzo**.

Si vuole quindi trovare la massima quantità di flusso che può essere mandata da s a t in modo che il flusso su ogni arco non superi la capacità dell'arco stesso e che, in ogni nodo, il flusso entrante sia uguale al flusso uscente.

8.1 Cosa è il flusso

E' detto **flusso** un vettore $\underline{x} = [x_{i,j}]$ che soddisfa le equazioni di bilancio ai nodi intermedi. Si parla di **flusso ammissibile** se soddisfa anche i vincoli di non-negatività e di capacità degli archi.

Si dice **valore del flusso** (indicato con v) la differenza tra il *totale dei flussi sugli archi uscenti dal nodo sorgente* ed il *totale dei flussi sugli archi entranti nel nodo sorgente*.

8.2 Formulazione PL del problema di massimo flusso

Per prima cosa individuiamo le variabili decisionali del problema:

- $x_{i,j}$: flusso sull'arco (i, j) .
- v : quantità immessa nel nodo s e prelevata al nodo t , cioè il valore del flusso (*modulo del peso dei nodi origine e destinazione*)

Abbiamo detto prima che deve valere la **conservazione del flusso**, cioè il flusso entrante deve essere uguale a quello uscente. Questa considerazione è necessaria per stabilire i vincoli funzionali del problema.

A questo punto, per il *nodo sorgente* si ha:

$$\text{flusso entrante} + \text{quantità immessa al nodo (da massimizzare)} = \text{flusso totale uscente}$$

$$\sum_{(j,s) \in BS(s)} x_{j,s} + v = \sum_{(s,j) \in FS(s)} x_{s,j}$$

Invece, per quanto riguarda i *nodi di trasferimento* $\forall i \neq s, t$ si ha:

$$\text{flusso entrante} = \text{flusso uscente}$$

$$\sum_{(j,i) \in BS(i)} x_{j,i} = \sum_{(i,j) \in FS(i)} x_{i,j}$$

Per ultimo, per il *nodo pozzo* deve valere:

$$\text{flusso entrante} = \text{quantità prelevata dal nodo (da massimizzare)} + \text{flusso uscente}$$

$$\sum_{(j,t) \in BS(t)} x_{j,t} = v + \sum_{(t,j) \in FS(t)} x_{t,j}$$

Gli ultimi vincoli che devono essere considerati sono quelli di **non negatività del flusso** e quelli di **capacità massima**:

$$0 \leq x_{i,j} \leq u_{i,j} \quad \forall (i,j) \in A$$

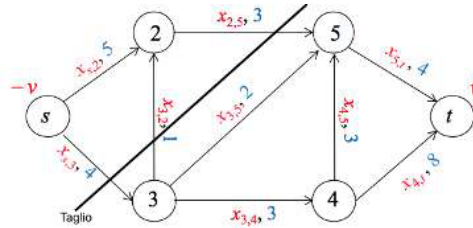
In base a quanto visto fino ad ora, la funzione obiettivo sarà semplicemente

$$\max \omega = v$$

Essendo un algoritmo di PL, esso può essere risolto con l'algoritmo del semplice, ma esistono algoritmi più efficienti (come **Ford-Fulkerson**) che si basano sulla struttura a grafo sulla quale il problema è definito.

8.2.1 Esempio di modello per problema del massimo flusso

Consideriamo il grafo seguente:



Allora il modello da ottimizzare, costruito facendo i bilanci dei flussi sui singoli nodi, è:

$$\left\{ \begin{array}{l} \max \quad \omega = v \\ \text{s.a.} \quad -x_{s,2} - x_{s,3} = -v \\ \quad x_{s,2} + x_{3,2} - x_{2,5} = 0 \\ \quad x_{s,3} - x_{3,2} - x_{3,4} - x_{3,5} = 0 \\ \quad x_{3,4} - x_{4,5} - x_{4,t} = 0 \\ \quad x_{2,5} + x_{3,5} + x_{4,5} - x_{5,t} = 0 \\ \quad x_{4,t} + x_{5,t} = v \\ \quad 0 \leq x_{s,2} \leq 5 \quad 0 \leq x_{3,2} \leq 1 \quad 0 \leq x_{4,5} \leq 3 \\ \quad 0 \leq x_{s,3} \leq 4 \quad 0 \leq x_{3,4} \leq 3 \quad 0 \leq x_{4,t} \leq 8 \\ \quad 0 \leq x_{2,5} \leq 3 \quad 0 \leq x_{3,5} \leq 2 \quad 0 \leq x_{5,t} \leq 4 \end{array} \right.$$

Come indicato nello schema del grafo, abbiamo individuato un **taglio** e, di conseguenza, possiamo individuare gli **archi di taglio**:

$$(s, 3), (3, 2), (s, 5)$$

Gli archi **diretti** del taglio $((s, 3), (s, 5))$ dovranno essere completamente saturi nella soluzione ottima, mentre quelli **inversi** $((3, 2))$ dovranno essere completamente vuoti nella soluzione ottima.

8.3 Definizioni

Data una rete di flusso $G = (N, A)$ con capacità $u_{i,j}$ sugli archi ed un flusso ammissibile $\underline{x}$, dove $x_{i,j}$ è il valore del flusso assegnato all'arco (i, j) , un arco si dice **saturo** se $x_{i,j} = u_{i,j}$ e si dice **scarico** se $x_{i,j} = 0$.

Data una rete di flusso $G = (N, A)$ con capacità $u_{i,j}$ sugli archi ed un flusso ammissibile $\underline{x}$, il **grafo residuale** (o rete incrementale) si ottiene da G sostituendo ogni arco $(i, j) \in A$ con due archi:

- Un arco diretto (i, j) di capacità residua $\bar{u}_{i,j} = u_{i,j} - x_{i,j}$ se (i, j) non è saturo.
- Un arco inverso (j, i) di capacità residua $\bar{u}_{u,j} = x_{i,j}$ se (i, j) non è scarico.

8.4 Algoritmo di Ford-Fulkerson

L'**algoritmo di Ford-Fulkerson** risolve il problema del massimo flusso in modo incrementale: dato un certo flusso iniziale, l'algoritmo verifica se tale flusso è migliorabile, ossia se è possibile inviare ulteriore flusso dalla sorgente s al pozzo t .

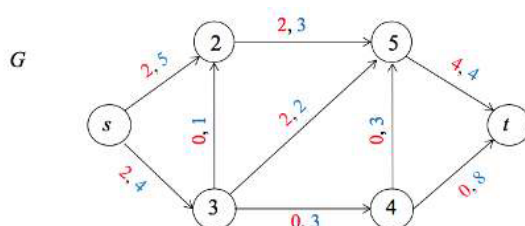
Per stabilire se tale miglioramento è possibile, viene utilizzato il **grafo residuale** $\bar{G}(x) = (N, \bar{A})$. In questo modo, il grafo residuale tiene conto della possibilità di:

- Aumentare il flusso sugli archi non saturi (dato che danno origine ad archi diretti con capacità residua non nulla).
- Diminuire il flusso sugli archi non scarichi (dato che danno origine ad archi inversi con capacità residua non nulla)

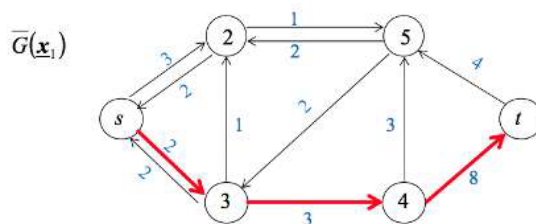
A questo punto, un cammino da s a t sul grafo residuale corrisponde alla possibilità di aumentare il flusso corrente e per questo viene chiamato **cammino aumentante**.

ATTENZIONE: il massimo aumento di flusso su un cammino aumentante è pari al minimo delle capacità residue degli archi che lo compongono.

Ad esempio, considerando la rete di flusso in figura:



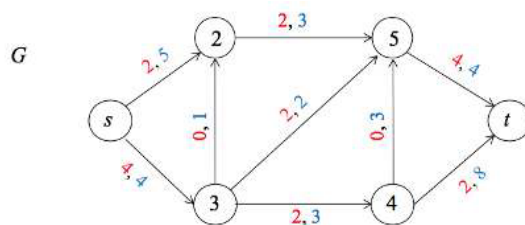
E' possibile costruire il grafo residuale associato al flusso iniziale x_1



Su questo grafo residuale è possibile aumentare il flusso, per esempio con il *cammino aumentante* $s, 3, 4, t$ di $\theta = \min\{2, 3, 8\} = 2$.

θ indica quindi la massima quantità che aggiunta agli archi diretti non produce flussi maggiori delle capacità e che sottratta agli archi inversi non produce flussi negativi.

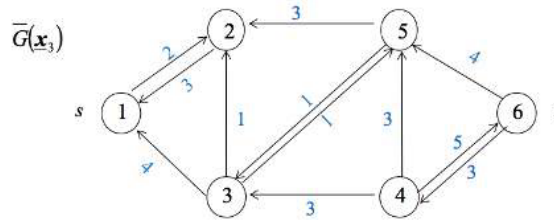
Poiché gli archi che compongono il cammino sono tutti diretti, il nuovo flusso x_2 ($v = 4 + 2 = 6$) si ottiene sommando θ a tutti i flussi precedenti sugli archi di G corrispondenti al cammino aumentante:



A questo punto bisogna testare l'ottimalità della soluzione, costruendo una nuova *rete incrementale* e controllando se su di essa è possibile costruire un nuovo *cammino aumentante*.

Se nel cammino aumentante trovato si ha la presenza di un arco inverso non presente nel grafo originale, al traffico sull'arco (i, j) , a cui corrisponde l'arco inverso, bisognerà sottrarre il valore θ trovato per il cammino.

Nella situazione in analisi, l'ultimo grafo residuale sarebbe:



Su tale grafo non è possibile trovare un cammino aumentante da s a t , infatti gli unici nodi raggiungibili dalla sorgente sono $\{s, 2\}$, dai quali poi non si può più proseguire.

Risulta così determinato il taglio s, t con $N_s = \{s, 2\}$ e $N_t = N \setminus N_s$:

- Gli archi diretti del taglio sono *saturi*.
- Gli archi inversi del taglio sono *scarichi*.

Quindi il taglio trovato ha la proprietà di avere flusso che lo attraversa uguale al valore della capacità:

$$X(N_s, N_t) = \sum_{(i,j) \in \delta^+(N_s)} x_{i,j} - \sum_{(i,j) \in \delta^-(N_s)} x_{i,j} = \sum_{(i,j) \in \delta^+(N_s)} u_{i,j} - 0 = U(N_s, N_t)$$

Tale taglio ha capacità minima e flusso di valore massimo. Questa proprietà è sempre verificata dall'algoritmo di *Ford-Fulkerson*.

PSEUDOCODICE DELL'ALGORITMO DI FORD-FULKERSON

Algoritmo Ford-Fulkerson

begin

$x=0$; // x -vettore

$v=0$;

 ottimo = false;

 while (ottimo=false) do

 begin

 Costruire il grafo residuale $G(x)$ associato al flusso x ;

 determinare, se esiste, un cammino aumentante P da s a t in $G(x)$;

 if P non esiste then

 ottimo=true;

 else

 begin

 calcolare $\text{Theta} = \min\{u(i,j) \text{ tale che } (i,j) \text{ appartiene a } P\}$;

 porre $v = v + \text{Theta}$;

 for each (i,j) in A do

 if (i,j) è un arco diretto then

 porre $x(i,j) = x(i,j) + \text{Theta}$

 else

 porre $x(i,j) = x(i,j) - \text{Theta}$

 end

 end

end

COMPLESSITÀ COMPUTAZIONALE DELL'ALGORITMO

Consideriamo il caso peggiore in cui, partendo da una rete scarica, si deve determinare un flusso massimo della seguente rete:

- Sorgente collegata direttamente agli $n - 2$ nodi intermedi.
- Ogni nodo intermedio è collegato direttamente al pozzo t .
- Tutti gli archi hanno capacità pari ad U .

In questo caso il flusso massimo ha valore $(n - 2) \cdot U$ (*capacità del taglio*). Se le capacità degli archi sono interi, nel caso peggiore ad ogni iterazione il flusso può aumentare lungo il cammino aumentante di una sola unità, quindi nel caso peggiore si avranno $(n - 2) \cdot U$ iterazioni.

Il costo computazionale totale sarà $O(m \cdot n \cdot U)$ in quanto ad ogni iterazione si deve trovare il cammino aumentante con costo $O(m)$. Questa complessità è dipendente non solo dalla dimensione dell'istanza ma anche da U e quindi dai dati del problema. Si parla di complessità *pseudopolinomiale*.

8.4.1 Teorema del massimo flusso-minimo taglio

Data una rete di flusso $G = (N, A)$ il valore del flusso massimo coincide con la capacità minima dei tagli $s - t$.

Infatti, essendo $v = x(N_s, N_t) = u(N_s, N_t)$ non può esistere un taglio (N'_s, N'_t) del grafo di capacità $u'(N'_s, N'_t)$ strettamente minore di $u(N_s, N_t)$: infatti, se un tale taglio esistesse, si avrebbe $u'(N'_s, N'_t) < x(N_s, N_t) = v$. Questo sarebbe in contraddizione con $u(N_s, N_t) \geq x(N_s, N_t) \quad \forall (N_s, N_t)$.

8.4.2 Determinazione del cammino aumentante

Un cammino aumentante può essere determinato mediante una visita del grafo residuo $G_{\underline{x}}$ a partire da s . Si possono presentare due casi:

- *La visita raggiunge t* : risulta individuato un cammino aumentante che permette di costruire un nuovo flusso $\underline{x}'$ di valore maggiore rispetto al flusso $\underline{x}$. Si è quindi dimostrata la non ottimalità del flusso $\underline{x}$ trovato fino a questo momento.
- *La visita non raggiunge t* : si definisce sull'insieme dei nodi N una partizione (N_s, N_t) dove il primo è il sottoinsieme dei nodi raggiunti dalla visita (*compreso il nodo s*) mentre il secondo è il sottoinsieme dei nodi non raggiunti dalla visita (*compreso il nodo t*). Se si arriva a questo punto, allora

$$v = u(N_s, N_t)$$

Il cammino aumentante deve essere scelto correttamente perché altrimenti il numero di iterazioni dell'algoritmo potrebbe diventare estremamente elevato.

Per superare questi problemi è presente una variante dell'algoritmo, sviluppata da **Edmonds e Karp** che consiste in un semplice accorgimento nella scelta del cammino aumentante: si deve sempre scegliere il cammino aumentante composto dal minor numero di archi. Questa variante ha un costo di $O(m)$ applicando un algoritmo di visita del grafo e gestendo i nodi visitati in un coda.ù

In questo modo la complessità totale dell'algoritmo con la variante passa da $O(mn)$ a $O(m^2n)$ quindi si ha una complessità polinomiale.

8.5 Esempio pratico di massimo flusso: abbinamento di cardinalità massima con vincoli di compatibilità

Dati i vincoli di questa tabella

	1	2	3	4	5
A			X		X
B	X	X		X	
C			X	X	
D			X	X	

formare le coppie (i, j) con $i \in \{A, B, C, D\}$ e $j \in \{1, 2, 3, 4, 5\}$ con il vincolo che ogni elemento i e j compaia al più in una coppia, in modo che il numero di coppie sia massimo?

Il problema si modella mediante un grafo bipartito in cui:

- Un insieme di nodi rappresenta il primo elemento della coppia $i \in \{A, B, C, D\}$
- Un insieme di nodi rappresenta il secondo elemento della coppia $j \in \{1, 2, 3, 4, 5\}$
- Un arco (i, j) collega un nodo i del primo insieme con il nodo j del secondo se e solo se i e j sono compatibili

Oltre a questi aggiungiamo:

- Un nodo sorgente s , collegato con ogni nodo del primo insieme con archi del tipo (s, i)
- Un nodo pozzo t , collegato con ogni nodo del secondo insieme con archi (j, t)

Ad ogni arco assegnamo capacità 1. Vogliamo determinare il flusso massimo da s a t .

8.6 Problema del flusso a costo minimo

E' data una rete di flusso $G = (N, A)$ dove

- Per ogni arco $(i, j) \in A$ sono dati una capacità $u_{i,j}$ ed un costo unitario $c_{i,j}$ per ogni unità di flusso che vi transita.
- Ad ogni nodo $i \in N$ è associato un valore b_i , detto *bilancio del nodo*, che rappresenta la differenza tra il flusso totale entrante nel nodo ed il flusso totale da esso uscente:
 - Se $b_i < 0$ il nodo è origine o sorgente, in quanto mette a disposizione del flusso.
 - Se $b_i > 0$ il nodo è destinazione o pozzo, in quanto richiede del flusso.
 - Se $b_i = 0$ il nodo è detto di *transito*, perché non richiede né offre flusso.

Il **problema del flusso a costo minimo** consiste nel determinare la distribuzione del flusso sulla rete in modo che:

- Dalle sorgenti esca tutti il flusso a disposizione.
- Alle destinazioni arrivi tutto il flusso richiesto.
- Le capacità degli archi non siano superate.
- Il costo complessivo sia minimo.

La prima cosa da fare è trovare un *flusso ammissibile iniziale per il problema di flusso di costo minimo*. Tale operazione non è banale, in quanto il flusso nullo non va bene, perché potrebbe violare i bilanci dei nodi.

Condizione necessaria (ma non sufficiente) per l'esistenza di un flusso ammissibile è che il problema sia bilanciato

$$\sum_{i \in N: b_i > 0} b_i = \sum_{i \in N: b_i < 0} b_i$$

Se tale condizione è verificata, allora il flusso ammissibile iniziale può essere cercato risolvendo un problema di flusso massimo sul grafo così composto:

- Si aggiungono al grafo G una sorgente fittizia s ed un pozzo fittizio f .
- Si collega s con ogni nodo avente bilancio negativo, assegnando a tale arco (s, i) la capacità $-b_i$.
- Si collega ogni nodo avente bilancio positivo con t , assegnando a tale arco (i, t) capacità b_i .
- Per tutti gli altri archi la capacità coincide con quella originaria.

Se risolvendo il problema di flusso massimo da s a t si ottiene un valore di flusso inferiore a $\sum_{i \in N: b_i > 0} b_i$ allora il problema è inammissibile perché le capacità degli archi risultano incompatibili con i bilanci dei nodi.

Come nel caso del problema del massimo flusso, utilizziamo le variabili $x_{i,j}$ per rappresentare le quantità di flusso che transitano su ogni arco $(i, j) \in A$ ed otteniamo il seguente problema di PL:

$$\begin{cases} \min & \varphi = \sum_{(i,j) \in A} c_{i,j} x_{i,j} \\ \text{s.a.} & \sum_{(j,i) \in BS(i)} x_{j,i} - \sum_{(i,j) \in FS(i)} x_{i,j} = b_i \quad \forall i \in N \\ & 0 \leq x_{i,j} \leq u_{i,j} \quad \forall (i,j) \in A \end{cases}$$

Questa formulazione può essere riscritta in forma più compatta utilizzando la matrice di **incidenza** E del grafo G ed inserendo

- I costi $c_{i,j}$ nel vettore $\underline{c} \in \mathbb{Z}^m$
- Le capacità $u_{i,j}$ nel vettore $\underline{u} \in \mathbb{Z}^m$
- I bilanci $b_{i,j}$ nel vettore $\underline{b} \in \mathbb{Z}^n$
- Le variabili $x_{i,j}$ nel vettore $\underline{x} \in \mathbb{Z}^m$

dove $m = |A|$ ed $n = |N|$:

$$\begin{cases} \min & \underline{c}^T \underline{x} \\ \text{s.a.} & E\underline{x} = \underline{b} \\ & \underline{0} \leq \underline{x} \leq \underline{u} \end{cases}$$

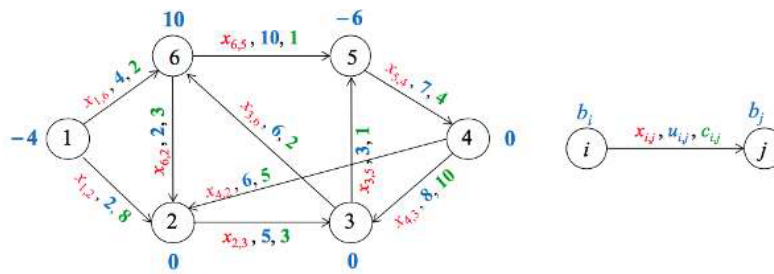
Essendo E una matrice *totalmente unimodulare* ed i termini noti interi, allora la soluzione ottima è intera, senza la necessità di richiederlo esplicitamente.

Non risulta però conveniente risolverlo come un problema di programmazione lineare ma conviene sfruttare la struttura di grafo su cui il problema è definito. Introdurremo l'**algoritmo di eliminazione dei cicli di costo negativo**.

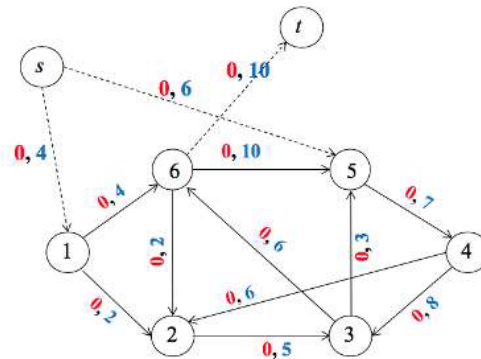
8.6.1 Eliminazione dei cicli di costo negativo

Dato un flusso ammissibile $\underline{x}$ si considera il **grafo residuale** $G(\underline{x}) = (N, A)$ al quale, in aggiunta a quanto visto in precedenza, aggiungiamo per ogni arco un costo residuale $\bar{c}_{i,j} = \begin{cases} c_{i,j} & \text{se l'arco è diretto} \\ -c_{i,j} & \text{se l'arco è inverso} \end{cases}$.

Consideriamo il grafo in figura:



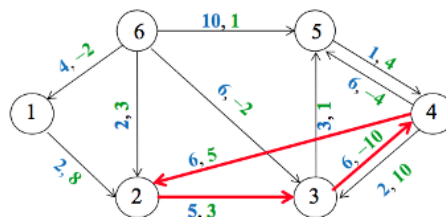
Per determinare un flusso ammissibile iniziale costruiamo la rete nella figura successiva, con sorgente fittizia s e pozzo fittizio t sulla quale risolvere il problema di flusso massimo da s a t :



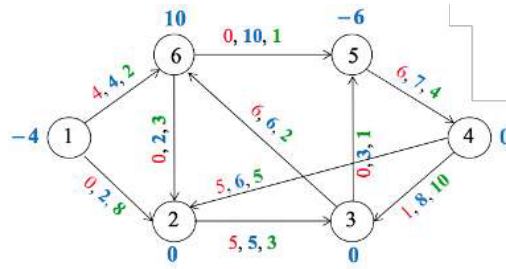
A questo punto si risolve il problema di massimo flusso sul grafo, ottenendo una soluzione ottima che soddisfa:

- Le capacità degli archi.
- Le condizioni di bilancio dei nodi.

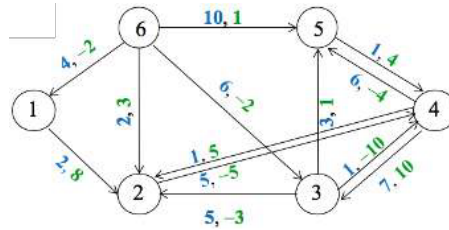
Per stabilire se tale flusso può essere ridotto di costo si costruisce il grafo residuale ad esso associato e si cerca un ciclo di costo negativo:



Il ciclo evidenziato nella figura ha costo $3 - 10 + 5 = 2$ e su di esso si possono redistribuire $\theta = \min\{6, 5, 6\} = 5$ unità di flusso, passando alla situazione seguente:



Da questo grafo si può costruire il grafo *residuale* sul quale si possono controllare tutti i cicli che contengono archi di costo negativo, verificando che non ci sono più cicli di costo negativo, quindi il flusso trovato risulta essere quello di costo minimo.



8.6.2 Formalizzazione dell'algoritmo di eliminazione dei cicli di costo negativo

Dato un flusso ammissibile $\underline{x}$ si considera il *grafo residuale* $G(\underline{x}) = (N, A)$ costruito in modo analogo a quello introdotto per il problema di massimo flusso, e associando ad ogni arco (i, j) del grafo residuale il costo residuale

$$\bar{c}_{i,j} = \begin{cases} c_{i,j} & \text{se l'arco è diretto} \\ -c_{i,j} & \text{se l'arco è inverso} \end{cases}.$$

L'unico modo di far variare il flusso ammissibile $\underline{x}$ senza violare i vincoli di conservazione del flusso è di farlo variare lungo un ciclo che, dato che si vuole diminuire il costo, deve essere di costo negativo nel grafo residuale $G(\underline{x})$.

Una volta individuato un ciclo C di costo negativo, la massima diminuzione di costo si ottiene inviandovi la quantità massima θ di flusso compatibile con le capacità degli archi di C :

$$\theta = \min\{\bar{u}_{i,j} : (i, j) \in C\}$$

La condizione di arresto dell'algoritmo è quindi l'assenza di cicli di costo negativo nel grafo residuale $G(\underline{x})$.

PSEUDOCODICE DELL'ALGORITMO

Algoritmo Eliminazione_Cicli_Negativi

begin

$x = \text{Flusso_ammissibile}(G, \text{ammissibile});$ //x-vettore

ottimo = false;

while (ottimo=false and ammissibile=true) do

begin

costruire il grado residuale $G(x)=(N,A)$ associato
al flusso x ;

determinare, se esiste, un ciclo di costo negativo
 C in $G(x)$;

if C non esiste then

ottimo = true;

else

begin

calcolare $\text{Theta} = \min\{u(i, j) : (i, j) \in C\}$;

for each (i, j) in A do

if (i, j) è un arco diretto then

$x(i, j) = x(i, j) + \text{Theta}$;

else

$x(i, j) = x(i, j) + \text{Theta}$;

end

end

end

8.7 Problema di dimensionamento dei lotti (Lot Sizing)

Si deve programmare l'utilizzo di una linea di produzione sull'arco temporale di un anno, discretizzato in mesi. Per ogni mese $i = 1, \dots, 12$ è nota la stima delle seguenti quantità:

- *Domanda* nel mese d_i
- *Costo unitario di produzione* nel mese c_i [€/Unità]
- *Quantità massima producibile* nel mese u_i

Inoltre, si suppone che le unità siano indivisibili. E' previsto che le unità prodotte nel mese i siano in parte vendute ed in parte immagazzinate.

Del magazzino si conosce:

- La capacità U
- Il costo unitario di immagazzinamento C [€/Unità]
- La consistenza del magazzino all'inizio dell'anno m_{iniz} [Unità]
- La consistenza finale richiesta alla fine dell'anno m_{fin} [Unità]

Determinare quanto produrre e quanto mettere a magazzino in ogni mese al fine di minimizzare i costi totali di produzione e di magazzino.

Le **variabili decisionali** sono

- x_i unità da produrre nel mese i .
- y_i unità in magazzino alla fine del mese i .

Sottoposte ai vincoli, per ogni mese i

$$0 \leq x_i \leq u_i \quad 0 \leq y_i \leq U$$

I **vincoli funzionali** legano tra di loro la produzione, l'immagazzinamento e la vendita:

$$m_{iniz} + \text{unità prodotte nel mese} = \text{unità vendute nel mese} \quad \forall i$$

In particolare avremo:

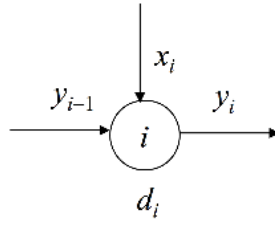
- MESE 1: $m_{iniz} + x_1 = d_1 + y_1$
- MESE i : $y_{i-1} + x_i = d_i + y_i \quad 2 \leq i \leq 11$
- MESE 12: $y_{11} + x_{12} = d_{12} + m_{fin}$

Otteniamo, perciò, il seguente modello di PL:

$$\begin{cases} \min & \sum_{i=1}^{12} c_i x_i + C \sum_{i=1}^{11} y_i \\ \text{s.a} & m_{iniz} + x_1 = d_1 + y_1 \\ & y_{i-1} + x_i = d_i + y_i \quad 2 \leq i \leq 11 \\ & y_{11} + x_{12} = d_{12} + m_{fin} \\ & 0 \leq x_i \leq u_i \text{ e intere} \\ & 0 \leq y_i \leq U \text{ e intere} \end{cases}$$

Come al solito passiamo al corrispondente modello di massimo flusso:

- Il nodo rappresenta il mese i ed il peso del nodo rappresenta la domanda del mese.
- La domanda del mese i può essere soddisfatta con
 - Le unità x_i prodotte nel mese.
 - Le unità y_{i-1} in magazzino all'inizio del mese.
- Le unità disponibili in eccesso, rispetto alla domanda del mese i , sono messe in magazzino.



Le variabili decisionali sono interpretabili come flussi sugli archi:

- x_i flusso sull'arco «produzione nel mese i »
- y_i flusso sull'arco «magazzino nel mese i »

Allora, per ogni nodo vale la relazione

$$x_i + y_{i-1} - y_i = d_i$$

con la variante del nodo 1

$$x_1 - y_1 = d_1 - m_{iniz}$$

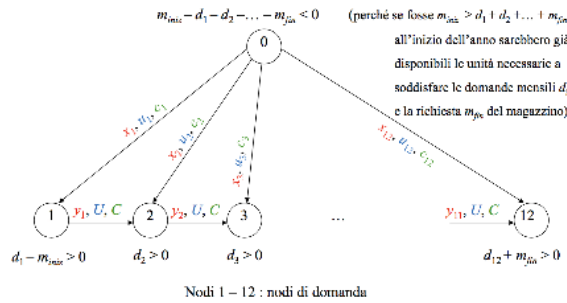
e del nodo 12

$$x_{12} + y_{11} = d_{12} + m_{fin}$$

Sommando le equazioni presentate fino ad ora, si ottiene l'*equazione di bilancio fonti-impieghi* relativa all'intero anno:

$$m_{iniz} + \sum_{i=1}^{12} x_i = \sum_{i=1}^{12} d_i + m_{fin}$$

La struttura del grafo che si viene a creare è quindi, aggiungendo un nodo 0 fittizio:



Si può passare alla formulazione come **problema di flusso di minimo costo**:

- x_i flusso sull'arco $(0, i)$ cioè $x_{0,i}$.
- y_i flusso sull'arco $(i, i + 1)$ cioè $x_{i,i+1}$

Otteniamo quindi il modello di flusso di minimo costo

$$\left\{ \begin{array}{l} \min \quad \sum_{i=1}^{12} c_i x_{0,i} + C \sum_{i=1}^{11} x_{i,i+1} \\ \text{s.a} \quad m_{iniz} + x_{0,1} = d_1 + x_{1,2} \\ \quad \quad x_{i-1,i} + x_{0,i} = d_i + x_{i,i+1} \quad 2 \leq i \leq 11 \\ \quad \quad x_{11,12} + x_{0,12} = d_{12} + m_{fin} \\ \quad \quad 0 \leq x_{0,i} \leq u_{0,i} \text{ e intere} \\ \quad \quad 0 \leq x_{i,i+1} \leq U \text{ e intere} \end{array} \right.$$

dove:

- L'arco dal nodo 0 al nodo i indica le unità prodotte nel mese i .
- L'arco $(i, i + 1)$ indica le unità in magazzino alla fine del mese i .

- Il bilancio al nodo 0 indica le unità prodotte in totale durante l'anno.
- Il bilancio al nodo 1 indica il numero di unità vendute nel mese 1 al netto delle unità in magazzino all'inizio dell'anno.
- Il bilancio nei nodi centrali indica le unità vendute nel mese i .
- Il bilancio al nodo 12 indica le unità vendute nel mese 12 al netto delle unità richieste in magazzino alla fine dell'anno.

8.8 Piano di carico di un aereo

Una compagnia aerea utilizza un aeromobile che trasporta P passeggeri per effettuare un servizio che fa scalo in n località. In particolare sono noti:

- Il numero $b_{i,j}$ di passeggeri potenziali, che vogliono viaggiare dalla località i alla località j .
- La tariffa $f_{i,j}$ per il viaggio dalla località i alla località j .

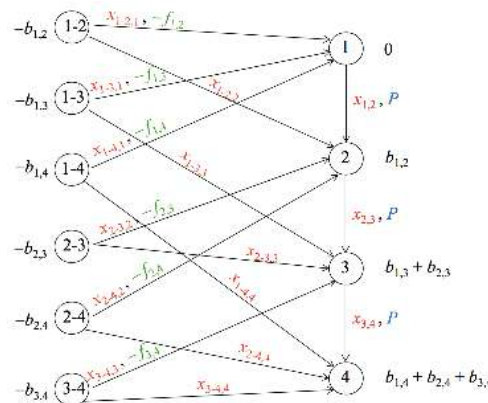
La compagnia aerea vuole determinare quante e quali richieste servire al fine di massimizzare il ricavo totale.

Se la capacità dell'aereo fosse infinita, la compagnia aerea potrebbe soddisfare tutte le richieste di trasporto, con un ricavo totale (supponendo 4 località) pari a

$$\sum_{i=1}^3 \sum_{j=i+1}^4 f_{i,j} \cdot b_{i,j}$$

Avendo l'aereo capacità finita P , per ogni tratta i, j si deve decidere quante richieste di servizio soddisfare, determinando una soluzione che massimizza il ricavo. Utilizziamo come variabili decisionali:

- Il numero delle richieste da soddisfare sulla tratta i, j , indicato con la variabile decisionale $x_{i-j,i}$.
- Introducendo gli archi $(i-j, j)$ per $1 \leq i \leq 3$ e $i+1 \leq j \leq 4$, la variabile decisionale $x_{i-j,j}$ rappresenta il numero delle richieste non soddisfatte sulla tratta $i-j$.
- La variabile decisionale $x_{i,i+1}$, $1 \leq i \leq n-1$, rappresenta il numero di passeggeri sull'aereo tra la località i e la località $i+1$.



Questo modello soddisfa tutti i bilanci ai nodi. In particolare, il modello matematico associato è:

$$\begin{aligned} \min \quad & - (f_{1,2} x_{1,2} + f_{1,3} x_{1,3} + f_{1,4} x_{1,4} + f_{2,3} x_{2,3} + f_{2,4} x_{2,4} + f_{3,4} x_{3,4}) \\ \text{s.t.} \quad & -x_{1,2} - x_{1,3} = -b_{1,2} \\ & -x_{1,3} - x_{1,4} = -b_{1,3} \\ & -x_{1,4} - x_{2,3} = -b_{1,4} \\ & -x_{2,3} - x_{2,4} = -b_{2,3} \\ & -x_{2,4} - x_{3,4} = -b_{2,4} \\ & -x_{3,4} - x_{3,4} = -b_{3,4} \\ & x_{1,2} + x_{1,3} + x_{1,4} = x_{1,2} \\ & x_{2,3} + x_{2,4} = x_{2,3} \\ & x_{3,4} + x_{3,4} = x_{3,4} \\ & x_{1,2} + x_{1,3} + x_{1,4} = x_{1,2} \\ & x_{2,3} + x_{2,4} = x_{2,3} \\ & x_{3,4} + x_{3,4} = x_{3,4} \\ & 0 \leq x_{i,j} \leq P \end{aligned}$$

Capitolo 9

Il problema dello zaino

Dati:

- Un insieme contenente n elementi tra i quali si deve fare una selezione: $E = \{1, 2, \dots, n\}$
- A ciascun elemento $j \in E$ sono associati:
 - $p_j > 0$ intero, cioè il *peso* dell'elemento j .
 - $v_j > 0$ intero, cioè il *valore* dell'elemento j .
- Una capacità massima dello zaino $\bar{P}$. Bisognerà selezionare un certo numero di oggetti in modo che la somma dei loro pesi non superi la capacità massima.

Per analizzare più semplicemente il problema, dato un sottoinsieme $S \subseteq E$ si definiscono:

- **Peso** $P(S)$ la somma dei pesi degli oggetti contenuti in S : $P(S) = \sum_{j \in S} p_j$
- **Valore** $V(S)$ la somma dei valori degli oggetti contenuti in S : $V(S) = \sum_{j \in S} v_j$

9.1 Formulazioni alternative del problema

Esistono due formulazioni alternative del problema:

- *In forma di massimo*: «tra tutti i sottoinsiemi S di E con peso $P(S) \leq \bar{P}$ determinarne uno con valore $V(S)$ massimo».
- *In forma di minimo*: «tra tutti i sottoinsiemi S di E con valore $V(S) \geq \underline{V} \geq 0$ determinarne uno con peso $P(S)$ minimo».

D'ora in poi considereremo la formulazione più frequente, cioè quella di massimo.

9.2 Condizioni sui dati

Dobbiamo supporre che valgano due condizioni sui dati iniziali:

- $p_j \leq \bar{P}$ perché in caso contrario l'oggetto j non potrebbe essere contenuto nello zaino, e quindi dovrebbe essere escluso dall'insieme di oggetti tra cui scegliere.
- $\sum_j p_j > \bar{P}$ perché in caso contrario il problema sarebbe banale e tutti gli oggetti potrebbero essere contenuti nello zaino.

9.3 Formulazione come problema di ottimizzazione combinatoria

La regione ammissibile F è formata da tutti i sottoinsiemi di oggetti il cui peso complessivo non supera la capacità $\bar{P}$ dello zaino:

$$F = \left\{ S \subseteq E : \sum_{j \in S} p_j \leq \bar{P} \right\}$$

Questo è un insieme finito di cardinalità $\leq 2^n$ dove n è il numero di tutti i possibili sottoinsiemi di E .
Come **funzione obbiettivo** utilizziamo la somma dei valori degli oggetti del sottoinsieme S

$$V(S) = \sum_{j \in S} v_j$$

Il problema richiede di determinare un sottoinsieme S^* di oggetti il cui peso complessivo non superi la capacità $\bar{P}$ dello zaino ed il cui valore complessivo sia massimo

$$V(S^*) \geq V(S) \quad \forall S \in F$$

La composizione della regione ammissibile, con numero finito di elementi, permetterebbe di utilizzare l'**enumerazione completa** per la risoluzione del problema, cioè di elencare tutti i possibili sottoinsiemi, per ciascuno calcolarne il valore, e scegliere l'elemento S con valore massimo.

9.3.1 Formulazione come PLI

Utilizziamo, come variabili decisionali

$$x_j = \begin{cases} 1 & \text{se } j \in S \text{ (cioè l'elemento è nello zaino)} \\ 0 & \text{se } j \in S \text{ (cioè l'elemento non è nello zaino)} \end{cases}$$

Possiamo poi costruire il modello

$$\begin{cases} \max & V = \sum_{j=1}^n x_j v_j \\ & \sum_{j=1}^n p_j x_j \leq \bar{P} \\ & x_j \in \{0, 1\} \quad j = 1, \dots, n \end{cases} \quad \text{oppure} \quad \begin{cases} \min & P = \sum_{j=1}^n p_j x_j \\ & \sum_{j=1}^n v_j x_j \geq \bar{V} \\ & x_j \in \{0, 1\} \quad j = 1, \dots, n \end{cases}$$

Data la struttura intera, esso può essere risolto con un algoritmo **branch and bound** basato sul rilassamento continuo

$$\begin{cases} \max & V = \sum_{j=1}^n x_j v_j \\ & \sum_{j=1}^n p_j x_j \leq \bar{P} \\ & 0 \leq x_j \leq 1 \quad j = 1, \dots, n \end{cases}$$

in cui gli oggetti vengono preventivamente ordinati in ordine non crescente per unità di peso (*VUNC: valori unitari non crescenti*)

$$i < j \quad \text{se} \quad \frac{v_i}{p_i} \geq \frac{v_j}{p_j}$$

Se queste condizioni sono verificate, allora la soluzione del problema è **in forma chiusa**, cioè abbiamo a disposizione una formula che fornisce direttamente le soluzioni, senza dover fare il semplice ad ogni iterazione del branch and bound:

$$x_j^* = \begin{cases} 1 & 1 \leq j \leq k-1 \\ \frac{\bar{P} - \sum_{i=1}^{k-1} p_i}{p_j} & j = k \\ 0 & k+1 \leq j \leq n \end{cases}$$

Questa soluzione ottima corrisponde a prendere gli oggetti in ordine non crescente di valore per unità di peso, fino a giungere all'oggetto aggiungendo il quale la capacità dello zaino viene superata, cioè l'oggetto di posto k .

9.3.1.1 Dimostrazione di ottimalità della soluzione

La soluzione trovata è sicuramente di base ammissibile per il problema di PL. Bisogna quindi determinare il problema duale, e controllare che anche per questo è ammissibile (*a questo punto sappiamo che è ottima per il primale*).

Scriviamo il problema primale in notazione vettoriale (*tra quadre sono riportate le variabili duali associate ai vincoli*):

$$\begin{cases} \max & V = \underline{v}^T \underline{x} \\ & \underline{p}^T \underline{x} \leq \bar{P} \quad [\lambda \geq 0] \\ & \underline{x} \leq \underline{1} \quad [\underline{\mu} \geq \underline{0}] \\ & \underline{x} \geq \underline{0} \end{cases}$$

oppure, passando alla notazione a blocchi:

$$\begin{cases} \max & V = \underline{v}^T \underline{x} \\ & \begin{bmatrix} \underline{p}^T \\ I \end{bmatrix} \underline{x} \leq \begin{bmatrix} \bar{P} \\ \underline{1} \end{bmatrix} \quad \begin{bmatrix} \lambda \\ \underline{\mu} \end{bmatrix} \geq \begin{bmatrix} 0 \\ \underline{0} \end{bmatrix} \\ & \underline{x} \geq \underline{0} \end{cases}$$

Il duale di questo problema è:

$$\begin{cases} \min & \begin{bmatrix} \bar{P} & \underline{1}^T \end{bmatrix} \begin{bmatrix} \lambda \\ \underline{\mu} \end{bmatrix} \\ & \begin{bmatrix} \underline{p} & I \end{bmatrix} \begin{bmatrix} \lambda \\ \underline{\mu} \end{bmatrix} \geq \underline{v} \\ & \begin{bmatrix} \lambda \\ \underline{\mu} \end{bmatrix} \geq \begin{bmatrix} 0 \\ \underline{0} \end{bmatrix} \end{cases} \longrightarrow \begin{cases} \min & \bar{P}\lambda + \sum_{j=1}^n \mu_j \\ & p_j \lambda + \mu_j \geq v_j \quad 1 \leq j \leq n \\ & \lambda \geq 0, \mu_j \geq 0 \quad 1 \leq j \leq n \end{cases}$$

In base alle condizioni di complementarità degli scarti, tra problema primale e duale, si ottengono le relazioni:

$$x_j(p_j \lambda + \mu_j - v_j) = 0 \quad 1 \leq j \leq n$$

$$\lambda(\bar{P} - \sum_{j=1}^n p_j x_j) = 0$$

$$\mu_j(1 - x_j) = 0 \quad 1 \leq j \leq n$$

In corrispondenza della soluzione ammissibile primale valgono le seguenti relazioni:

1. $p_j \lambda + \mu_j - v_j = 0$, per $1 \leq k \leq k-1$
2. $p_k \lambda + \mu_k - v_k = 0$
3. $\mu_k = 0$
4. $\mu_j = 0$, per $k+1 \leq j \leq n$

Unendo le relazioni 2 e 3 segue che

$$\lambda = \frac{v_k}{p_k} \quad (5)$$

Unendo le relazioni 1 e 5 segue che

$$p_j \frac{v_k}{p_k} + \mu_j - v_j = 0 \longrightarrow \mu_j = v_j + p_j \frac{v_k}{p_k} \quad \text{per } 1 \leq j \leq k-1$$

Unendo le relazioni 3 e 4 segue che

$$\mu_j = 0 \quad \text{per } k \leq j \leq n$$

A questo punto possiamo controllare l'ammissibilità della soluzione duale complementare determinata:

- $\lambda = \frac{v_k}{p_k} > 0$ in quanto è un rapporto di numeri positivi.
- $\mu_j = v_j + p_j \frac{v_k}{p_k}$ per $1 \leq j \leq k-1$ perché $\frac{v_j}{p_j} \geq \frac{v_k}{p_k}$ essendo $j < k$.

Poiché le due soluzioni primale e duale sono ammissibili per i rispettivi problemi e soddisfano le condizioni degli scarti complementari, allora sono ottime per i rispettivi problemi.

Capitolo 10

Il problema del commesso viaggiatore (TSP)

Siano date:

- n località
- $c_{i,j}$ reale positivo, cioè il costo del viaggio dalla località i alla località j .

Il commesso viaggiatore si trova in una località a partire dalla quale deve visitare una ed una sola volta ciascuna delle altre $n - 1$ località, e quindi tornare nella località di partenza.

Note le distanze tra ogni coppia di località, si vuole organizzare il viaggio in modo da **minimizzare la distanza percorsa** (o i costi di percorrenza).

10.1 Tipiche applicazioni di TSP

Sono problemi di TSP:

- Problemi di trasporto.
- Consegne e rifornimenti.
- Raccolta di rifiuti.
- Decisione di come servire in modo ottimale n richieste di lettura/scrittura su un disco magnetico minimizzando il ritardo dovuto ai movimenti della testina.

10.2 Problema visto come su un grafo

Il problema del commesso viaggiatore è rappresentato da un grafo $G = (N, A)$:

- N insieme dei nodi che rappresentano le n località.
- A insieme degli m archi che uniscono le località. Ad ogni arco $(i, j) \in A$ è associato un costo reale positivo $c_{i,j}$.
- Il grafo viene supposto completo (*cioè esiste un arco per ogni coppia ordinata di nodi*). Infatti, se non lo fosse si potrebbero aggiungere un qualsiasi numero di archi con costo opportunamente elevato.

Si possono presentare due situazioni:

- **TSP Simmetrico:** Il grafo non è orientato, quindi $c_{i,j} = c_{j,i}$ per ogni $(i, j) \in A$, quindi la matrice dei costi sarà simmetrica.
- **TSP Non Simmetrico:** Il grafo è orientato, quindi in generale $c_{i,j} \neq c_{j,i}$ per ogni $(i, j) \in A$, quindi la matrice dei costi non sarà simmetrica.

La visita in sequenza delle n località con rientro al punto di partenza corrisponde ad un **ciclo Hamiltoniano** ossia ad una sequenza di archi che inizia e termina nello stesso nodo, passando per tutti gli altri nodi una ed una sola volta.

La lunghezza di un ciclo Hamiltoniano $C = (N, A_C)$ è la somma dei pesi degli n archi attraversati dal ciclo

$$\sum_{(i,j) \in A_C} c_{i,j}$$

cioè rappresenta la «distanza» complessiva percorsa lungo il circuito. Il problema del commesso viaggiatore vuole determinare il ciclo hamiltoniano tra tutti più corto, cioè coincide con il problema del *ciclo Hamiltoniano di lunghezza minima*.

10.2.1 Formulazione PLI di TSP

Utilizziamo come **variabili decisionali** $x_{i,j} = \begin{cases} 1 & \text{se } (i,j) \in C \\ 0 & \text{se } (i,j) \notin C \end{cases}$

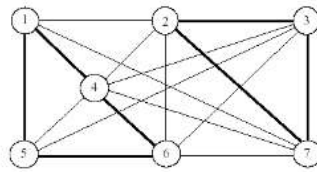
Vogliamo, per risolvere il problema di TSP, minimizzare la quantità:

$$\min \sum_{(i,j) \in A} c_{i,j} \cdot x_{i,j}$$

Per avere la **copertura dei cicli**, cioè garantire che in ogni nodo incidano esattamente due archi, imponiamo il vincolo:

$$\sum_{j \in N \setminus \{i\}} x_{i,j} = 2 \quad \forall i \in N$$

Questo vincolo è una condizione necessaria ma non sufficienti, in quanto potrebbero essere soddisfatte anche da **cicli disgiunti**:



Per evitare di includere anche *cicli disgiunti* aggiungiamo anche **vincoli di connessione sui tagli**, cioè imponiamo che gli archi selezionati formino un **unico ciclo Hamiltoniano**

$$\sum_{(i,j) \in A(N', N'')} x_{i,j} \geq 1 \quad \forall (N', N'')$$

Questo impone che almeno uno degli archi di taglio debba appartenere al ciclo C : il numero dei vincoli di connessione è pari al numero dei *tagli* (partizioni dei nodi in due sottoinsiemi N' e N'') che si possono definire sul grafo, cioè i due sottoinsiemi N' ed N'' non devono essere disgiunti.

Mettendo insieme tutte queste considerazioni otteniamo che la formulazione PLI di TSP Simmetrico è:

$$\begin{cases} \min & \sum_{(i,j) \in A} c_{i,j} x_{i,j} \\ & \sum_{j \in N \setminus \{i\}} x_{i,j} = 2 \quad \forall i \in N \\ & \sum_{(i,j) \in A(N', N'')} x_{i,j} \geq 1 \quad \forall (N', N'') \\ & x_{i,j} \in \{0, 1\} \quad \forall (i, j) \in A \end{cases}$$

10.2.2 Formulazione come ottimizzazione combinatoria di TSP

L'insieme delle **soluzioni ammissibili** F è l'insieme di tutti i cicli Hamiltoniani $C = (N, A_C)$ sul grafo G . Come funzione obbiettivo utilizziamo $c(C)$ cioè la lunghezza del ciclo Hamiltoniano C , da minimizzare.

OSSERVAZIONE: Se il grafo è orientato e completo, il numero di cicli hamiltoniani, cioè si possibili soluzioni, è pari a $(n-1)!$ dove n è il numero di nodi.

Nel caso in cui il TSP sia simmetrico, cioè $c_{i,j} = c_{j,i}$, il problema può essere rappresentato da un *grafo non orientato* con un solo arco non orientato tra ogni coppia di nodi. In questa situazione, il numero totale di cicli hamiltoniani è $\frac{(n-1)!}{2}$.

La prima procedura risolutiva potrebbe essere quella di **enumerazione completa**, cioè la verifica «manuale» di tutti i cicli hamiltoniani presenti, il calcolo del loro costo, e la scelta di quello con un costo inferiore. E' facilmente dimostrabile, però, che questa procedura è praticabile solo per istanze di dimensione molto piccola.

Un problema si dice **risolvibile con una data procedura** se la procedura restituisce una soluzione in tempi ragionevoli.

10.2.2.1 Algoritmo di Branch and Bound per TSP simmetrico

Osserviamo che un **cicli hamiltoniano** è l'unione di un **cammino hamiltoniano**, cioè un cammino che attraversa tutti i nodi del grafo, e del lato che chiude il ciclo.

Inoltre, il cammino hamiltoniano è un albero di supporto i cui nodi hanno la proprietà di avere **grado al più 2**.

L'idea è quella di costruire un albero di supporto di costo minimo, con nodi di grado al più 2, al quale si aggiunge il lato di costo minimo tra quelli che non appartengono all'albero, ottenendo una struttura chiamata **1-albero**:

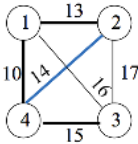
- Se il lato che collega i due estremi del cammino Hamiltoniano ha il costo minimo tra quelli non appartenenti all'albero, allora l'1-albero è un ciclo hamiltoniano, cioè tutti i suoi nodi hanno grado 2.
- Se il lato che collega i due estremi del cammino Hamiltoniano non ha costo minimo tra quelli non appartenenti all'albero, il lato aggiunto incide su un nodo che non è un estremo del cammino. Non si otterrà quindi un ciclo hamiltoniano, ma un 1-albero con costo che è un lower-bound del ciclo hamiltoniano di costo minimo.

Da queste considerazioni si capisce che il problema di determinare un 1-albero è un *rilassamento* del problema di trovare un ciclo Hamiltoniano di costo minimo.

Nei nodi dell'albero di Branch And Bound in cui l'1-albero di costo minimo non è un ciclo hamiltoniano, si esegue il *branching* secondo la regola di «imporre alternativamente la presenza e l'assenza del lato di costo minimo tra quelli incidenti al nodo di grado massimo nell'1-albero».

Esempio di ricerca di un 1-albero su un grafo

{1, 4}	10
{1, 2}	13
{2, 4}	14 : non lo aggiungo perché crea un sottociclo
{3, 4}	15 : stop, perché ho il cammino 2 - 1 - 4 - 3
{1, 3}	16
{2, 3}	17



Se, sull'esempio dell'immagine si volesse proseguire, aggiungerebbero il lato $\{2, 4\}$ che è quello di minimo costo tra i lati non appartenenti al cammino. Esso però genera una soluzione non ammissibile per il problema originario, perché non chiude il ciclo, quindi permette di ottenere un *lower bound*:

$$LB_0 = 10 + 13 + 15 + 14 = 52$$

Da questo punto, si esegue il branching considerando che il nodo di grado massimo è 4 e che su di esso incide il lato di costo minimo $\{1, 4\}$, ottenendo due problemi:

- P_1 : determinare un 1-albero che contenga il lato $\{1, 4\}$
- P_2 : determinare un 1-albero che non contenga il lato $\{1, 4\}$

10.2.2.2 Algoritmo GREEDY per il problema del commesso viaggiatore

Ricordiamo che un algoritmo *greedy* è un processo iterativo, nel quale per ogni iterazione si fa la migliore scelta localmente. Un algoritmo greedy può terminare con:

- Soluzione ammissibile ottima.
- Soluzione ammissibile ma non ottima.

- Soluzione non ammissibile.

L'algoritmo greedy per TSP è composto da:

- INIZIALIZZAZIONE:
 - L'insieme S degli archi appartenenti al ciclo è vuoto.
 - Il nodo corrente è posto uguale al nodo iniziale.
- PASSO ITERATIVO:
 - Si individuano tutti gli archi che uniscono il nodo corrente i a nodi non ancora toccati dagli archi contenuti in S .
 - Si seleziona tra essi l'arco (i, j) «più promettente» secondo un dato criterio euristico e lo si aggiunge ad S .
L'euristica più utilizzata è quella del *Nearest Neighbour* cioè si sceglie l'arco di lunghezza minima, partendo dal nodo corrente i .
- Si definisce j come nuovo nodo corrente.
- L'algoritmo termina quando tutti i nodi sono toccati dagli archi contenuti in S : si inserisce quindi l'arco di ritorno dall'ultimo nodo al nodo iniziale.

Se il grafo G è completo, l'algoritmo greedy costruisce sicuramente un ciclo hamiltoniano. Altrimenti può accadere che tutti gli archi uscenti dal nodo corrente i portino a nodi già visitati dagli archi di S , quindi l'algoritmo *greedy* non può produrre una soluzione ammissibile, anche qualora la soluzione ammissibile esista.

10.2.2.3 Euristica alternativa al Nearest Neighbour: Twice Around MST

Se:

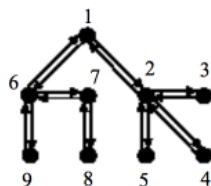
- Il grafo G è completo.
- I costi sugli archi sono *non negativi* e soddisfano la *disuguaglianza triangolare*, cioè

$$c_{i,k} \leq c_{i,j} + c_{j,k} \quad \forall i, j, k$$

è possibile costruire un'euristica greedy, detta **twice around MST** (*Minimum Spanning Tree*) avente, nel caso peggiore, errore relativo pari ad 1.

Le principali **fasi dell'algoritmo** sono:

- Si determina un **albero di copertura di costo minimo** T^* .
- Si duplicano gli archi di T^* e si attribuiscono agli archi orientati gli stessi costi degli archi orientati originali. Quindi, il costo totale del ciclo orientato (non semplice) è il doppio del costo di T^* .



- Dal ciclo orientato ottenuto al passo precedente, si ottiene il ciclo hamiltoniano H , eliminando la ripetizione dei nodi mediante l'operazione di **scorciatoia**:
 - Si parte da un qualsiasi nodo (*detto nodo 1*).
 - Il successore del nodo corrente i nel ciclo hamiltoniano viene determinato esaminando i nodi successori di i sul ciclo orientato non semplice fino ad incontrare un nodo h non ancora raggiunto dal ciclo hamiltoniano. Il cammino da i ad h sul ciclo orientato non semplice è sostituito nel ciclo hamiltoniano dalla «scorciatoia» (i, h) che esiste sempre in quanto il grafo è completo.
 - Il passo precedente viene iterato a partire dal nodo h .
 - Quando viene visitato l'ultimo nodo, il ciclo viene chiuso con l'aggiunta dell'arco che conduce al nodo iniziale, ossia operando una «scorciatoia» nell'ultimo nodo visitato al nodo 1.

- Per il ciclo Hamiltoniano ottenuto con questa euristica, vale che $c(H^*) \leq c(H)$ dove H^* denota il ciclo hamiltoniano di costo minimo.
- Eliminando un qualsiasi arco da H^* si ottiene un cammino P che è un albero di copertura su G : poiché P è ottenuto da H^* eliminando un arco, vale che $c(P) \leq c(H^*)$.
- Per l'albero P di copertura su G vale che $c(T^*) \leq c(P)$ con T^* che denota l'albero di copertura di costo minimo.

Da questi ultimi tre punti si ottiene la seguente catena di disuguaglianze:

$$c(T^*) \leq c(P) \leq c(H^*) \leq c(H) \leq 2c(T^*)$$

dove:

- H^* è la soluzione ottima di TSP.
- H è la soluzione costruita con l'euristica.
- $c(H^*)$ è il minimo possibile.
- T^* è l'albero a costo minimo da cui si è partiti, calcolato con Kruskal o Prim.

L'errore relativo compiuto dall'euristica basata su MST è definito da:

$$ErrRel_{taMST} = \frac{c(H) - c(H^*)}{c(H^*)}$$

Sommando membro a membro le disuguaglianze

$$c(H) \leq 2c(T^*) \quad e \quad -c(H^*) \leq -c(T^*)$$

si ottiene

$$c(H) - c(H^*) \leq c(T^*)$$

Utilizzando inoltre la disuguaglianza $c(H^*) \geq c(T^*)$ si ottiene che l'errore relativo è

$$ErrRel_{taMST} = \frac{c(H) - c(H^*)}{c(H^*)} \leq \frac{c(T^*)}{c(T^*)} = 1$$

E' comodo sapere questo lower-bound perché il valore di $ErrRel_{taMST}$ non è calcolabile, in quanto non si conosce a priori il valore ottimo H^* .

Capitolo 11

Problemi basati su variabili intere binarie

11.1 Ordinamento di lavori su macchine con minimizzazione del numero di macchine (MCMS)

Supponiamo di avere

- n **lavori**, per ciascuno dei quali sono noti t_i cioè il **tempo di inizio** dell'esecuzione del lavoro e d_i la durata della lavorazione. Di conseguenza possiamo sapere che il lavoro i terminerà al tempo $t_i + d_i$.
- m **macchine uguali** su cui eseguire i lavori.

Vogliamo minimizzare il numero di macchine, sotto al vincolo di assegnare ad ogni macchina solo lavori i cui tempi di esecuzione non si sovrappongano.

Le soluzioni al problema sono quindi partizioni dell'insieme $N = \{1, 2, \dots, n\}$ in m sottoinsiemi $N(1), N(2) \dots N(m)$ dove il sottoinsieme j -esimo rappresenta i lavori assegnati alla j -esima macchina.

Problemi di questo tipo sono presenti in ambito manifatturiero, nella gestione dei sistemi informatici e nella gestione di progetti.

11.1.1 Formulazione PLI del problema MCMS

Utilizziamo, come **variabili decisionali**, $m \cdot n$ *variabili binarie*

$$x_{i,j} = \begin{cases} 1 & \text{se il lavoro } i \text{ è eseguito sulla macchina } j \\ 0 & \text{altrimenti} \end{cases}$$

Le soluzioni ammissibili devono sottostare ad alcune condizioni:

- **VINCOLO DI ASSEGNAMENTO DI RIGA:** ogni lavoro deve essere assegnato ad una ed una sola macchina. Pertanto, costruendo una matrice $X = \{x_{i,j}\}$, in cui sulle righe abbiamo i lavori e sulle colonne le macchine, la somma di ogni riga deve essere pari ad 1

$$\sum_{j=1}^m x_{i,j} = 1 \quad i = 1, \dots, n$$

- **VINCOLO DI COMPATIBILITÀ:** I lavori sulla stessa macchina devono essere disgiunti dal punto di vista temporale. Pertanto, due lavori i ed h con intervalli di lavorazione $[t_i, t_i + d_i]$ e $[t_h, t_h + d_h]$ possono essere assegnati alla stessa macchina solo se gli intervalli di lavorazione non si sovrappongono.

Per esprimere i vincoli di compatibilità, costruiamo, elaborando i dati di input, l'insieme $S(i)$ di incompatibilità associato al lavoro i , ossia l'insieme dei lavori h , con $h > i$ incompatibili con il lavoro i :

$$S(i) = \{h \in \{i+1, \dots, n\} : [t_i, t_i + d_i] \cap [t_h, t_h + d_h] \neq \emptyset\}$$

	1-5	3-8	4-6	5-9	6-7	8-11	9-13	10-12	11-15	13-16
1-5		•	•							
3-8			•	•	•					
4-6				•						
5-9					•	•				
6-7										
8-11							•	•		
9-13								•	•	
10-12									•	
11-15										•
13-16										

I **vincoli di compatibilità** impediscono che le variabili relative a due lavori incompatibili assumano entrambe valore 1:

$$x_{i,j} + x_{h,j} \leq 1 \quad i = 1, \dots, n-1, \quad h \in S(i), \quad j = 1, \dots, m$$

Il numero di questo tipo di vincoli:

- Dipende dai conflitti tra gli intervalli di tempo, e quindi dalla somma delle cardinalità degli insiemi di incompatibilità moltiplicato per il numero m di macchine.
- Sicuramente è inferiore a $\frac{m \cdot n \cdot (n-1)}{2}$

La **funzione obiettivo** deve esprimere il numero di macchine utilizzate, quindi deve contenere un'informazione sul fatto che se il lavoro i è eseguito sulla macchina j allora essa è utilizzata. Questo viene fatto introducendo una variabile per ogni macchina

$$y_j = \begin{cases} 1 & \text{se la macchina } j \text{ è utilizzata} \\ 0 & \text{altrimenti} \end{cases} \in \{0, 1\}$$

ed i $m \cdot n$ vincoli

$$x_{i,j} \leq y_j \quad i = 1, \dots, n, \quad j = 1, \dots, m$$

Di conseguenza la **funzione obiettivo**, che minimizza il numero di variabili utilizzate, è:

$$\varphi = \min \sum_{j=1}^m y_j$$

Cioè si ha un modello di PLI come il seguente:

$$\left\{ \begin{array}{ll} \min & \sum_{j=1}^m y_j \\ & \sum_{j=1}^m x_{i,j} = 1 \quad i = 1, \dots, n \\ & x_{i,j} + x_{h,j} \leq 1 \quad i = 1, \dots, n-1, \quad h \in S(i), \quad j = 1, \dots, m \\ & x_{i,j} \leq y_j \quad i = 1, \dots, n, \quad j = 1, \dots, m \\ & x_{i,j} \in \{0, 1\} \quad i = 1, \dots, n, \quad j = 1, \dots, m \\ & y_j \in \{0, 1\} \quad j = 1, \dots, m \end{array} \right.$$

11.1.2 Algoritmi greedy per MCMS

- INIZIALIZZAZIONE:
 - Nessun lavoro è assegnato
 - Nessuna macchina è utilizzata: $N(j) = \emptyset, \forall j$
- ITERAZIONE:

- Selezionare secondo un criterio euristico il lavoro i tra quelli non assegnati.
Un particolare criterio di scelta può essere quello che prevede il selezionamento del lavoro in ordine di tempo di inizio t_i non crescente.
- Tra tutte le macchine già in uso selezionare, secondo un criterio euristico, la macchina j alla quale assegnare il lavoro i .
Un particolare criterio di scelta può essere quello che prevede di ordinare le macchine in un ordine arbitrario e, per l'inserzione del lavoro corrente, esaminarle sempre in tale ordine
- Se i non può essere eseguito su nessuna delle macchine in uso, lo si assegna ad una nuova macchina che si aggiunge all'insieme di macchine utilizzate.
Sulla base del «particolare criterio di scelta» spiegato nei due punti precedenti, se il lavoro non può essere inserito nelle macchine già in uso, viene attivata la successiva macchina nell'ordinamento.
Questo algoritmo costruisce un assegnamento che utilizza sempre il minor numero possibile di macchine.

11.2 Ordinamento di lavori su macchine con minimizzazione del tempo di completamento (MMMS)

Sono assegnati:

- m macchine.
- n lavori.
- d_i cioè la durata dell'esecuzione del lavoro i .

A differenza del problema precedente non viene fissato alcun tempo di inizio di ciascun lavoro, che quindi può essere fissato a piacimento.

Si vuole decidere come suddividere i lavori tra le macchine affinché tutti i lavori siano completati nel più breve tempo possibile.

11.2.1 Formulazione PLI del problema MMMS

Se, ad esempio, supponiamo di assegnare:

- MACCHINA 1: lavoro 1 (durata d_1), lavoro 2 (durata d_2). Otteniamo un tempo di completamento della macchina pari a $T_1 = d_1 + d_2$.
- MACCHINA 2: lavoro 3 (durata d_3), lavoro 4 (durata d_4). Otteniamo un tempo di completamento della macchina pari a $T_2 = d_3 + d_4$.

Quello che si vuole minimizzare è il tempo di completamento totale:

$$\max_j \{T_j\} = \max_j \left\{ \sum_{i=1}^n d_i x_{i,j} \right\}$$

La funzione obbiettivo (*presente come ultima formula nel box qui sopra*) dovrà quindi tenere conto del tempo di completamento t di tutti i lavori, che coincide con il tempo di completamento della macchina più carica.

In base a come è stato presentato, questo è un **problema di partizione dell'insieme dei lavori**, con le seguenti $m \cdot n$ **variabili decisionali**:

$$x_{i,j} = \begin{cases} 1 & \text{se il lavoro } i \text{ viene eseguito sulla macchina } j \\ 0 & \text{altrimenti} \end{cases}$$

Essendo un problema di PLI avremo dei **vincoli di semiassegnamento**: ogni lavoro deve essere eseguito su una ed una sola macchina:

$$\sum_{j=1}^m x_{i,j} = 1 \quad i = 1, \dots, n$$

Per formulare la funzione obbiettivo dobbiamo esprimere il massimo tra m quantità: a tal fine introduciamo una nuova variabile t che rappresenta una **approssimazione per eccesso** del tempo di completamento:

Per ogni macchina j deve valere $T_j \leq t$

Quindi il modello di PLI sarà:

$$\begin{cases} \min & t \\ & \sum_{j=1}^m x_{i,j} = 1 & i = 1, \dots, n \\ & \sum_{i=1}^n d_i x_{i,j} \leq t & i = 1, \dots, m \\ & x_{i,j} \in \{0, 1\} & i = 1, \dots, n \quad j = 1, \dots, m \end{cases}$$

Nella soluzione ottima, il valore di t coincide con il tempo di lavorazione della macchina più carica (*cioè ci sarà almeno una macchina per cui il vincolo di upper-bound non sarà soddisfatto con il $\leq$ ma con $=$*), cioè nella soluzione ottima la variabile t fornisce il tempo di completamento e non una sua approssimazione.

11.2.2 Algoritmi greedy di List Scheduling

Questi tipi di problemi vengono utilizzati quando la dimensione del problema è elevata e quindi non potrebbe essere utilizzato il *branch and bound*, che ha il problema dell'esplosione della dimensione dell'albero.

- INIZIALIZZAZIONE:
 - Tutte le macchine sono scariche $N(j) = \emptyset \quad 1 \leq j \leq m$
 - Nessun lavoro è assegnato.
 - I lavori vengono ordinati in ordine *non decrescente* (SPT - *Shortest Processing Time*) o non crescente (LPT - *Longest Processing Time*) dei tempi di esecuzione. In generale, LPT è quello che produce soluzioni di migliore qualità, ma varia da caso a caso.
- ITERAZIONE:
 - Selezionare il lavoro i tra quelli non ancora assegnati.
 - Selezionare la «macchina più scarica», ossia quella che nella soluzione corrente ha il tempo di lavoro più basso. Indichiamo tale macchina con j .
 - Si assegna il lavoro i alla macchina j , ovvero inseriamo i in N_j e si incrementa il tempo di utilizzo della macchina j : $T_j = \sum_{i=1}^n d_i x_{i,j}$
 - Il procedimento termina quando tutti i lavori sono stati assegnati.

Soluzioni *greedy* commettono un errore relativo pari a:

$$ErrRel_{LS} = \frac{\varphi_{LS} - \varphi_{MMMS}}{\varphi_{MMMS}}$$

Dato che LS è un problema di minimo e che la soluzione data dall'euristica LS è ammissibile ma in generale non ottima, vale la relazione $\varphi_{MMMS} \leq \varphi_{LS}$.

11.2.2.1 Analisi dell'errore relativo di LS

Si vogliono ora determinare due valori a e b tali che $a < \varphi_{MMMS}$ e $b > \varphi_{LS}$:

- Una soluzione con tempo di completamento a è non ammissibile, in quanto per ottenere un valore della funzione obiettivo più basso dell'ottimo dovremmo rilassare obbligatoriamente alcuni vincoli.

In particolare, se rilassiamo il vincolo di indivisibilità dei lavori, una soluzione del genere sarebbe possibile, e prevederebbe di dividere in modo uniforme gli n lavori tra le m macchine. Essa avrebbe valore

$$a = \frac{\sum_{i=1}^n d_i}{m}$$

- Una soluzione con tempo di completamento b è il più alto (*quindi peggiore*) valore che si può ottenere con l'euristica. Tale valore è ottenibile quando all'assegnamento dell'ultimo lavoro con durata d_h tutte le macchine sono bilanciate, cioè sono utilizzate per un tempo

$$\frac{\sum_{i=1}^n d_i - d_h}{m}$$

In questo caso si dovrà scegliere arbitrariamente una macchina su cui assegnare l'ultimo lavoro ed il tempo di completamento diventa

$$b = \frac{\sum_{i=1}^n d_i - d_h}{m} + d_h$$

Quindi, la maggiorazione dell'errore relativo delle euristiche LS è:

$$ErrRel_{LS} = \frac{\varphi_{LS} - \varphi_{MMMS}}{\varphi_{MMMS}} \leq \frac{b - a}{a} = (m - 1) \frac{d_h}{\sum_{i=1}^n d_i}$$

E' possibile ottenere una maggiorazione dell'errore in funzione del solo numero di macchine m se per il denominatore si usa il minorante d_h

$$ErrRel_{LS} \leq \frac{m - 1}{m}$$

11.2.2.2 Particolarità dell'ordinamento SPT

Se si utilizza l'ordinamento SPT è possibile ottenere $ErrRel_{LS} = \frac{m - 1}{m}$ se si hanno:

- $(m - 1) \cdot m$ lavori di durata 1.
- 1 lavoro di durata $m > 1$.

L'assegnamento ottimo consiste nell'assegnare ad una macchina l'unico lavoro di durata m e distribuire uniformemente sulle restanti $m - 1$ macchine gli $(m - 1) \cdot m$ lavori di durata 1.

In questo caso tutte le macchine terminano in tempo m .

11.3 Problema di Graph Colouring

Il problema di **graph colouring** è il tipico problema che si presenta quando si vogliono, ad esempio, assegnare m frequenze su n stazioni radio, non assegnando la stessa frequenza a stazioni adiacenti, per evitare che tra queste si creino interferenze. Ad ogni frequenza è anche abbinato un costo, quindi tra tutte le situazioni che escludono la possibilità di avere interferenze si vuole determinare la soluzione a costo minimo (*se tutte le frequenze hanno lo stesso costo, il problema si riduce al trovare il numero minimo di frequenze da utilizzare*).

Si parla di *graph colouring* perché un problema di questo tipo si ha anche quando si vogliono colorare i nodi del grafo con dei colori f_i sottostando al vincolo di non avere colori uguali tra nodi adiacenti, minimizzando il numero di colori utilizzati.

11.3.1 Formulazione PLI del problema

Utilizziamo $n \cdot m$ **variabili** binarie $x_{i,f} = \begin{cases} 1 & \text{se la frequenza } f \text{ viene assegnata all'antenna } i \\ 0 & \text{altrimenti} \end{cases}$ sottoposte al

vincolo di semiassegnamento, cioè al vincolo che ad ogni antenna venga assegnata una ed una sola frequenza:

$$\sum_{f=1}^m x_{i,f} = 1 \quad i = 1, \dots, n$$

Definiamo inoltre una **variabile binaria** per ogni frequenza: $y_f = \begin{cases} 1 & \text{se la frequenza } f \text{ è utilizzata} \\ 0 & \text{altrimenti} \end{cases}$.

Con l'utilizzo di questa ulteriore variabile binaria si introduce anche un altro vincolo, cioè che se la frequenza f è assegnata ad un'antenna, allora deve essere contata tra quelle utilizzate:

$$x_{i,f} \leq y_f \quad 1 \leq i \leq n, 1 \leq f \leq m$$

L'ultimo vincolo da introdurre è il **vincolo di non adiacenza**: definiamo un grafo non orientato in cui

- I nodi rappresentano le n antenne.
- Si definisce un arco (i, j) per ogni coppia di antenne adiacenti.

Il vincolo di non adiacenza viene espresso in termine degli archi di questo grafo, cioè definendo una relazione che vincoli la frequenza f ad essere assegnata al più ad una stazione tra quelle adiacenti:

$$x_{i,f} + x_{j,f} \leq 1 \quad (i,j) \in A, \quad f = 1, \dots, m$$

Riassumendo tutti questi elementi, il modello di PLI è:

$$\left\{ \begin{array}{ll} \min & \sum_{f=1}^m y_f \\ & \sum_{f=1}^m x_{i,f} = 1 \quad i = 1, \dots, n \\ & x_{i,f} + x_{j,f} \leq 1 \quad (i,j) \in A, \quad f = 1, \dots, m \\ & x_{i,f} \leq y_f \quad y_f 1 \leq i \leq n, \quad 1 \leq f \leq m \\ & x_{i,f} \in \{0, 1\} \quad i = 1, \dots, n, \quad f = 1, \dots, m \\ & y_f \in \{0, 1\} \quad f = 1, \dots, m \end{array} \right.$$

11.4 Problemi di selezione di sottoinsiemi

Supponiamo di avere:

- Un insieme finito di elementi $E = \{1, 2, \dots, m\}$
- Una famiglia di $n \leq 2^{|E|} = 2^m$ sottoinsiemi ammissibili di E , $F = \{F_1, F_2, \dots, F_n\}$ con $F_j \subseteq E$, $j = 1, \dots, n$.
A ciascun sottoinsieme F_j è associato un costo c_j

Si vuole determinare una sottofamiglia $D \subseteq F$ di costo minimo che soddisfi assegnati vincoli.

11.4.1 Rappresentazione della famiglia F di sottoinsiemi di E

La famiglia F viene rappresentata tramite una matrice:

$$A = [a_{i,j}], \quad i = 1, \dots, m, \quad j = 1, \dots, n, \quad a_{i,j} = \begin{cases} 1 & \text{se } i \in F_j \\ 0 & \text{altrimenti} \end{cases}$$

Questa matrice ha tante righe quante sono gli elementi di E e tante colonne quanti sono i sottoinsiemi ammissibili di E , cioè gli elementi di F .

11.4.2 Come individuare una sottofamiglia $D \subseteq F$

Definiamo n variabili binarie $x_1, \dots, x_n$, con $x_j = \begin{cases} 1 & \text{se si sceglie di inserire } F_j \text{ in } D \\ 0 & \text{altrimenti} \end{cases}$.

Il costo della scelta della sottofamiglia D è

$$\sum_{j=1}^n c_j x_j$$

Si possono individuare tre diversi problemi di selezione di sottoinsiemi:

- DI COPERTURA (PC)
- DI PARTIZIONE (PP)
- DI RIEMPIMENTO (PR)

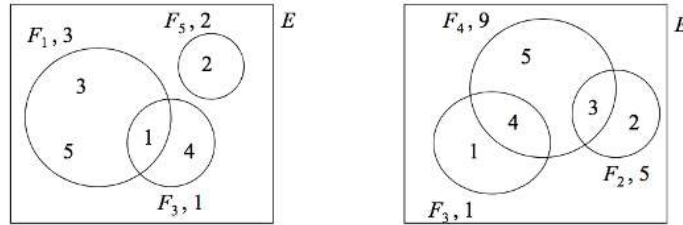
11.4.2.1 Problema di copertura

La sottofamiglia D deve essere scelta in modo che ogni elemento di E appaia in **almeno un** $F_j \in D$.

Si dice quindi che una sottofamiglia D con tale proprietà **copre** l'insieme degli elementi di E . Un esempio di problema di copertura è il seguente:

$$F = \{\underbrace{\{1, 3, 5\}}_{F_1}, \underbrace{\{2, 3\}}_{F_2}, \underbrace{\{1, 4\}}_{F_3}, \underbrace{\{3, 4, 5\}}_{F_4}, \underbrace{\{2\}}_{F_5}, \underbrace{\{5\}}_{F_6}, \underbrace{\{1, 5\}}_{F_7}\}$$

$$\underline{c}(F) = [3 \ 5 \ 1 \ 9 \ 2 \ 4 \ 1]^T$$



Si vuole determinare la copertura D di E mediante sottoinsiemi scelti da F che minimizza il costo totale di copertura.

Per fare questo deve essere rispettato il **vincolo di copertura**, cioè per ogni $i \in E$ deve esistere almeno un indice j per il quale contemporaneamente si ha $a_{i,j} = 1$ e $x_{i,j} = 1$:

$$\sum_{j=1}^n a_{i,j} x_j \geq 1 \quad i = 1, \dots, m = |E|$$

Il modello di PLI del problema è:

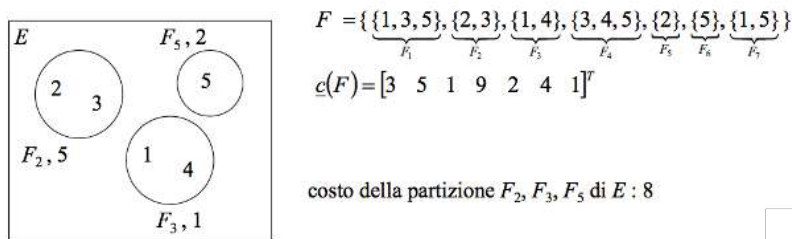
$$\begin{cases} \min & \sum_{j=1}^n c_j x_j \\ & \sum_{j=1}^n a_{i,j} x_j \geq 1 \quad i = 1, \dots, m \\ & x_j \in \{0, 1\} \quad j = 1, \dots, n \end{cases}$$

Un **esempio** di problema di copertura potrebbe essere l'organizzazione di un gruppo di giornalisti ed operatori, che lavorano presso una rete televisiva, per coprire m servizi fuori sede.

11.4.2.2 Problema di partizione

La sottofamiglia D deve essere scelta in modo che ogni elemento di E appaia in **uno ed un solo** $F_j \in D$. Si dice che una sottofamiglia D con tale proprietà *partiziona* l'insieme degli elementi di E .

Un esempio di partizione è il seguente:



Si vuole determinare la partizione D di E mediante sottoinsiemi scelti da F che minimizza il costo totale di partizione. In forma matematica, il **vincolo di partizione** che deve essere soddisfatto è:

$$\sum_{j=1}^n a_{i,j} x_j = 1 \quad i = 1, \dots, m$$

Cioè per ogni $i \in E$ esiste un solo indice j per il quale è contemporaneamente $a_{i,j} = 1$ e $x_j = 1$.

Il modello di PLI del problema è:

$$\begin{cases} \min & \sum_{j=1}^n c_j x_j \\ & \sum_{j=1}^n a_{i,j} x_j = 1 \quad i = 1, \dots, m \\ & x_j \in \{0, 1\} \quad j = 1, \dots, n \end{cases}$$

Un **esempio** di problema di partizione potrebbe essere l'organizzazione di un appalto di m diversi lavori tra n possibili incarichi di appalto.

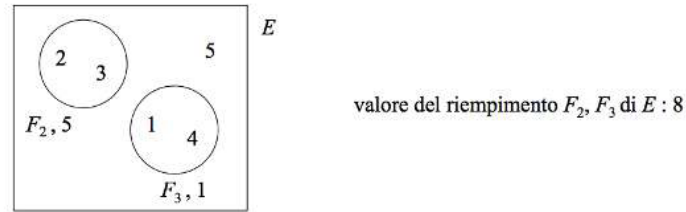
11.4.2.3 Problema di riempimento

La sottofamiglia D deve essere scelta in modo che ogni elemento di E appaia **al più** in un $F_j \in D$, cioè qualche elemento di E potrebbe non essere presente ma l'importante è che nessun elemento sia ripetuto.

Si dice che una sottofamiglia D con tale proprietà *riempie* l'insieme degli elementi di E .

Nei problemi di riempimento, il coefficiente c_j associato al sottoinsieme F_j rappresenta un **valore**: si vuole determinare il riempimento D di E mediante sottoinsiemi scelti da F che massimizza il valore totale del riempimento.

Un esempio di riempimento è il seguente:



Il vincolo **di riempimento** che deve essere soddisfatto è:

$$\sum_{j=1}^n a_{i,j} x_j \leq 1 \quad i = 1, \dots, m$$

Cioè per ogni $i \in E$ esiste al più un indice j per il quale è contemporaneamente $a_{i,j} = 1$ e $x_j = 1$.

Il modello di PLI del problema è:

$$\begin{cases} \min & \sum_{j=1}^n c_j x_j \\ & \sum_{j=1}^n a_{i,j} x_j \leq 1 \quad i = 1, \dots, m \\ & x_j \in \{0, 1\} \quad j = 1, \dots, n \end{cases}$$

11.4.2.4 Famiglia di algoritmi greedy per PC

- INIZIALIZZAZIONE:

- L'insieme S dei sottoinsiemi selezionati è inizializzato a $\emptyset$.

- PASSO ITERATIVO:

- Si seleziona uno dei sottoinsiemi F_j ancora da esaminare, secondo un certo criterio euristico: se il nuovo sottoinsieme «copre» almeno un elemento di E che non era coperto dalla soluzione parziale precedente, ossia se $F_j \not\subseteq F_S = \cup_{F_i \in S} F_i$, allora F_j è aggiunto ad S .
- L'algoritmo termina quando tutti i sottoinsiemi F_j sono stati esaminati, oppure quando tutti gli elementi di E sono «coperti» da S .

I criteri utilizzabili per la scelta del sottoinsieme F_j sono:

- **Costi non decrescenti:** si sceglie il sottoinsieme F_j con costo c_j più basso.

Con questo criterio gli elementi sono ordinati una sola volta all'inizio, quindi ad ogni iterazione si deve solo scorrere una lista ordinata (complessità $n \cdot \log(n)$).

- **Costi unitari non decrescenti:** Si calcola $\frac{c_j}{|F_j|}$ cioè il costo medio di copertura di un elemento di E mediante il sottoinsieme di F_j che tiene conto del numero di elementi che il sottoinsieme F_j copre.

Con questo criterio gli elementi sono ordinati una sola volta all'inizio, quindi ad ogni iterazione si deve solo scorrere una lista ordinata (complessità $n \cdot \log(n)$).

- **Costi unitari attualizzati non decrescenti:** Si calcola $\frac{c_j}{|F_j/F_S|}$ cioè il costo medio di copertura di un elemento di E , non ancora coperto dalla soluzione parziale S , mediante il sottoinsieme F_j . E' il criterio più efficiente ma più costoso, in quanto tiene conto del numero di elementi che il sottoinsieme F_j copre al netto di quelli già coperti dalla soluzione parziale S .

Il maggior costo è dato anche dal fatto che l'ordinamento deve essere ricalcolato ogni volta che F_S aumenta a seguito dell'inserimento di un elemento S .

Capitolo 12

Weighted vertex cover (WVC)

Supponiamo di avere un grafo orientato che rappresenta una rete sui cui archi fluiscono informazioni. Installando un apposito dispositivo nel nodo j è possibile controllare le informazioni che fluiscono sugli archi incidenti nel nodo j .

Il costo dell'installazione di tale dispositivo nel nodo j è $c_j > 0$.

Vogliamo determinare in quali nodi installare il dispositivo di controllo in modo che:

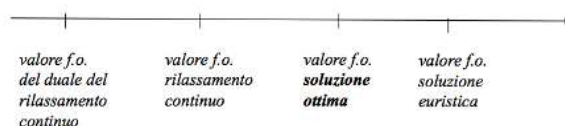
- Tutti gli archi siano controllati.
- Sia minimizzato il costo complessivo dei dispositivi da installare.

La **formulazione matematica** di un problema di *vertex cover* è:

$$\begin{cases} \min & \sum_{j=1}^n c_j x_j \\ \text{s.a.} & x_i + x_j \geq 1 \quad \forall (i, j) \in A \\ & x_j \in \{0, 1\} \quad \forall j \in N \end{cases}$$

12.1 Algoritmo greedy per WVC

Vogliamo costruire un algoritmo *greedy* per il problema di Weighted Vertex Cover, con **valutazione dell'approssimazione della soluzione calcolata**.



Dato che il problema di *Weighted Vertex Cover* è un problema di minimo, la stima dell'errore compiuto da un'euristica richiede la conoscenza di una *valutazione inferiore* a del valore ottimo della funzione obbiettivo.

Tale valutazione inferiore è ottenuta risolvendo il rilassamento continuo del problema. Infatti, la formulazione con variabili binarie è equivalente alla formulazione con variabili intere non-negative seguente:

$$\begin{cases} \min & \sum_{j=1}^n c_j x_j \\ \text{s.a.} & x_i + x_j \geq 1 \quad \forall (i, j) \in A \\ & x_j \geq 0 \text{ e intero} \quad \forall j \in N \end{cases} \quad (WVC)$$

Il corrispondente rilassamento continuo richiede che le variabili siano non-negative reali, anziché intere:

$$\begin{cases} \min & \sum_{j=1}^n c_j x_j \\ \text{s.a.} & x_i + x_j \geq 1 \quad \forall (i, j) \in A \\ & x_j \geq 0 \quad \forall j \in N \end{cases} \rightarrow \text{In notazione matriciale} \rightarrow \begin{cases} \min & \varphi = \underline{c}^T \underline{x} \\ \text{s.a.} & E^T \underline{x} \geq \underline{1} \\ & \underline{x} \geq \underline{0} \end{cases}$$

dove la matrice dei coefficienti dei vincoli funzionali, indicata con E^T è la trasposta della matrice di incidenza del grafo non orientato G .

Di conseguenza, il problema duale sarà:

$$\begin{cases} \max & \omega = \underline{1}^T \underline{y} \\ \text{s.a.} & E\underline{y} \leq \underline{c} \\ & \underline{y} \geq \underline{0} \end{cases} \longrightarrow \begin{cases} \max & \sum_{(i,j) \in A} y_{i,j} \\ \text{s.a.} & \sum_{(i,j) \in S(j)} y_{i,j} \leq c_j \quad \forall j \in N \\ & y_{i,j} \geq 0 \quad \forall (i,j) \in A \end{cases}$$

dove $S(j)$ è la **stella** del nodo j , ovvero l'insieme degli archi incidenti nel nodo j .

Fatte queste considerazioni, vogliamo costruire un algoritmo per WVC che calcola **contemporaneamente** le seguenti soluzioni ammissibili:

- Una soluzione ammissibile per il rilassamento continuo, che sia anche a variabili intere, cioè ammissibile per WVC.
- Una soluzione ammissibile per il duale del rilassamento continuo con cui valutare la bontà della soluzione primale ottenuta.

Se le due soluzioni ammissibili del primale e del duale soddisfano le condizioni di complementarità allora sono ottime per i rispettivi problemi. L'algoritmo *greedy* però richiede solamente il soddisfacimento delle condizioni di complementarità ai nodi, ammettendo il non soddisfacimento di tali condizioni agli archi, da cui segue che non è in grado di imporre il raggiungimento dell'ottimalità:

- CONDIZIONI DEGLI SCARTI COMPLEMENTARI AI NODI

$$x_j \left(c_j - \sum_{(i,j) \in S(j)} y_{i,j} \right) = 0 \quad \forall j \in N$$

- CONDIZIONI DEGLI SCARTI COMPLEMENTARI AGLI ARCHI

$$(x_i + x_j - 1) y_{i,j} = 0 \quad \forall (i,j) \in A$$

12.1.1 Algoritmo «Greedy-WVC»

L'idea che sta alla base dell'algoritmo **Greedy-WVC** è quella di costruire contemporaneamente sia una soluzione primale intera che una soluzione ammissibile per il duale del rilassamento continuo.

PSEUDOCODICE DELL'ALGORITMO

Procedure Greedy-WVC ($G, \underline{c}, S$):

begin

$S := \emptyset; Q := A;$

forall $(i, j) \in A$ **do** $y_{ij} := 0;$

repeat

$(i, j) := \text{Next}(Q); Q := Q \setminus \{(i, j)\};$

$y_{ij} := \min \left\{ c_i - \sum_{(i,h) \in S(i)} y_{ih}, c_j - \sum_{(j,h) \in S(j)} y_{jh} \right\};$

if $c_i = \sum_{(i,h) \in S(i)} y_{ih}$

then begin

$S := S \cup \{i\};$

forall $(i, h) \in A$ **do** $Q := Q \setminus \{(i, h)\};$

end

if $c_j = \sum_{(j,h) \in S(j)} y_{jh}$

then begin

$S := S \cup \{j\};$

forall $(j, h) \in A$ **do** $Q := Q \setminus \{(j, h)\};$

end

until $Q = \emptyset;$

end

ALGORITMO

• INIZIALIZZAZIONE:

- Si inizializza a $\emptyset$ l'insieme S degli archi scelti, cioè si inizializzano le variabili primali $x_i = 0$.
- L'insieme Q degli archi non ancora coperti dalla soluzione corrente S viene inizializzato all'insieme totale di tutti gli archi.
- Si inizializzano le variabili duali $y_{i,j} = 0$. Tale soluzione iniziale è ammissibile per duale perché i costi dei nodi sono positivi.

• PASSO ITERATIVO:

- Si seleziona un **qualsiasi arco** (i, j) non ancora coperto.
- Si assegna alla corrispondente variabile $y_{i,j}$ il massimo valore possibile che soddisfa le condizioni degli scarti complementari ai nodi i e j .

Per tale valore di $y_{i,j}$ è soddisfatta almeno una delle condizioni

$$c_i = \sum_{(i,h) \in S(i)} y_{i,h} \quad o \quad c_j = \sum_{(j,h) \in S(j)} y_{j,h}$$

Se vale la prima condizione, i viene aggiunto ad S e tutti gli archi incidenti in i sono eliminati da Q .

Se vale la seconda condizione, j viene aggiunto ad S e tutti gli archi incidenti in j sono eliminati da Q .

- La soluzione y costruita dall'algoritmo è duale ammissibile, in quanto lo è all'inizio (*dati i costi positivi*) e lo rimane fino alla fine (*dato che una volta che un arco i viene aggiunto ad S i suoi costi rimangono «congelati»*).
- L'algoritmo si arresta quando S copre tutti i nodi ($Q = \emptyset$) ed è quindi ammissibile per il primale.

12.1.1.1 Valutazione a priori dell'efficacia dell'algoritmo Greedy-WVC

L'osservazione fondamentale è che ad ogni passo dell'algoritmo si ha:

$$c(S) = \sum_{j \in S} c_j \leq 2 \sum_{(i,j) \in A} y_{i,j}$$

Infatti, quando il generico nodo j è stato inserito in S si aveva il *vincolo duale soddisfatto con l'uguaglianza*:

$$c_j = \sum_{(i,j) \in S(j)} y_{i,j}$$

e gli $y_{i,j}$ non sono più cambiati da quella iterazione, ma ciascun $y_{i,j}$ può contribuire ad al più due sommatorie (*quella corrispondente ad i e quella corrispondente ad j*).

Da queste considerazioni emerge che $R_{GreedyWVC} \leq 1$: la soluzione ottenuta dall'algoritmo *Greedy-WVC* può costare al più il doppio della soluzione ottima.

Quindi, a priori, possiamo affermare che questo algoritmo greedy compie al massimo un errore del 100%.

Capitolo 13

Smart Grids

13.1 Differenti approcci

La rete di distribuzione dell'energia elettrica ha subito diversi cambiamenti nel corso del tempo, attraversando principalmente due stadi:

- **RETE DI DISTRIBUZIONE PASSIVA:** viene consegnato al consumatore l'energia prodotta da grandi centrali, utilizzando la rete di trasmissione ad alta tensione. Gli elementi della rete tradizionale sono:
 - Nodo di alta tensione
 - Trasformatore a rapporto fisso
 - Nodi di consumo
 - Linee
- **RETE DI DISTRIBUZIONE ATTIVA:** la generazione dell'energia è «bidirezionale» e non solo nelle centrali, grazie anche a fonti di energia rinnovabili. Gli elementi della rete attiva sono:
 - Nodo di alta tensione.
 - Trasformatore a rapporto fisso o variabile.
 - Nodi di consumo, generazione e *storage*
 - Linee

Nella distribuzione di energia elettrica devono essere soddisfatte alcune condizioni:

- Requisiti di sicurezza in termini di vincoli della corrente presente sulle linee.
- Vincoli sulla tensione ai nodi.
- Bilancio tra la domanda e la produzione, in ogni momento.

La **schedulazione della produzione** viene fatta attualmente dal produttore il giorno precedente in base a :

- Previsioni del carico e del prezzo dell'elettricità.
- Previsioni meteo, utili per le fonti di energia «*non programmabili*» e quindi non controllabili. La non controllabilità di queste fonti è l'aspetto che più complica l'analisi dei bilanci di energia necessari da parte del provider di energia.

13.2 Progetto di ricerca

Si vuole costruire e testare una rete di prototipo, con una **generazione distribuita, elementi di storage e commutatori dipendenti dal carico**.

Questa rete è coadiuvata da un **software** per analisi *online* riguardo a come utilizzare le risorse utili per la regolazione, al fine di minimizzare il costo dell'azione di controllo, esercitata dal fornitore di energia, per soddisfare le condizioni richieste.

Due condizioni imprescindibili erano:

- Sviluppo di un software *licence-free*.
- Utilizzare una discretizzazione di 15 minuti, quindi è richiesto il calcolo di una soluzione in pochi minuti.

13.2.1 Modello non lineare

Per ogni generatore g ed ogni periodo h vengono calcolate le variazioni rispetto alla produzione schedulata $(P_{g,h}^0, Q_{g,h}^0)$ di **potenza attiva** $(\Delta P_{g,h}^+, \Delta P_{g,h}^-)$ e **reattiva** $(\Delta Q_{g,h}^+, \Delta Q_{g,h}^-)$, in modo da determinare se deve essere richiesta o immagazzinata energia nei dispositivi di storage.

Lo scopo del modello è quello di minimizzare:

- I costi totali di variazione di potenza attiva e reattiva in output.
- I costi di carica e scarica dei dispositivi di storage.
- Il valore delle perdite di potenza attiva.

Secondo questo modello, per ogni generatore g , in ogni periodo h , viene calcolata la potenza attiva in output:

$$P_{g,h} = \text{previsioni} + \text{variazioni}$$

$$\underline{P}_{g,h} \leq P_{g,h}^0 + \Delta P_{g,h}^+ - \Delta P_{g,h}^- \leq \bar{P}_{g,h}$$

con $\Delta P_{g,h}^+ \geq 0$ e $\Delta P_{g,h}^- \geq 0$, ma non entrambi positivi nella soluzione ottimale, dato che la funzione obbiettivo contiene un riferimento al costo in base ai termini:

$$c_{g,h}^{\Delta P^+} \Delta P_{g,h}^+ + c_{g,h}^{\Delta P^-} \Delta P_{g,h}^- \quad \text{con entrambi } c_{g,h}^{\Delta P} > 0$$

Allo stesso moto, per ogni generatore g , in ogni periodo h , viene calcolata la potenza reattiva in output:

$$Q_{g,h} = \text{previsioni} + \text{variazioni}$$

$$\underline{Q}_{g,h} \leq Q_{g,h}^0 + \Delta Q_{g,h}^+ - \Delta Q_{g,h}^- \leq \bar{Q}_{g,h}$$

con $\Delta Q_{g,h}^+ \geq 0$ e $\Delta Q_{g,h}^- \geq 0$, ma non entrambi positivi nella soluzione ottimale, dato che la funzione obbiettivo contiene un riferimento al costo in base ai termini:

$$ct_{g,h}^+ \Delta Q_{g,h}^+ + ct_{g,h}^- \Delta Q_{g,h}^- \quad \text{con entrambi } ct_{g,h} \geq 0$$

Valutata la potenza in output, è possibile fare un **bilancio di potenza attiva** sui nodi, per ogni periodo $h \in H$:

$$\sum_{g \in G_i} \left(P_{g,h}^0 + \Delta P_{g,h}^+ - \Delta P_{g,h}^- \right) = c_{i,h} + G_{i,h} V_{i,h}^2 + \sum_{(i,j) \in L} TA_{i,j,h}$$

dove:

- La parte sinistra è la somma delle potenze attive di tutti i generatori nel nodo i .
- $c_{i,h}$ è il *carico di potenza attiva* sul singolo nodo.
- $G_{i,h}$ è la *conduttanza di derivazione*.
- $\sum_{(i,j) \in L} TA_{i,j,h}$ è la somma dei flussi di potenza attiva dal nodo i .
- $V_{i,h}$ deve essere tra un valore minimo ed un massimo prestabiliti.

Valutiamo la stessa situazione, ma facendo un **bilancio di potenza reattiva** sui nodi:

$$\sum_{g \in G_i} \left(Q_{g,h}^0 + \Delta Q_{g,h}^+ - \Delta Q_{g,h}^- \right) = D_{i,h} + B_{i,h} V_{i,h}^2 + \sum_{(i,j) \in L} TR_{i,j,h}$$

dove:

- La parte sinistra è la somma delle potenze reattive di tutti i generatori nel nodo i .
- $D_{i,h}$ è il *carico di potenza reattiva* sul singolo nodo.
- $B_{i,h}$ è la *suscettanza di derivazione*.
- $\sum_{(i,j) \in L} TR_{i,j,h}$ è la somma dei flussi di potenza reattiva dal nodo i .

- $V_{i,h}$ deve essere tra un valore minimo ed un massimo prestabiliti.

Valutiamo, a questo punto, i **flussi di potenza attiva** sulle linee, per ogni periodo $h \in H$:

$$TA_{i,j,h} = \left(\frac{\cos \delta_{i,j,h}}{Z_{i,j,h}} + x_{i,j,h} \right) V_{i,h}^2 - \frac{\cos(\delta_{i,j,h} + \theta_{i,h} - \theta_{j,h})}{Z_{i,j,h}} V_{i,h} V_{j,h}$$

Diagram annotations:
 - $\cos \delta_{i,j,h}$: loss angle of series impedance of line (i,j)
 - $\theta_{i,h}$: phase angle of node i
 - $TA_{i,j,h}$: active power flow on line (i,j)
 - $Z_{i,j,h}$: series impedance
 - $x_{i,j,h}$: series impedance
 - $V_{i,h}^2$: transversal conductance of node i
 - $V_{i,h} V_{j,h}$: transversal conductance of node i

L'angolo di fase $\theta_{i,h}$ deve trovarsi tra un valore minimo ed uno massimo prestabiliti, rispettando che nel periodo 1 si abbia $\theta_{i,1} = 0$.

Per quanto riguarda la **potenza reattiva** vale la seguente relazione:

$$TR_{i,j,h} = \left(\frac{\sin \delta_{i,j,h}}{Z_{i,j,h}} - y_{i,j,h} \right) V_{i,h}^2 - \frac{\sin(\delta_{i,j,h} + \theta_{i,h} - \theta_{j,h})}{Z_{i,j,h}} V_{i,h} V_{j,h}$$

Diagram annotations:
 - $\sin \delta_{i,j,h}$: loss angle of series impedance of line (i,j)
 - $\theta_{i,h}$: phase angle of node i
 - $TR_{i,j,h}$: reactive power flow on line (i,j)
 - $Z_{i,j,h}$: series impedance
 - $y_{i,j,h}$: transversal susceptance of node i
 - $V_{i,h}^2$: transversal conductance of node i
 - $V_{i,h} V_{j,h}$: transversal conductance of node i

Per quanto riguarda i **trasformatori** ed il flusso di **potenza attiva** che li interessa si ha una distinzione legata all'utilizzo di:

- RAPPORTO FISSO DI TRASMISSIONE

$$TA_{i,j,h} = \left(\frac{Sn_{i,j,h} \cos \delta_{i,j,h}}{Vn_{i,j,h}^2 Z_{i,j,h}} + x_{i,j,h} \right) V_{i,h}^2 - \frac{Sn_{i,j,h} \cos(\delta_{i,j,h} + \theta_{i,h} - \theta_{j,h})}{Vn_{i,j,h} Vn_{j,i,h} Z_{i,j,h}} V_{i,h} V_{j,h}$$

Diagram annotations:
 - $Sn_{i,j,h}$: rated power of transformer
 - $Vn_{i,j,h}$: rated voltage at node i of transformer
 - $x_{i,j,h}$: series impedance of transformer
 - $TA_{i,j,h}$: active power flow on line (i,j)

- RAPPORTO VARIABILE

$$TA_{i,j,h} = \left(\frac{Sn_{i,j,h} \cos \delta_{i,j,h}}{TC_{i,j,h}^2 Z_{i,j,h}} + x_{i,j,h} \frac{Vn_{i,j,h}^2}{TC_{i,j,h}^2} \right) V_{i,h}^2 - \frac{Sn_{i,j,h} \cos(\delta_{i,j,h} + \theta_{i,h} - \theta_{j,h})}{TC_{i,j,h} Vn_{i,j,h} Vn_{j,i,h} Z_{i,j,h}} V_{i,h} V_{j,h}$$

Diagram annotations:
 - $TC_{i,j,h}$: voltage at node i of tap changer (i,j) $\in \mathcal{L}_1^T$
 - $TA_{i,j,h}$: active power flow on line (i,j)

Lo stesso vale per il flusso di **potenza reattiva**:

- RAPPORTO FISSO DI TRASMISSIONE

$$TR_{i,j,h} = \left(\frac{Sn_{i,j,h} \sin \delta_{i,j,h}}{Vn_{i,j,h}^2 Z_{i,j,h}} - y_{i,j,h} \right) V_{i,h}^2 - \frac{Sn_{i,j,h} \sin(\delta_{i,j,h} + \theta_{i,h} - \theta_{j,h})}{Z_{i,j,h} Vn_{i,j,h} Vn_{j,i,h}} V_{i,h} V_{j,h}$$

Diagram annotations:
 - $Sn_{i,j,h}$: rated power of transformer
 - $Vn_{i,j,h}$: rated voltage at node i of transformer
 - $y_{i,j,h}$: series impedance of transformer
 - $TR_{i,j,h}$: reactive power flow on line (i,j)

- RAPPORTO VARIABILE

$$TR_{i,j,h} = \left(\frac{Sn_{i,j,h} \sin \delta_{i,j,h}}{Z_{i,j,h} TC_{i,j,h}^2} - y_{i,j,h} \frac{Vn_{i,j,h}^2}{TC_{i,j,h}^2} \right) V_{i,h}^2 - \frac{Sn_{i,j,h} \sin(\delta_{i,j,h} + \theta_{i,h} - \theta_{j,h})}{Z_{i,j,h} TC_{i,j,h} Vn_{i,j,h} Vn_{j,i,h}} V_{i,h} V_{j,h}$$

Diagram annotations:
 - $TC_{i,j,h}$: voltage at node i of tap changer (i,j) $\in \mathcal{L}_1^T$
 - $TR_{i,j,h}$: reactive power flow on line (i,j)

Anche in questo caso, sia per potenza attiva che potenza reattiva, si ha un limite imposto a priori.

13.2.1.1 Flussi di corrente

La corrente che transita nella linea e sui trasformatori è anch'essa sottoposta al vincolo di sicurezza di avere un valore non «eccessivo» ed è calcolabile come:

$$TI_{i,j,h} = \frac{1}{\sqrt{3}V_{i,h}} \sqrt{TA_{i,j,h}^2 + TR_{i,j,h}^2} \leq \overline{TI}_{i,j,h}$$

13.2.1.2 Vincoli sulle batterie

Nelle linee attive abbiamo la presenza anche di dispositivi di *storage* quindi bisogna considerare anche i vincoli imposti dalle batterie. Essi sono più complici di quelli visti fino ad ora in quanto sono caratterizzati dall'essere lineari:

- **Potenza attiva**

$$E_{b,h} = E_{b,h-1} + Pb_{b,h}\Delta h \quad \underline{E}_{b,h} \leq E_{b,h} \leq \overline{E}_{b,h}, \quad \underline{Pb}_{b,h} \leq Pb_{b,h} \leq \overline{Pb}_{b,h}$$

Dove:

- $E_{b,0}$ è l'elettricità immagazzinata nel device b all'inizio del periodo 1.
- $Pb_{b,h}$ è lo scambio di potenza attiva tra batteria e rete.
- Δh è la lunghezza del periodo.

- **Perdite di potenza attiva durante la carica e la scarica**

$$Pb_{b,h} = \Delta Pb_{b,h}^+ - \Delta Pb_{b,h}^-$$

Dove:

- $\Delta Pb_{b,h}^+ > 0$ indica la scarica di potenza attiva nel periodo h . Questa operazione avviene con un rendimento $\eta_{b,h}^+$.
- $\Delta Pb_{b,h}^- > 0$ indica la carica di potenza attiva nel periodo h . Questa operazione avviene con un rendimento $\eta_{b,h}^-$.

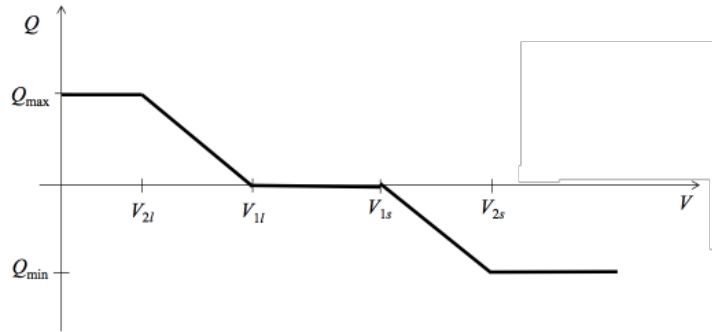
- **Potenza reattiva**

La potenza reattiva $Qb_{b,h}$ scambiata tra i dispositivi di storage e la rete in un periodo h deve essere compresa tra un valore minimo ed un valore massimo.

$$Qb_{b,h} = \Delta Qb_{b,h}^+ - \Delta Qb_{b,h}^-$$

13.2.1.3 Controllori locali

In ogni nodo, la relazione tra la potenza reattiva e la tensione deve essere rappresentata dal grafico seguente:



Si parla di **vincoli di controllo locale**. Questi sono caratterizzati da una difficoltà di gestire le discontinuità della derivata prima del vincolo, risolvibile solo complicando di molto il modello ed introducendo delle variabili binarie che permettano di capire in quale «pezzo» ci si trova.

La soluzione utilizzata è una approssimazione del modello con una funzione che non presenti discontinuità, cioè con delle **spline** che permettono di considerare delle terne di punti e garantire che nel punto di congiunzione delle funzioni ci sia la continuità della funzione e delle derivate.

13.2.1.4 Funzione obiettivo

La funzione obiettivo deve richiedere la **minimizzazione dei costi delle operazioni** cioè della somma di:

- Somma dei costi dovuti alle variazioni di produzione rispetto alla potenza attiva e reattiva prevista.
- Somma dei costi di scarica, carica e produzione di potenza reattiva.
- Somma dei costi delle perdite di potenza attiva.

13.2.1.5 Modello non lineare

Utilizziamo un modello di ottimizzazione espresso come di seguito:

- Utilizziamo come variabili del modello x_j , $1 \leq j \leq n$.
- Lavoriamo su un modello del tipo:

$$\begin{cases} \min & f(x_1, \dots, x_n) \\ \text{s.a.} & g_i(x_1, \dots, x_n) = 0 \quad 1 \leq i \leq m \\ & l_j \leq x_j \leq u_j \quad 1 \leq j \leq n \end{cases}$$

Esprimiamo i vincoli su x_j come uguaglianze, introducendo delle variabili «scarto» y_j e z_j :

$$[P] = \begin{cases} \min & f(x_1, \dots, x_n) \\ \text{s.a.} & g_i(x_1, \dots, x_n) = 0 \quad 1 \leq i \leq m \\ & x_j + y_j = u_j \\ & x_j - z_j = l_j \\ & y_j \geq 0, z_j \geq 0 \end{cases}$$

Introduciamo una nuova modifica, cioè creiamo un problema $[P^k]$ con gli stessi vincoli del problema $[P]$ ma con una modifica nella funzione obiettivo che sarà composta da quella di $[P]$ con, sommato, un termine logaritmico:

$$[P^k] = \begin{cases} \min & f(x_1, \dots, x_n) - \mu_k \sum_{j=1}^n (\log(y_j) + \log(z_j)) \\ \text{s.a.} & g_i(x_1, \dots, x_n) = 0 \quad 1 \leq i \leq m \\ & x_j + y_j = u_j \\ & x_j - z_j = l_j \\ & y_j \geq 0, z_j \geq 0 \end{cases}$$

L'aggiunta fatta alla funzione obiettivo è poco influente, in quanto per $\mu_k \rightarrow 0$ la soluzione di $[P^k]$ tende a quella di $[P]$.

La soluzione di questo problema è un punto stazionario della funzione lagrangiana $\mathcal{L}$:

$$\mathcal{L} = f(x_1, \dots, x_n) - \mu_k \sum_{j=1}^n (\log(y_j) + \log(z_j)) + \sum_{i=1}^m v_i g_i(x_1, \dots, x_n) + \sum_{j=1}^n r_j (x_j + y_j - u_j) + \sum_{j=1}^n s_j (l_j - x_j + z_j)$$

cioè la soluzione è:

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_j} = 0 \\ \frac{\partial \mathcal{L}}{\partial v_i} = 0 \\ \frac{\partial \mathcal{L}}{\partial y_j} = 0 \\ \frac{\partial \mathcal{L}}{\partial z_j} = 0 \\ \frac{\partial \mathcal{L}}{\partial r_j} = 0 \\ \frac{\partial \mathcal{L}}{\partial s_j} = 0 \end{cases} \quad \begin{cases} \frac{\partial f(x_1 \dots x_j \dots x_n)}{\partial x_j} + \sum_{i=1}^m v_i \cdot \frac{\partial g_i(x_1 \dots x_j \dots x_n)}{\partial x_j} + r_j - s_j = 0 \\ g_i(x_1 \dots x_j \dots x_n) = 0 \\ -\mu_k \frac{1}{y_j} + r_j = 0 \\ -\mu_k \frac{1}{z_j} + s_j = 0 \\ x_j + y_j - u_j = 0 \\ l_j - x_j + z_j = 0 \end{cases}$$

Introduciamo la seguente notazione:

- Y matrice diagonale, con $Y_{j,j} = y_j$.
- Z matrice diagonale, con $Z_{j,j} = z_j$.
- $\Delta f(\underline{x}) = \left[\frac{\partial f}{\partial x_j} \right]$

$$\bullet J(\underline{x}) = \Delta \underline{g}(\underline{x}) = \begin{bmatrix} \frac{\partial g_1}{\partial x_1} & \dots & \frac{\partial g_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial g_m}{\partial x_1} & \dots & \frac{\partial g_m}{\partial x_n} \end{bmatrix}$$

Allora, il sistema può essere riscritto anche come:

$$\left[\begin{array}{l} \nabla_{\underline{x}} \mathcal{L} = \underline{0} \\ \nabla_{\underline{y}} \mathcal{L} = \underline{0} \\ \mathbf{Y} \nabla_{\underline{r}} \mathcal{L} = \underline{0} \\ \mathbf{Z} \nabla_{\underline{s}} \mathcal{L} = \underline{0} \\ \nabla_{\underline{t}} \mathcal{L} = \underline{0} \\ \nabla_{\underline{z}} \mathcal{L} = \underline{0} \end{array} \right] \left[\begin{array}{l} \nabla f(\underline{x}) + \mathbf{v}^T J(\underline{x}) + \underline{r} - \underline{s} = \underline{0} \\ \underline{g}(\underline{x}) = \underline{0} \\ \mathbf{Y} \underline{r} = \underline{e} \mu_k \\ \mathbf{Z} \underline{s} = \underline{e} \mu_k \\ \underline{x} + \underline{y} - \underline{u} = \underline{0} \\ \underline{t} - \underline{x} + \underline{z} = \underline{0} \end{array} \right]$$

L'idea per la risoluzione del sistema è quella di utilizzare un'approssimazione di Newton:

- Si parte da una soluzione iniziale $\underline{x}^0$.
- Si costruisce un $\underline{x}_{k+1} = \underline{x}_k + \underline{d}_k$.
- Si calcola $\Delta F(\underline{x}_k) \underline{d}_k = -F(\underline{x}_k)$

SCHEMA DELL'ALGORITMO

```

assegna mu(1)
for k=1 to KMax
    risolvi il problema P^k
    se il test di convergenza è soddisfatto , stop
    aggiorna mu(k)

```

13.3 Conclusioni

- Le performance sono molto legate alla precisione dello stimatore dello stato.
- Il controllo centralizzato è generalmente migliore rispetto alla soluzione con controllori locali.